

A $(2+\varepsilon)$ -Approximation Algorithm for Metric k -Median

Vincent Cohen-Addad*

Fabrizio Grandoni[†]

Euiwoong Lee[‡]

Chris Schwiegelshohn[§]

Ola Svensson[¶]

Abstract

In the classical NP-hard (metric) k -median problem, we are given a set of n clients and centers with metric distances between them, along with an integer parameter $k \geq 1$. The objective is to select a subset of k *open* centers that minimizes the total distance from each client to its closest open center.

In their seminal work, Jain, Mahdian, Markakis, Saberi, and Vazirani presented the Greedy algorithm for facility location, which implies a 2-approximation algorithm for k -median that opens k centers in *expectation*. Since then, substantial research has aimed at narrowing the gap between their algorithm and the best achievable approximation by an algorithm guaranteed to open *exactly* k centers, as required in the k -median problem. During the last decade, all improvements have been achieved by leveraging their algorithm (or a small improvement thereof), followed by a second step called bi-point rounding, which inherently adds an additional factor to the approximation guarantee.

Our main result closes this gap: for any $\varepsilon > 0$, we present a $(2 + \varepsilon)$ -approximation algorithm for the k -median problem, improving the previous best-known approximation factor of 2.613. Our approach builds on a combination of two key algorithms. First, we present a non-trivial modification of the Greedy algorithm that operates with only $O(\log n / \varepsilon^2)$ adaptive phases. Through a novel walk-between-solutions approach, this enables us to construct a $(2 + \varepsilon)$ -approximation algorithm for k -median that consistently opens at most $k + O(\log n / \varepsilon^2)$ centers: via known results, this already implies a $(2 + \varepsilon)$ -approximation algorithm that runs in quasi-polynomial time. Second, we develop a novel $(2 + \varepsilon)$ -approximation algorithm tailored for stable instances, where removing any center from an optimal solution increases the cost by at least an $\Omega(\varepsilon^3 / \log n)$ fraction. Achieving this involves several ideas, including a sampling approach inspired by the k -means++ algorithm and a reduction to submodular optimization subject to a partition matroid. This allows us to convert the previous result into a polynomial time algorithm that opens exactly k centers while maintaining the $(2 + \varepsilon)$ -approximation guarantee.

*Google Research, France.

[†]IDSIA, USI-SUPSI, Switzerland. Supported in part by the SNF Grants 200021-200731 and 200021-236706.

[‡]University of Michigan, Ann Arbor, MI, USA. Supported in part by NSF 2236669.

[§]Aarhus University, Denmark. Supported in part by the Independent Research Fund Denmark (DFF) under grant No 1051-00106B and a Google Research Award.

[¶]EPFL, Switzerland. Supported in part by the Swiss State Secretariat for Education, Research and Innovation (SERI) under contract number MB22.00054.

Contents

1	Introduction	1
1.1	Our Results	2
1.2	Related Work	3
1.3	Preliminaries	3
2	Technical Overview	5
2.1	$(2 + \varepsilon)$ -Approximation with $O_\varepsilon(\log n)$ Extra Centers	5
2.1.1	Logarithmic Adaptivity	6
2.1.2	Merging Two Solutions	7
2.2	$(2 + \varepsilon)$ -Approximation for $O_\varepsilon(\frac{1}{\log n})$ -Stable Instances	9
2.2.1	Local Optima and Stability	9
2.2.2	Identifying the Non-Pure Clusters	10
3	$(2 + \varepsilon)$-Approximation with $O(\log n / \varepsilon^2)$ Extra Centers	12
3.1	Review of the Greedy Algorithm	13
3.2	LOGADAPTIVE	14
3.3	Analysis of LOGADAPTIVE	17
3.3.1	Approximation Guarantee.	17
3.3.2	Dual Feasibility	18
3.4	Walking Between Two Solutions: Setup	22
3.5	MERGESOLUTIONS	26
3.5.1	Making Parameters Identical	27
3.5.2	Maximize Common Prefix of H_p and H'_p	28
3.5.3	Removing Extra Facilities	31
4	$(2 + \varepsilon)$-Approximation for $(\frac{\zeta}{\log n})$-Stable Instances	32
4.1	Properties of a Locally Optimal Solution	35
4.1.1	Pure Clusters	35
4.2	Cheap Clusters and D -Sample Process to Hit Expensive Clusters	37
4.3	Identifying Balls of Expensive Clusters	39
4.3.1	Dummy Centers and Mixed Solutions M_O and M_D	41
4.4	Removal of Expensive Centers of S_0	42
4.4.1	Consistently Assigning Clients in Mixed Solutions and the sets $\mathcal{U}$ and $\mathcal{R}$	43
4.5	Removal of Cheap Centers in S_0	46
4.5.1	Mixed Solutions after the Removal of $\tilde{\mathcal{U}}$	49
4.6	Finding S^* via Submodular Optimization	52
4.6.1	Proof of Lemma 4.15	55
4.7	Putting Everything Together – Proof of Theorem 1.3	59
A	Analysis of Robust LOGADAPTIVE	60
A.1	Approximation Guarantee.	61
A.2	Dual Feasibility	62

1 Introduction

Given a collection D of n points, a clustering is a partition of the points into (typically) disjoint subsets (the *clusters*) such that points in the same cluster are *similar* and points in different clusters are *dissimilar*. One of the classical and best-studied clustering problems is k -median (see also Table 1). Here, in addition to the set D (points are also called *clients* in this framework), we are given an integer parameter $k > 0$ and a set F^1 (the *facilities* or *centers*), with metric distances $d(a, b)$ for $a, b \in D \cup F$. Our goal is to select k centers $S \subseteq F$ (*open* centers) so as to minimize the sum of the distances from each client to the closest open center. In other words, the goal is to minimize the following objective function

$$\text{cost}(S) := \sum_{j \in D} \min_{i \in S} d(i, j).$$

The k -clustering induced by S is obtained by considering, for each $c \in S$, the subset of points that have c as a closest center in S (breaking ties arbitrarily). We use opt_k to denote the optimal cost, and simply use opt when k is clear from the context.

For most clustering variants, it is NP-hard to compute an optimal clustering and metric k -median is no exception. For this reason a lot of effort was devoted to the design of (polynomial-time) approximation algorithms for this problem, i.e., algorithms that compute a feasible solution S whose cost is provably within some factor $\alpha > 1$ of opt (the *approximation factor* or *approximation ratio/guarantee*).

Most of the best-known approximation algorithms for k -median, including the current-best one [CGLS23, GPST23], are based on the following high-level approach (see Section 1.2 for alternative approaches in the literature). One considers the associated facility location problem, where instead of k we are given a (uniform) facility opening cost f . In this problem a feasible solution S can contain an arbitrary number of centers/facilities (rather than k), however in the objective function, besides the *connection cost* $\text{cost}(S)$, one has to pay also the *opening cost* $f|S|$. Then one uses a Lagrangian Multiplier Preserving (LMP) α_{LMP} -approximation for the problem (formal definition in Section 1.3) to construct a *bi-point* solution for k -median, i.e., a convex combination of a solution S_1 (opening fewer than k centers) and a solution S_2 (opening more than k centers), that opens exactly k centers in a fractional sense. In particular, sampling one of the two solutions according to the respective coefficients would give a solution of expected cost $\alpha_{\text{LMP}} \cdot \text{opt}$ that opens k centers *in expectation*. Finally, a bi-point rounding algorithm is used to derive a feasible k -median solution, while losing another factor α_{BPR} in the approximation. The product $\alpha_{\text{LMP}} \cdot \alpha_{\text{BPR}}$ gives the overall approximation factor. We remark that the state of the art leaves a significant gap between the approximation factor for a solution with k centers *in expectation* and a true k -median solution.

Reference	Approximation factor	Technique
[Bar96]	$O(\log n \log \log n)$	tree embeddings
[CCGG98]	$O(\log k \log \log k)$	tree embeddings
[CGTS99]	6.667	dependent LP rounding
[JV01]	6	LMP + bi-point rounding
[JMS02, JMM ⁺ 03]	4	LMP + bi-point rounding
[AGK ⁺ 04]	$3 + \varepsilon$	local search
[LS16]	2.733	LMP + bi-point rounding
[BPR ⁺ 17]	2.675	LMP + bi-point rounding
[CGLS23, GPST23]	2.613	LMP + bi-point rounding
Theorem 1.1	$2 + \varepsilon$	LMP interpolation + stable algorithm

Table 1: Progression of approximation factors for k -median. Dependencies on ε are omitted due to numerical overestimation, except for [AGK⁺04] and the final result.

In more detail, for the LMP stage, [JV01] gave a 3-approximation algorithm which was subsequently improved to 2 by [JMS02, JMM⁺03], and then to $2 - \eta$ for some $\eta > 2.25 \cdot 10^{-7}$ by [CGLS23].

¹Sometimes in the literature one assumes $F = D$ or $D \subseteq F$. We consider the more general case and make no such assumption.

[JV01] also presented a factor 2 algorithm for the bi-point rounding stage. [LS16] subsequently improved this to a factor $\frac{1+\sqrt{3}}{2} \approx 1.366$ when allowing to use $k + O(1)$ many centers², and also gave a polynomial-time algorithm for obtaining a proper k -median clustering from a $k + O(1)$ clustering essentially without any loss in the approximation ratio. This approach was further refined by [BPR⁺17] who gave a 1.337-approximate bi-point rounding algorithm and also showed that no LP-based rounding algorithm can do better than $\frac{1+\sqrt{2}}{2}$. The state of the art for rounding methods is due to [GPST23] who gave a 1.306-approximation algorithm, while showing that no algorithm can do better than $\sqrt{\phi} \approx 1.272$, where $\phi = \frac{1+\sqrt{5}}{2}$ is the golden ratio.

Unfortunately, there is an inherent limitation to this approach. No approximation algorithm for the LMP stage can do better than $1 + 2/e$ [JMS02, JMM⁺03], and using the lower bound for bi-point rounding algorithms by [GPST23], the best possible analysis cannot yield an approximation factor better than $(1 + 2/e)\sqrt{\phi} \approx 2.207$. This leaves a significant gap to the inapproximability lower bound of $(1 + 2/e) \approx 1.735$ [JMS02, JMM⁺03], and even to the best-known approximation algorithm that opens k -centers in expectation [CGLS23], which achieves an approximation guarantee of $2 - \eta$ for some small $\eta > 0$.

1.1 Our Results

Our main result is stated as follows. (As commonly done, we use “with high probability” to mean that an event occurs with probability at least $1 - 1/\text{poly}(n)$.)

Theorem 1.1. *For every $\varepsilon > 0$, there is a randomized polynomial-time algorithm for k -median that returns a solution with cost at most $(2 + \varepsilon)\text{opt}$ with high probability.*

The algorithm in our main theorem combines two novel algorithms. First, we address the limitations of the bi-point rounding method by introducing a non-trivial variant of the classic Greedy algorithm [JMS02, JMM⁺03]. The Greedy algorithm can be highly *adaptive* in that tiny changes to the opening cost f may lead to vastly different solutions S and S' , where $|S| < k$ and $|S'| > k$, forcing previous bi-point rounding approaches to incur a large additional factor in the approximation guarantee. We resolve this high-adaptivity by presenting a low-adaptivity version, which makes adaptive decisions in only $O(\log(n)/\varepsilon^2)$ phases. This modification intuitively renders the algorithm less adaptive than the original, where each choice depends heavily on previous ones. We then introduce a novel walk-between-solutions algorithm that enables us to return a solution with at most $k + O(\log(n)/\varepsilon^2)$ centers, with no additional loss in the approximation factor (up to $1 + \varepsilon$). Specifically, we achieve the following result.

Theorem 1.2. *For every constant $\varepsilon \in (0, 1/6)$, there is a polynomial-time algorithm for k -median that returns a solution containing at most $k + O(\log n / \varepsilon^2)$ many centers and of cost at most $(2 + \varepsilon)\text{opt}$.*

Theorem 1.2 is proved in Section 3, and an overview of the main ideas is given in Section 2.1.

Combining this result with the [LS16] reduction from k -median pseudo-solutions to true k -median solutions already yields a $(2 + \varepsilon)$ -approximation in quasi-polynomial time. In order to improve this to a polynomial time algorithm, we use a completely different algorithm. We say that an instance is β -stable if $\text{opt}_{k-1} \geq (1 + \beta) \cdot \text{opt}_k$. Our result is as follows.

Theorem 1.3. *For any two constants $\varepsilon, \zeta > 0$, there exists a randomized polynomial-time algorithm that, given a $(\zeta / \log n)$ -stable k -median instance, returns a solution of cost at most $(2 + \varepsilon)\text{opt}$ with high probability.*

The algorithm for stable instances is inspired by a recent fixed-parameter-tractable approach that essentially identifies, by enumeration, small-radius balls guaranteed to contain the optimal centers and then solves the problem by maximizing a submodular function subject to a partition matroid constraint. However, there are two major challenges: first, the running time of [CGK⁺19] is not polynomial even when $k = O(\log n)$; and second, because k may be large (potentially polynomial in n), we need to identify almost all clusters of the optimal solution before proceeding with any guessing/enumeration steps. We address these challenges through several novel ideas, including a sampling approach based on the k -means++ algorithm. The proof of Theorem 1.3 is presented in Section 4, with an overview of the techniques provided in Section 2.2.

²Solutions opening slightly more than k centers are sometimes called *pseudo-solutions* in the literature.

Let us see why Theorems 1.2 and 1.3 combine to yield Theorem 1.1. We compute a few feasible solutions. Let Δ be the surplus number of centers used in Theorem 1.2. Let $k' = \max\{1, k - \Delta\}$. One solution is obtained by computing the optimum solution with one center for $k' = 1$, and otherwise by running the algorithm from Theorem 1.2 with the number of centers being $k' = k - \Delta$. If $\text{opt}_{k'} \leq \text{opt}_k \cdot (1 + \varepsilon)$, this is a good enough approximation. Then we run the algorithm from Theorem 1.3 for every $k'' \in (k', k]$. If the solution with k' centers computed before is not good enough, then there exists some $k'' \in (k', k]$ such that $\text{opt}_{k''-1} \geq (1 + \beta) \cdot \text{opt}_{k''}$ with $\beta \in \Omega(\varepsilon/\Delta) = \Omega(\varepsilon^3/\log n)$ and $\text{opt}_{k''} \leq (1 + \varepsilon) \cdot \text{opt}_k$. In this case, Theorem 1.3 yields the desired result for a proper choice of the parameters. Outputting the best of all computed solutions yields Theorem 1.1.

For Euclidean spaces, the set of candidate centers is infinite. By a standard reduction, we can find a polynomial-sized set $F \subset \mathbb{R}^d$ in polynomial time such that the cost of any constant factor approximate solution is approximated by a set of centers obtained from F , see for example Lemma 5.3 of [KSS10]. Thus we also obtain a $(2 + \varepsilon)$ -approximation in Euclidean spaces, improving on 2.406 [CEMN22].

Finally, let us remark that Theorem 1.1 essentially closes the gap between the approximation guarantee of the best-known algorithm that opens k centers in expectation [CGLS23] and algorithms that consistently open at most k centers. However, a major open problem remains: obtaining a tight approximation result for k -median, as it is only known that it is hard to approximate the problem within a factor less than $(1 + 2/e)$ [JMS02]. Building on this work, a natural next step would be to resolve this question or to significantly improve the approximation guarantee, even in the setting where k centers are opened in expectation.

1.2 Related Work

We already reviewed the LMP-based approximation algorithms earlier. But there are several other important results for k -median and closely related problems which we cover here.

There also exist other approaches for designing approximation algorithms for metric k -median, most notably local search [AGK⁺04, CGH⁺22, GT08, KMN⁺04], combinatorial algorithms [MP03], metric embeddings [Bar98, Bar96, CCGG98], or other LP-based rounding approaches [ARS03, CGTS02, CL12]. There has also been a lot of work for the related problems k -means and k -center, which minimize the sum of squared distances and the maximum distance, respectively. Specifically for k -center, [Gon85, HS86] gave a 2-approximation which is optimal unless $P \neq NP$. k -means admits a $(9 + \varepsilon)$ -approximation in general metrics [ANSW20]. In addition, both k -median and k -means have been studied in d -dimensional Euclidean spaces, where a PTAS is possible when either d or k are fixed [ARR98, KSS10, FRS19, CFS19]. If neither d nor k are fixed, the approximation factors for Euclidean k -median and k -means are also better than their general metric counterparts [ANSW20, GOR⁺22], with the best known approximation ratio being 5.913 for k -means and, prior to our work, being 2.406 for k -median by [CEMN22]. Both in high dimensional Euclidean spaces as well as general metrics, all of these aforementioned problems are APX hard [ACKS15, CCSL21, CK19, GI03, JMS02, LSW17].

LMP-based approaches are also very relevant for the related facility location problem. Mahdian et al. [MYZ06] showed that an LMP $(1 + 2/e)$ -approximation yields a $(\gamma_{GK} \approx 1.463)$ -approximation for UFL, which matches the hardness result of Guha and Khuller [GK99] where γ_{GK} is the solution of $\gamma = 1 + 2e^{-\gamma}$. Nevertheless, most improvements do not rely on an LMP approach. Following a long line of research [BA10, CG05, CS03, JMM⁺03, JMS02, KPR00, MYZ06, STA97], Li presented an algorithm with the currently best known approximation ratio of roughly 1.488 [Li13].

1.3 Preliminaries

We are given a set of n clients D and set of m potential centers or facilities F . Given a set $S \subseteq F$, we use $d(j, S) = \min_{i \in S} d(j, i)$, where d is the underlying distance function. $\tilde{O}(x)$ suppresses polylog(x) factors, and $O_\varepsilon(x)$ suppresses polynomial factors in ε . We use $[a]^+ = \max(a, 0)$. We already defined the k -median problem. The uncapacitated facility location problem (UFL) with uniform opening costs is given a non-negative number f and asks to find a set $S \subseteq F$ minimizing

$$\text{cost}_{\text{UFL}}(S) = \sum_{j \in D} d(j, S) + f \cdot |S|.$$

We refer to $\sum_{j \in D} d(j, S)$ as the *connection cost* and $f \cdot |S|$ as the *opening cost*. Let $x_{i,j}$ denote in the indicator variable for serving client j with facility i and let y_i be the indicator variable for opening

facility i . Relaxing these to continuous variables in $[0, 1]$, the LP relaxation to k -median and UFL are given by the following equations:

$$\begin{array}{ll}
\min \sum_{i \in F, j \in D} d(i, j) x_{i,j} & (LP_{km}) \\
\text{s.t.} \sum_{i \in F} x_{i,j} \geq 1 & \forall j \in D \\
y_i - x_{i,j} \geq 0 & \forall j \in D, i \in F \\
\sum_{i \in F} y_i \leq k & \\
x, y \geq 0 &
\end{array}
\quad
\begin{array}{ll}
\min \sum_{i \in F, j \in D} d(i, j) x_{i,j} + f \cdot \sum_{i \in F} y_i & (LP_{UFL}(f)) \\
\text{s.t.} \sum_{i \in F} x_{i,j} \geq 1 & \forall j \in D \\
y_i - x_{i,j} \geq 0 & \forall j \in D, i \in F \\
x, y \geq 0 &
\end{array}$$

The constraint $\sum_{i \in F} x_{i,j} \geq 1$ ensures that every client is served by some facility and the constraint $y_i - x_{i,j} \geq 0$ ensures that a client j can only be served by facility i if i is open. Moreover, the dual for the facility location LP relaxation is given by the equations:

$$\begin{array}{ll}
\max \sum_{j \in D} \alpha_j & (DP_{UFL}(f)) \\
\text{s.t.} \sum_{j \in D} [\alpha_j - d(i, j)]^+ \leq f & \forall i \in F \\
\alpha \geq 0 &
\end{array}$$

The usual interpretation of the α_j is that they are the budgets for each client, accounting for both connecting the client to a facility, as well as opening a facility. The opening contribution dual variables $\beta_{i,j}$ obtained from the $y_i - x_{i,j} \geq 0$ constraints, as well as the dual constraints $\alpha_j - \beta_{i,j} \leq d(i, j)$ are removed here, as we can always set $\beta_{i,j} = [\alpha_j - d(i, j)]^+$. Recall that $\text{opt} = \text{opt}_k$ is the cost of an optimal k -median clustering. We use opt_{LP} and $\text{opt}_{LP}(f)$ to denote the cost of an optimal solution to LP_{km} and $LP_{UFL}(f)$, resp. Trivially $\text{opt} \geq \text{opt}_{LP}$. Since any feasible solution for LP_{km} is also feasible for $LP_{UFL}(f)$, one has $\text{opt}_{LP} \geq \text{opt}_{LP}(f) - kf$. We will use the fact that, by weak duality, any feasible solution to $DP_{UFL}(f)$ induces a lower bound on $\text{opt}_{LP}(f)$. In particular, for any feasible solution α to $DP_{UFL}(f)$, we have $\text{opt} \geq \text{opt}_{LP}(f) - kf \geq \sum_{j \in D} \alpha_j - kf$.

We say that an algorithm is a Lagrangian Multiplier Preserving (LMP) α -approximation algorithm for UFL if it returns a solution S with

$$\text{cost}(S) \leq \alpha \cdot (\text{opt}_{LP}(f) - f|S|).$$

Finally, we will make the assumption that the distances are integers in $[1, n^3/\epsilon]$. The justification is given by the following standard reduction.

Lemma 1.4. *For any constants $\epsilon > 0$ and $\alpha > 1$, given a polynomial-time α -approximation algorithm for k -median on instances with distances in $\{1, \dots, n^3/\epsilon\}$, there exists a polynomial-time $\alpha(1 + O(\epsilon))$ -approximation algorithm for k -median on general instances.*

Proof. Consider any input instance $(D \cup F, d)$ of k -median. Assume that $n := |D|$ is large enough w.r.t. α , otherwise the problem can be solved by brute force in polynomial time. We guess $M := \max_{j \in D} d(j, \text{OPT})$, where OPT is some optimal k -median solution, by trying all the $n \cdot m$ possibilities. Next, for each $(i, j) \in F \times D$, define $d'(j, i) = d'(i, j) = \max\{1, \lceil \frac{d(i, j)}{M} \rceil \frac{n}{\epsilon}\}$ if $d(i, j) \leq M$. Set all the remaining $d'(a, b)$, $a \neq b$, to $\frac{n^3}{\epsilon}$. Finally, replace the $d'(i, j)$'s with the corresponding metric closure. We run the given α -approximation algorithm on the k -median instance $(D \cup F, d')$, hence obtaining a solution S . It is easy to check that S is a good enough approximation for the input instance. $\square$

2 Technical Overview

In this section, we give an overview of our result. Section 2.1 introduces ideas behind Theorem 1.2 that achieves a $(2 + \varepsilon)$ -approximation while opening $k + O(\log n / \varepsilon^2)$ centers, and Section 2.2 discusses Theorem 1.3 for stable instances.

2.1 $(2 + \varepsilon)$ -Approximation with $O_\varepsilon(\log n)$ Extra Centers

Let us motivate our high-level plan by revisiting the previous framework. The previous best approximation algorithms [LS13, BPR⁺17, GPST23, CGLS23] for k -median start by the following initialization step: starting from two LMP approximate solutions for UFL $S, S' \subseteq F$ with the respective facility costs f, f' such that $|S| < k, |S'| > k^3$ and f possibly much larger than f' , the algorithm uses binary search to obtain two new solutions $(S, f), (S', f')$ such that $|S| < k, |S'| > k$ and $|f - f'| \leq 2^{-n}$. This step can be already viewed as a *merging* step that takes two solutions whose number of facilities sandwich k and makes them (specifically the facility cost) closer. Then, the natural question is *can we merge S and S' into the (almost) same set of open facilities so that both open (almost) exactly k facilities?*

One naive implementation of this idea might be the following (from now on, let us assume $f = f'$). Just like binary search, suppose that we find an LMP approximate solution S'' which is *strictly between* S and S' in some formal sense. If $|S''| > k$, we will let $S' \leftarrow S''$ and otherwise $S \leftarrow S''$. Then the invariant $|S| < k, |S'| > k$ is maintained while the two solutions got *strictly closer*. So we can continue this process until they become the same!

Of course, the main question is whether one can find such S'' given S and S' . Just like S and S' , S'' must be an outcome of the LMP approximation algorithm, or at least close to such an outcome in some formal sense. Since *being a valid outcome of an algorithm* is a highly complex property, especially when the algorithm has a large number of *adaptive* steps whose action depends on the previous actions, this task remains challenging for a general LMP approximation algorithm.

However, there is an example of success for Jain and Vazirani's primal-dual [JV01] algorithm, which gives a LMP 3-approximation. At a high level, this algorithm consists of two phases, where the first phase yields a set T of tentative open facilities, and the second phase is allowed to choose an arbitrary $S \subseteq T$ that forms a maximal independent set in an auxiliary graph, so there is a freedom in choosing the independent set. Ahmadian et al. [ANSW20] used this freedom to give a walking procedure that finds a maximal independent set in the auxiliary graph of cardinality exactly k , which led to an improved approximation ratio for Euclidean k -means/ k -median and metric k -means.

However, the Greedy algorithm [JMS02, JMM⁺03] with dual-fitting analysis, which guarantees a better 2-LMP approximation, is more adaptive; in fact, it can be a completely deterministic algorithm without leaving any freedom. Therefore, in order to implement our idea, it would be better to find a low-adaptive version of the Greedy algorithm, where the algorithm runs in a small $(O_\varepsilon(\log n))$ in our case) number of *phases*, and each phase presents some freedom of choices for the algorithm.

The Greedy Algorithm. Let us recall the Greedy algorithm [JMS02, JMM⁺03]. The algorithm maintains the set A of *active* clients and the set S of open facilities. Initially, all clients are active, i.e., $A = D$, and no facilities are opened, i.e., $S = \emptyset$. Additionally, the algorithm maintains dual variables $(\alpha_j)_{j \in D}$, initialized to 0.

While there is at least one active client, the algorithm uniformly increases the α -values of the active clients until there is a facility i whose "bids" are sufficient to open it (or an active client becomes "close" to an already opened facility). The bids from a client j to a facility i depend on whether j is active or not. If $j \in A$, then a portion of its budget α_j , specifically $d(i, j)$, is allocated to cover the connection cost, and the remainder, $\alpha_j - d(i, j)$, is used to bid towards the facility's opening cost $\hat{f} := 2f$. For a client $j \notin A$, which is already connected to an open facility at a distance $d(j, S)$, the bid towards i is the difference in distance, i.e., $d(j, S) - d(i, j)$ (or 0 if i is farther). Finally, the algorithm marks any client j with $\alpha_j \geq d(j, S)$ as inactive, ensuring that at any point in time, active clients j satisfy $\alpha_j \leq d(j, S)$.

In order to prove that the algorithm yields an LMP 2-approximation, let $(\alpha_j^*)_{j \in D}$ be the final α values and S^* be the set of opened facilities. The design of the algorithm guarantees that the α^*

³Throughout this overview, we assume no solution opens exactly k centers, because we are done if this happens.

values pay for both the opening costs of the facilities in S^* and the connection costs of the clients, leading to $\sum_{j \in D} \alpha_j^* = \sum_{j \in D} d(j, S^*) + \hat{f} \cdot |S^*|$.

The more interesting part of the analysis is to show that $\alpha^*/2$ is a dual feasible solution for the dual facility location LP with facility cost f , namely $\sum_{j \in D} [\alpha_j^* - 2d(i, j)]^+ \leq \hat{f}$ for every facility i . The dual feasibility follows from two key properties that, at any point in time, (a) $\alpha_j \leq d(j, S)$ for every $j \in A$ and (b) for every $i \in F$, no facility is overbid:

$$\sum_{j \in A} [\alpha_j - d(i, j)]^+ + \sum_{j \in D \setminus A} [d(j, S) - d(j, i)]^+ \leq \hat{f}. \quad (1)$$

2.1.1 Logarithmic Adaptivity

As stated, the GREEDY is a highly adaptive algorithm; for instance, consider the case when two facilities i and i' both become paid for at the same time θ . Whether we open i or i' can dramatically change the bids to future facilities and completely change the trace of the algorithm.

To reduce such high adaptivity, we limit the number of time steps where such decisions are made. Specifically, our high-level goal is to only consider the time steps of the form $(1 + \varepsilon^2)^0, (1 + \varepsilon^2)^1, (1 + \varepsilon^2)^2, \dots$. Then, as the minimum nonzero distance is 1 and the maximum distance is $\text{poly}(n)$, the total number of time steps that we need to consider is $O(\log n / \varepsilon^2)$, and this is what we refer to as $O(\log n)$ adaptivity.

The main difficulty in implementing this idea is to ensure (1). Consider the situation where all active clients have α value $\theta = (1 + \varepsilon^2)^t$. If their α values jump from θ to $(1 + \varepsilon^2)\theta$, it is possible that a previously underbid facility becomes strictly overbid.

We fix this issue in two steps. The first easy fix is to *scale down the metric* by a factor of $(1 - \delta)$ where $\delta = O(\varepsilon)$; that is, we consider a facility i paid for if

$$\sum_{j \in A} [\alpha_j - (1 - \delta)d(i, j)]^+ + (1 - \delta) \sum_{j \in D \setminus A} [d(j, S) - d(j, i)]^+ \geq \hat{f}. \quad (2)$$

At the cost of losing a factor $1/(1 - \delta)$ in the approximation guarantee, this has the following benefit. If i is not paid according to (2) when all active clients have $\alpha_j = \theta$, then even when α_j 's are increased to $(1 + \varepsilon^2)\theta$, for a client j with $d(j, i) \geq \varepsilon\theta$, its contribution $[(1 + \varepsilon^2)^{t+1} - d(i, j)]^+$ to (1) is at most its previous contribution $[(1 + \varepsilon^2)^t - (1 - \delta)d(i, j)]^+$ to (2), which makes (1) unlikely to be violated.

Then our second fix deals with active clients close to co-located with i . We do so by allowing the clients j in the ball $B(i, \varepsilon\theta)$ to increase their bids for i up to $\min((1 + \varepsilon^2)\theta, (1 - \delta)d(j, S))$, so that if i could not be opened even with increased bids from nearby clients, then increasing active clients' α values from θ to $(1 + \varepsilon^2)\theta$ should not make i overbid. These increased bids bring other considerations (e.g., ensuring dual feasibility of other facilities), leading to the following definition of *openable* facilities, which can be efficiently checked by a linear program.

Definition 1. A facility i is *openable with respect to the algorithm's state* (α, S, A, θ) if there are bids $(\tau_j)_{j \in A}$ of active clients that satisfy the following conditions.

- Only nearby clients are increased:

$$\tau_j = \alpha_j \quad \text{for every } j \in A - B(i, \varepsilon\theta).$$

- Nearby clients are only increased slightly:

$$\alpha_j \leq \tau_j \leq \min\{(1 - \delta)d(j, S), (1 + \varepsilon^2)\theta\} \quad \text{for every } j \in A \cap B(i, \varepsilon\theta).$$

- Facility i is paid for:

$$\sum_{j \in A} [\tau_j - (1 - \delta)d(i, j)]^+ + (1 - \delta) \sum_{j \in D \setminus A} [d(j, S) - d(i, j)]^+ \geq \hat{f}.$$

- Dual feasibility: for every facility i_0 and $\ell \in A$,

$$\sum_{j \in A} [\tau_j - d(i_0, j)]^+ + \sum_{j \in D \setminus A} [\tau_\ell - 2d(j, i_0) - d(\ell, i_0)]^+ \leq \hat{f}.$$

Algorithm 1 (LOGADAPTIVE).

Initialization: Set $\hat{f} = 2f$, $S = \emptyset$, $A = D$, $\theta = 1$ and $\alpha_j = \theta$ for every $j \in A = D$.

Repeat the following while $A \neq \emptyset$:

1. While there is an unopened facility i that is openable (chosen arbitrarily if there are many),

Calculate $(\tau_j)_{j \in A}$ so that i is openable with bids $(\tau_j)_{j \in A}$. Open i (i.e., add it to S), let $\alpha_j \leftarrow \tau_j$ for every $j \in A$ (note that only the values of the clients in $A \cap B(i, \varepsilon\theta)$ change), mark all clients $j \in A$ such that $\alpha_j \geq (1 - \delta)d(j, S)$ as served (i.e., remove them from A).

2. Perform the following step and proceed to the next phase.

For every remaining $j \in A$, let $\alpha_j \leftarrow \min((1 + \varepsilon^2)\theta, (1 - \delta)d(j, S))$, remove it from A when $\alpha_j = (1 - \delta)d(j, S)$. Update $\theta = (1 + \varepsilon^2)\theta$.

Given this definition of openability, Section 3.2 presents our modified Greedy algorithm with $O(\log n)$ phases, LOGADAPTIVE, which is restated here as Algorithm 1. Here a *phase* is defined to be the run of the algorithm for a fixed value of $\theta = 1, (1 + \varepsilon^2), (1 + \varepsilon^2)^2, \dots$, which is broken down to two *stages* according to the description of Algorithm 1. Note that at least in the beginning, stage 1 allows opening of any openable facility, providing the desired freedom to walk between two solutions within the phase. While stage 1 still presents some adaptivity as an opening of a facility might impact the openability of another, it is sufficient for the merging procedure described in Section 2.1.2.

For the analysis, in Section 3.3, we prove that LOGADAPTIVE yields an LMP $(2 + O(\varepsilon))$ -approximate solution for UFL. It is a nontrivial extension of the dual-fitting analysis of the Greedy algorithm for our definition of openability and design of the algorithm.

2.1.2 Merging Two Solutions

We describe our final algorithm MERGESOLUTIONS that maintains two LMP approximate solutions whose number of facilities sandwich k and gradually merges them until one of them opens almost exactly k facilities.

Setup. Based on the discussion about LOGADAPTIVE, we will describe a solution $\mathcal{H}$ as $(H_1, \dots, H_L)$ with $L = O(\log n / \varepsilon^2)$, where H_i is a *valid execution* of stage 1 of phase i (i.e., when $\theta = (1 + \varepsilon^2)^{i-1}$), which is formally represented as a sequence of opened facilities. H_i 's validity also requires *maximality*, meaning that no openable facility remains in the phase. In order to facilitate the merging, given a phase, we extend the notion of a valid execution to include the following actions.

- We allow opening a *free facility*, at most three times per phase. A free facility $\tilde{i}$ is based on a regular facility $i \in F$, and when it is created, we can choose the distance parameter $u(\tilde{i}) = d(\tilde{i}, i)$. All other distances $d(\tilde{i}, x)$ are defined to be $u(\tilde{i}) + d(i, x)$.
- We slightly relax the notion of openability so that one can open a facility i by paying $\hat{f} - n\eta$ instead of $\hat{f}$ for parameter $\eta := 2^{-n}$ (in the third bullet of Definition 1).

Call $\mathcal{H} = (H_1, \dots, H_L)$ a *solution of η -valid sequences* if H_i is a valid execution (i.e., an η -valid sequence) of the i th phase in the above relaxed notion given $H_1, \dots, H_{i-1}$. We extend the analysis of LOGADAPTIVE in Appendix A to show that such $\mathcal{H}$ in the above sense still yields an LMP $(2 + O(\varepsilon + n\eta))$ -approximation, if we do not count the opening cost of the free facilities (while using the connections they provide). Therefore, a solution of η -valid sequences that opens exactly k regular facilities, and at most 3 free facilities per phase, is a $(2 + O(\varepsilon + n\eta))$ -approximation to k -median that opens at most $O(L) = O(\log n / \varepsilon^2)$ extra facilities.

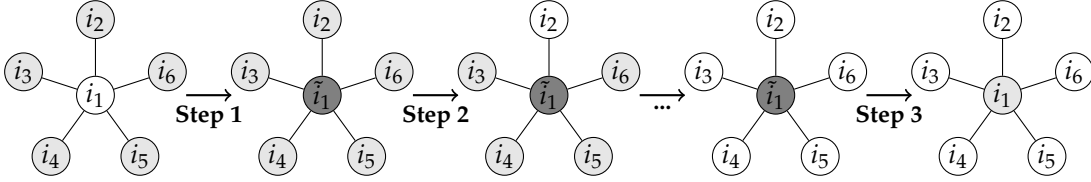


Figure 1: How we walk from H'_p to H_p . Dark grey color denotes a free facility and light grey color denotes regular open facilities.

Initially, by the standard binary search on the facility cost and running LOGADAPTIVE, we start with two solutions $\mathcal{H} = (H_1, \dots, H_L)$ and $\mathcal{H}' = (H'_1, \dots, H'_L)$ with facility costs f and f' such that $|f - f'| \leq \eta$ and their numbers of opened facilities sandwich k . (We will associate parameters (f, u) for $\mathcal{H}$ and (f', u') for $\mathcal{H}'$. Also, note that every facility in $\mathcal{H}$ (resp. $\mathcal{H}'$) has cost f (resp. f').) Throughout the algorithm, we will maintain the invariants that (1) $\mathcal{H}$ and $\mathcal{H}'$ open the same set of free facilities (so that u and u' are defined on the same set), (2) their number of opened regular facilities sandwich k , and (3) $(f, u) = (f', u')$ except for one parameter f or $u(\tilde{i})$ for one free $\tilde{i}$, whose values in the two solutions differ by at most η . Call such a parameter the *difference parameter*.

For each $i = 1, \dots, L$, we will outline a procedure that, given that the prefixes $(H_1, \dots, H_{p-1})$ and $(H'_1, \dots, H'_{p-1})$ are identical, modifies the suffixes $(H_p, \dots, H_L)$ and $(H'_p, \dots, H'_L)$ such that H_p and H'_p become identical while satisfying the above invariants, unless we already find a solution that opens exactly k regular facilities. This will allow us to gradually merge the solution until one of them opens exactly k regular facilities (and at most $O(L)$ free facilities).

How to walk in each phase. Finally, we focus on phase p and introduce ideas behind ensuring $H_p = H'_p$. Our first step is to change the difference parameter value of $\mathcal{H}'$ to that of $\mathcal{H}$; formally, we let $\mathcal{H}'' = \text{COMPLETESOLUTION}_{(f,u)}(H'_1, \dots, H'_p)$, which is the solution resulting from running LOGADAPTIVE from a given partial solution $(H'_1, \dots, H'_p)$ with parameters (f, u) instead of (f', u') . Such a change of the difference parameter is the reason for introducing the $n\eta$ slack for η -valid sequences; for instance, if H_i was the exact execution with parameters (f', u') , our Claim 2 shows that it is still η -valid with respect to (f, u) .

If $\mathcal{H}''$ opens k regular facilities we are done. Also, if the number of open regular facilities of $\mathcal{H}''$ and $\mathcal{H}'$ sandwich k , then we already made progress as they now coincide on the p th phase while still having one difference parameter. Therefore, we only need to handle the case where $\mathcal{H}''$ and $\mathcal{H}$ sandwich k , where the difference parameters already became identical but H_p and H'_p are different. Let us rename $\mathcal{H}''$ to $\mathcal{H}'$.

Now our goal is to gradually change H'_p to H_p (i.e., walk from H'_p to H_p). Instead of describing the full procedure, which is outlined in Section 3.5, we illustrate the procedure on a particular example. Suppose that $H_p = \langle i_1 \rangle$, $H'_p = \langle i_2, \dots, i_6 \rangle$ where one can open at most one between i_1 and i_j for any $j \in \{2, \dots, 6\}$; this constraint can be expressed as a star graph with i_1 as the center vertex and $i_2, \dots, i_6$ are only connected to i_1 , and a valid solution (i.e., η -valid sequence) here means a maximal independent set. (Suppose that this is the only constraint on openings, so H_p and H'_p can be thought of as sets instead of sequences.)

In this example, H_p and H'_p are the only two maximal independent sets, so it is impossible to smoothly walk from H'_p to H_p . Here is where the notion of a free facility becomes useful because we can assume that having a free facility helps *maximality* while not hurting *independence*. Then, from H'_p , by (Step 1) adding a free copy of i_1 called $\tilde{i}_1$ with $u(\tilde{i}_1) = 0$, (Step 2) removing $i_2, \dots, i_6$ one-by-one, and (Step 3) converting $\tilde{i}_1$ to a regular i_1 , we obtain a sequence of maximal independent sets where a consecutive pair of independent sets differ by at most one facility. Figure 1 illustrates this process.

For each intermediate H'_p , we consider its completion (i.e., $\text{COMPLETESOLUTION}_{(f,u)}(H'_1, \dots, H'_p)$) and see how many regular facilities it opens. Since the completions from the initial H'_p and H_p sandwich k , there must be a consecutive pair in the above sequence of independent sets (say H_p^a and H_p^b) that sandwich k , and they differ by only one facility i_q (say H_p^a contains i_q while H_p^b does not). For future execution of the algorithm, H_p^a and H_p^b are the same as the solutions that have the free copy $\tilde{i}_q$

(instead of i_q) with the $u(\tilde{i}_q)$ value 0 (for H_p^a) and ∞ (or sufficiently large number, for H_p^b). Therefore, by performing binary search on the $u(\tilde{i}_q)$, just like we do on the facility cost f , we yield two solutions that are now identical up to phase p except the η difference on the only one different parameter $u(\tilde{i}_q)$, which archives our goal of merging two solutions at phase p !

In general, the fact that H_p and H'_p are not just sets but sequences of opened facilities requires more technical consideration. However, the full procedure is still an extension of this idea, trying to increase the length of the common prefix between H_p and H'_p by walking from H'_p to H_p by one coordinate at a time, until two consecutive solutions are sandwiching k .

2.2 $(2 + \varepsilon)$ -Approximation for $O_\varepsilon(\frac{1}{\log n})$ -Stable Instances

The other component of our algorithm is an improved approximation to k -median assuming that the instance has a weak version of a structural property known as stability, initially formulated by [ORSS12]. Recall that an instance is β -stable if $\text{opt}_{k-1} \geq (1 + \beta)\text{opt}_k$. For any β -stable instance, [ABS10, CS17] showed how to obtain a PTAS for the problem in time $n^{\beta^{-1}\varepsilon^{-O(1)}}$ and [ABS10] left as an open question as whether one could obtain a PTAS with running time $f(\beta, \varepsilon)n^{O_\varepsilon(1)}$ for any computable function f for general metrics (they show it is possible for Euclidean metrics). We would indeed like to have an algorithm running in time $2^{\text{poly}(\beta, \varepsilon)}n^{O_\varepsilon(1)}$ to solve our $(\varepsilon/\log n)$ -stable instances.

Unfortunately, since any k -median instance is $\Omega(1/k)$ -stable, obtaining a PTAS with the above running time would violate Gap-ETH [CGK⁺19]: The k -median problem cannot be approximated better than $1 + 2/e$ in time $f(k, \varepsilon)n^{O_\varepsilon(1)}$ assuming Gap-ETH.

The saving grace here is that we are not after a $(1 + \varepsilon)$ -approximation algorithm but we can use any $(2 + O(\varepsilon))$ -approximation with the above running time. Since a connection between β -stable and FPT algorithm exists (as explained in the above discussion), it is tempting to make the journey from FPT algorithms back to β -stable instances and use the state-of-the-art approximation algorithm of [CGK⁺19] to solve β -stable instances. Unfortunately the algorithm presented in [CGK⁺19] runs in time $(\log n)^{\tilde{O}(k/\varepsilon)}n^{O(1)}$ and so importing the techniques directly would result in a running time of the form $(\log n)^{\tilde{O}((\beta/\varepsilon)^{-1})}n^{O(1)}$ and not to a polynomial running time for our setting. We must thus go beyond the state-of-the-art techniques for FPT and β -stable k -median instances.

2.2.1 Local Optima and Stability

One particularly important tool when dealing with stable solutions are properties of local optima. The local search procedure starts with any feasible solution S , and searches for an alternative solution S' such that $|S \triangle S'| = 2$ (i.e., S and S' differ by one center) and $\text{cost}(S') < \text{cost}(S)$. If such an S' exists, S is replaced by S' and the process is iterated. At the end of the procedure we say that S is locally optimal. The running time of this procedure is polynomial, where we exploit the assumption that distances are polynomially bounded integers (by Lemma 1.4).

We first introduce a bit of notation. Suppose we are given a locally optimal solution S . We say that a cluster C^* of the optimal solution OPT is *pure* with respect to S , if there exists a cluster C' induced by S such that both clusters agree on all but $O(\varepsilon) \cdot \min(|C^*|, |C'|)$ of the points. We begin by examining structural properties for β -stable solutions, summarized by in Section 4.1.

- For any given constant-factor approximation S , at most $O_\varepsilon(\beta^{-1})$ many clusters are not pure, see Lemma 4.3.
- In any locally optimal solution S , we have

$$\sum_{p \in C^*, C^* \text{ pure}} d(p, S) \leq \sum_{p \in C^*, C^* \text{ pure}} d(p, \text{OPT}) + O(\varepsilon) \cdot \text{opt},$$

that is the cost of the clients in pure clusters is well approximated.

More precisely, in the second bullet we show that all the points in each pure cluster C^* with center c^* can be assigned to the *same* center $t(c^*)$ in S (i.e., the center c' of C') with a moderate increase of their cost (see Lemma 4.4).

The first property relies on the fact that for every non-pure cluster, S must merge at least two clusters of the optimum. If S is a constant-factor approximation, this cannot happen more than $O_\epsilon(\beta^{-1})$ many times before the approximation guarantee no longer holds. The second property is implied by the local optimality of S . Specifically, swapping out the center $c' \in S$ serving C' for the center c^* of the pure cluster C^* cannot change the cost by much as these two clusters agree on most of their points.

In a nutshell, a locally optimal solution S approximates all but $O_\epsilon(\beta^{-1})$ many clusters of the optimal solution very well. In what follows, we will describe how we may extend S to a $(2 + \epsilon)$ -approximation overall.

2.2.2 Identifying the Non-Pure Clusters

To find a good approximation for the non-pure clusters, we would like to identify these clusters. By the above discussion, we may assume that we have a locally optimal solution S for which all but $O_\epsilon(\beta^{-1}) = O_\epsilon(\log n)$ many clusters are well approximated. Immediate ideas, such as determining the badly approximated clusters by brute force, will not improve over the quasi-polynomial running time. Instead, we look towards fixed parameter tractable (FPT) approaches, parameterized by the number of centers. In fact, we would like an FPT approximation algorithm *parameterized by the number of non-pure clusters*. Starting point is an FPT algorithm based on submodular maximization, initially due to [CGK⁺19], which we first review. However, recall that the running time of this algorithm is $(\log k)^{\tilde{O}(k\epsilon^{-1})}$ and so will not be enough for our purpose. We thus introduce new ideas to speed things up.

Review of the FPT approximation algorithm of [CGK⁺19]. Suppose c^* is a center of cluster C^* in an optimal solution OPT. [CGK⁺19] introduce the notion of a *leader* client $\ell \in C^*$ which minimizes the distance to c^* . Using coresets to first reduce the number of clients to $\tilde{O}(k \log n / \epsilon^2)$, the leaders of all clusters, as well as the distances $d(\ell, c^*)$ can be identified in FPT-time, i.e. in time $(\log n)^{\tilde{O}(k\epsilon^{-1})} \cdot n^{O(1)}$. A solution S obtained by simply selecting an arbitrary facility within distance $d(\ell, c^*)$ from ℓ yields a 3-approximation. [CGK⁺19] improve upon this by showing that for any two candidate sets of centers S' and Λ

$$\text{cost}(\Lambda) - \text{cost}(\Lambda \cup S')$$

is a monotone submodular function.

The first part which consists in guessing the leaders is the computational bottleneck since optimizing a monotone submodular function can be done in polynomial time.

We next describe how we deal with the clusters that are not well approximated. Recall that we first compute a locally optimal solution S . By the properties of local optima, S is already a sufficiently good solution for the pure clusters. For the remaining $O_\epsilon(\log n)$ non-pure clusters, we only have coarser approximation bounds.

Leader Identification in Polynomial Time via D^2 -Sampling – Sections 4.2 and 4.3. Our next high-level goal is to find leaders of the non-pure clusters and guess their (approximate) distances to the respective optimum centers in time $2^{\beta^{-1}\epsilon^{-O(1)}} n^{O(1)}$ for β -stable instances.

[CGK⁺19] identify leaders in time $\binom{k \log n}{k}$ by sparsifying the instance via coresets and then determining the distance of leaders to the optimal solution in time $(\log n)^k$. Our overall approach significantly speeds up these two steps by way of distance sampling similar to k -means++ [AV07].

To identify the leaders faster, we lift the stringent requirement that the leader of cluster C^* is the client closest to the center c^* in the optimal solution. Instead, we consider any client among the $\epsilon \cdot |C^*|$ closest points to c^* as a valid leader with $\text{avg}_{C^*, \text{OPT}}$ being the maximum distance of any leader of C^* to c^* . We later show that this only induces a minor cost increase in the total approximation bound.

The second relaxation is that we only aim to target clusters that are sufficiently expensive in our locally stable solution S . More precisely, a non-pure cluster C^* with center c^* is *basic-cheap* if (a) the total cost of its leaders in S is at most $\epsilon^{O(1)} \cdot \beta \cdot \text{opt} = \frac{\epsilon^{O(1)}}{\log n} \cdot \text{opt}$ or (b) $d(c^*, S) \leq (1 + \epsilon) \text{avg}_{C^*, \text{OPT}} + \frac{\epsilon}{|C^*|} \cdot \sum_{p \in C^*} (d(p, c^*) + d(p, S))$. Then we can show that S is already an almost 2-approximation for

these clusters modulo some additive slack, namely

$$\sum_{p \in C^*, C^* \text{ basic-cheap}} d(p, S) \leq 2 \sum_{p \in C^*, C^* \text{ basic-cheap}} d(p, \text{OPT}) + O(\varepsilon \cdot \text{opt}).$$

More precisely, similarly to the case of pure clusters, we can show that all the points of a basic-cheap cluster can be reassigned to the center $t(c^*) \in S$ which is closest to c^* with an increase of the cost of the above type (see Lemma 4.6). Indeed, by triangle inequality $d(p, S) \leq d(p, c^*) + d(c^*, S)$ for any p . If C^* is basic-cheap by condition (b), one directly obtains the desired upper bound on $d(c^*, S)$. Otherwise, i.e. if C^* is basic-cheap by condition (a), we use the fact that the number of basic-cheap clusters (which is at most the number of non-pure clusters) is $\frac{\log n}{\varepsilon^{O(1)}}$ so the sum of their costs in S is $O(\varepsilon \cdot \text{opt})$. This essentially allows us to focus only on non-pure and non-basic-cheap clusters.

We use these two relaxations as follows. Let W be a sample of clients picked independently and proportionately to $\frac{d(p, S)}{\text{cost}(S)}$. With probability $\varepsilon^{O(1)} / \log n$, we are picking a leader from a given non-pure, non-basic cheap cluster: in this case we say that C^* is hit by the sample. Thus, when picking $O_\varepsilon(\log n)$ samples, we have picked a leader from all but a small fraction of non-pure, non-basic cheap clusters. Thereafter, we can show that with good probability, we can assign all the points of a non-pure non-basic-cheap non-hit cluster C^* , with center c^* , to the center $t(c^*) \in S$ which is closest to c^* with a small cost (see Lemma 4.5 in Section 4.2). We next call *expensive* the non-pure non-basic-cheap clusters which are hit, and *cheap* the non-pure clusters which are either basic-cheap or non-basic-cheap and non-hit. Thus the clusters are partitioned into pure, cheap and expensive ones. Since we have sampled $O_\varepsilon(\log n)$ clients, we can enumerate all the subsets in polynomial time and focus on a subset that contains exactly one leader of each expensive cluster.

Using a constant number of potential guesses per leader, we then show how to get a sufficiently good approximation of the distance ρ from each leader to its closest optimum center, see Lemma 4.8 in Section 4.3. Intuitively, the cost of the leader in solution S is a rough approximation of ρ , so we can interpolate around it.

Existence of a Good Solution. We now use W from last section to identify a modified solution from S that is $(2 + O(\varepsilon))$ -approximate. Let $\hat{\text{OPT}}$ denote the set of centers of expensive clusters. The cost of all the other optimal clusters, i.e., the pure and cheap ones, is within a factor roughly 2 from the optimum. We thus wish to add to S the optimal centers of $\hat{\text{OPT}}$ and remove an equally sized set $S_0 \subseteq S$. An important constraint on S_0 is that it contains no center $t(c^*)$, where c^* is the center of an optimal cluster which is pure or cheap. Intuitively, such centers $t(c^*) \in S$ guarantee that the cost of points in pure and cheap clusters is within a factor roughly 2 from the optimum. More precisely, in Lemma 4.9 we show that such an S_0 exists so that $M_O := S - S_0 \cup \hat{\text{OPT}}$ is a $(2 + O(\varepsilon))$ -approximation: If a cluster of OPT is pure or cheap, there is one center $t(c^*)$ in $S - S_0$ serving their clients up to a $2 + O(\varepsilon)$ factor; for the remaining ones, we would have their optimum center (in $\hat{\text{OPT}}$).

So our goal from now on is to approximate the solution M_O , where $|S_0| = |\hat{\text{OPT}}| = O_\varepsilon(\log n)$, and this final goal has two key components: (1) removing some existing centers from S and (2) adding the centers from $\hat{\text{OPT}}$. The issue is that stated as such this seems as hard as solving the k -median problem. We first address (1), and we will address (2) by making use of the leaders we have correctly guessed for the centers in $\hat{\text{OPT}}$ at the previous steps.

Center Removal. Let us first try to address (1). Remember we start with solution S , and the leader and radii guesses from the previous steps. The key issue with (1) is that the clients in some of the clusters we would like to remove (i.e., served by S_0 in solution S) are actually served by a center in $\hat{\text{OPT}}$, but we do not know which are these centers yet! We will thus make use of some approximate loose bound on their cost in solution $S - S_0 \cup \hat{\text{OPT}}$ by introducing *dummy* centers Λ . For each leader ℓ and guessed distance/radii ρ , we will introduce a dummy center at distance ρ from ℓ and at distance $d(p, \ell) + \rho$ from any other vertex p .

If we promise to open a center in the ball of radii ρ around ℓ – which solution $S - S_0 \cup \hat{\text{OPT}}$ does – then we will guarantee that there will be a center not further away than the dummy centers in the final solution. We now work with this solution $S \cup \Lambda$ which provides a worst-case fall back on where the centers in $\hat{\text{OPT}}$ could be. We can show that $M_D := S \cup \Lambda - S_0$ is a $(3 + O(\varepsilon))$ -approximate solution.

Equipped with this we proceed to our 3-step center removal. We know that we have at most $|S_0| = O_\varepsilon(\log n)$ centers we need to remove. Our first procedure (see Section 4.4) aims at removing the centers of clusters of high cost in S_0 . Start with solution $S \cup \Lambda$. For $O_\varepsilon(\log n)$ times, we sample a center $c \in S$ proportionally to the cost of its cluster in solution $S \cup \Lambda$, remove c from S with probability $1/2$, and repeat on the resulting solution. Let $\mathcal{Q}$ be the centers removed from S . Then with probability at least $2^{-O_\varepsilon(\log n)}$ the procedure succeeds by either obtaining $\mathcal{Q} = S_0$ or ensuring that the total cost of the clusters of $S_0 - \mathcal{Q}$ in the solution is at most $O(\varepsilon \cdot \text{opt})$. This indeed follows from the fact that, if we only removed centers $\mathcal{Q} \subseteq S_0$ up to a certain step, the solution is never more than a $(3 + O(\varepsilon))$ -approximation (since $S \cup \Lambda - S_0$ is a $(3 + O(\varepsilon))$ -approximation) and so the probability of sampling a new center of $S_0 - \mathcal{Q}$ is $\Omega(\varepsilon)$, unless the total cost of the clusters of $S_0 - \mathcal{Q}$ is small. Therefore, repeating the process $n^{O_\varepsilon(1)}$ times, we have that one of the runs succeeds with high probability.

Thus, after this step, we have removed a set $\mathcal{Q}$ of centers of S_0 and the total cost of the clusters of $S_0 - \mathcal{Q}$ is $O(\varepsilon \cdot \text{opt})$. Following this procedure, we only have to remove inexpensive clusters. While here it is not possible to identify the correct ones, making a mistake is not too costly. For any $c \in S_0 - \mathcal{Q}$, we can now think of its whole cluster as being assigned entirely to a unique center in $S - S_0 \cup \text{OPT}$ since by triangle inequality this would only increase its cost by $O(\varepsilon \cdot \text{opt})$ in total. We can thus think of the remaining centers $S_0 - \mathcal{Q}$ as either being assigned to OPT (we call this set $\mathcal{R}$), or to a center in $S - S_0$ (we call this set $\mathcal{U}$). Suppose we know how many centers fall into these two types, a figure we determine by trying all possibilities. The next step would be ideally to remove $\mathcal{U}$. We instead remove a *proxy* $\tilde{\mathcal{U}}$ of $\mathcal{U}$, $|\tilde{\mathcal{U}}| = |\mathcal{U}|$ (see Section 4.5). A natural starting point to find a center for reassignment is to do so by greedily removing the center c for which the reassignment cost is minimized. However, we must be careful. First, the reassignment costs are dynamic, that is when removing a center, it affects reassignment costs of subsequently removed ones. Second, if a center c was used to bound the reassignment cost of an deleted center, we cannot delete it later in order to prevent the error from adding up. We design a procedure that produces a $\tilde{\mathcal{U}}$ satisfying all these constraints (see Lemma 4.13). We remark that $\tilde{\mathcal{U}} \cap \mathcal{R} = \emptyset$, hence removing $\mathcal{R}$ is still a valid option in the later stages.

Thus, for the purpose of analysis (the algorithm does not know $\mathcal{R}$), we obtain the solutions $M'_D = S - (\mathcal{Q} \cup \tilde{\mathcal{U}} \cup \mathcal{R}) \cup \Lambda$ and $M'_O = S - (\mathcal{Q} \cup \tilde{\mathcal{U}} \cup \mathcal{R}) \cup \text{OPT}$. Moreover, the costs satisfy $\text{cost}(M'_O) + \text{cost}(M'_D) \leq (4 + O(\varepsilon))\text{OPT}$, which is the same as summed up costs of M_O and M_D , see Lemma 4.12 and Claim 7.

Improvement via Submodular Maximization. We can now use submodular maximization techniques (specifically algorithms to maximize a submodular function over a partition matroid constraint) in a similar but more general setting as in the work of [CGK⁺19]. In more detail, we wish to find the best $|\text{OPT}|$ centers X to open (instead of the dummy centers Λ), one at the prescribed distance from each leader, to improve the cost of M'_D as much as possible. This is almost identical to the work of [CGK⁺19] except that we also need to close $|\mathcal{R}|$ other centers $\tilde{\mathcal{R}}$ in $S - \mathcal{Q} - \tilde{\mathcal{U}}$. Ideally, we would like to open OPT and close $\mathcal{R}$. We show that selecting a good enough pair $(X, \tilde{\mathcal{R}})$ can be reduced to a submodular maximization problem under a partition matroid constraint. In particular, we carefully define our submodular function so that its value, for a given subset X of centers, is calculated by optimally selecting the set $\tilde{\mathcal{R}}$ to close (for the choice X). Then we can use state-of-the-art algorithms to optimize this and *simultaneously* approximate the centers OPT to open and the set $\mathcal{R}$ to close.

3 $(2 + \varepsilon)$ -Approximation with $O(\log n / \varepsilon^2)$ Extra Centers

In this section, we present a polynomial-time algorithm that opens $k + O(\log n / \varepsilon^2)$ many centers and returns a solution of cost at most $(2 + \varepsilon)\text{opt}$. We restate the theorem for convenience of the reader.

Theorem 1.2. *For every constant $\varepsilon \in (0, 1/6)$, there is a polynomial-time algorithm for k -median that returns a solution containing at most $k + O(\log n / \varepsilon^2)$ many centers and of cost at most $(2 + \varepsilon)\text{opt}$.*

Our algorithm is a non-trivial extension of the Greedy algorithm with dual fitting analysis [JMS02, JMM⁺03], next called GREEDY, so as to guarantee that we only open $k + O(\log n / \varepsilon^2)$ centers instead of only doing so in expectation. We first recall the basics of GREEDY in Section 3.1. Section 3.2

presents a modified algorithm LOGADAPTIVE that runs in $O(\log n/\varepsilon^2)$ adaptive steps (that we call *phases*) and Section 3.3 analyzes the latter algorithm.

Finally, building on additional setups in Section 3.4, Section 3.5 presents our final algorithm MERGESOLUTIONS achieving the guarantee of Theorem 1.2. It is an extension of LOGADAPTIVE maintaining two solutions that open at least and at most k centers respectively, while opening at most three *free* centers per phase; this replaces the *bipoint rounding* [JV01, LS16, BPR⁺17, GPST23] and directly returns a $(2 + \varepsilon)$ approximation that opens at most $k + O(\log n/\varepsilon^2)$ centers. Its analysis, which is an extension of the analysis of Section 3.3, is presented in Appendix A.

3.1 Review of the Greedy Algorithm

In this section, we provide a brief overview of the classic 2-approximate LMP algorithm from [JMS02, JMM⁺03]. Our algorithms are derived by modifying this procedure, and these modifications become more intuitive when we revisit the basic concepts of the original algorithm. We present it in Algorithm 2 as a primal-dual algorithm to emphasize the changes we introduce.

Algorithm 2 (GREEDY).

Initialization: Set $\hat{f} = 2f, S = \emptyset, A = D$ and $\alpha_j = 0$ for every $j \in A = D$.

While $A \neq \emptyset$:

Increase α -values of active clients $(\alpha_j)_{j \in A}$ uniformly until one of the following events happen:

1. There is an unopened facility i whose bids pay for it:

$$\sum_{j \in A} [\alpha_j - d(i, j)]^+ + \sum_{j \in D-A} [d(j, S) - d(j, i)]^+ = \hat{f}.$$

Open i (i.e., add it to S) and mark all clients $j \in A$ such that $\alpha_j \geq d(j, S)$ as served (i.e., remove them from A).

2. There is a client $j \in A$ so that

$$\alpha_j = d(j, S).$$

The client j is removed from the set A of active clients.

The algorithm maintains the set A of *active* clients and the set S of opened facilities. Initially, all clients are active, i.e., $A = D$, and no facility is open, i.e., $S = \emptyset$. Additionally, the algorithm maintains an approximately feasible dual solution α , where each client $j \in D$ has a corresponding value α_j . These values are initialized to 0.

While there is at least one active client, the algorithm uniformly increases the α -values of the active clients until there is a facility i whose “bids” are sufficient to open it or an active client “gets connected” to an already opened facility. The bids from a client j to a facility i depend on whether j is active or not. If $j \in A$, then a portion of its budget α_j , specifically $d(i, j)$, is allocated to cover the connection cost of j , and the remainder, $\alpha_j - d(i, j)$, is used to bid towards the facility’s opening cost $\hat{f} := 2f$. For a client $j \notin A$, which is already connected to an open facility at a distance $d(j, S)$, the bid towards i is the difference in distance, i.e., $d(j, S) - d(i, j)$ (or 0 if i is farther away from j than S).

Finally, the algorithm marks any client j with $\alpha_j \geq d(j, S)$ as inactive, ensuring that at any point in time, active clients j satisfy $\alpha_j \leq d(j, S)$.

An overview of the analysis of GREEDY is as follows. We consider the final α -values denoted by $(\alpha_j^*)_{j \in A}$, and the final set S^* of opened facilities. The definition of the bids is to guarantee that the α^* -values pay for both the opening costs of the facilities in S^* and the connection costs of the clients:

Lemma 3.1. $\sum_{j \in D} \alpha_j^* = \sum_{j \in D} d(j, S^*) + \hat{f} \cdot |S^*|.$

This is complemented by showing that $\alpha^*/2$ is a feasible dual solution.

Lemma 3.2. For a facility i , $\sum_{j \in D} [\alpha_j^* - 2d(i, j)]^+ \leq \hat{f}$.

To prove dual-feasibility, two key properties are used:

No overbidding: At any point in time, for every $i \in F$,

$$\sum_{j \in A} [\alpha_j - d(i, j)]^+ + \sum_{j \in D-A} [d(j, S) - d(j, i)]^+ \leq \hat{f}.$$

No active clients bid on open facilities: At any point in time, $\alpha_j \leq d(j, S)$ for $j \in A$.

Specifically, if we give a total order of the clients so that $j \leq \ell$ if $\alpha_j^* \leq \alpha_\ell^*$, the above two properties are used to prove the following technical lemma; intuitively, at time right before α_ℓ^* , client $j \in D - A$ satisfies $d(j, S) + d(j, i) + d(i, \ell) \geq \alpha_\ell^*$, which implies $d(j, S) - d(j, i) \geq \alpha_\ell^* - 2d(i, j) - d(i, \ell)$.

Lemma 3.3. For any $\ell \in D$,

$$\sum_{j \in D: j \geq \ell} [\alpha_\ell^* - d(i, j)]^+ + \sum_{j \in D: j < \ell} [\alpha_\ell^* - 2d(i, j) - d(i, \ell)]^+ \leq \hat{f}.$$

Lemma 3.2 then follows by summing up the above inequality for all $\ell \in D$. Let $\text{opt}_{\text{LP}}(f)$ be the value of an optimal solution to the linear programming relaxation when opening costs are f . Then, we get the following theorem by combining Lemma 3.1 and Lemma 3.2.

Theorem 3.4. The set S^* of facilities output by GREEDY satisfies

$$\sum_{j \in D} d(j, S^*) \leq 2 \cdot (\text{opt}_{\text{LP}}(f) - f \cdot |S^*|).$$

Recall that $\text{opt}_{\text{LP}}(f) - f \cdot k$ is a lower bound on the optimal solution to the k -median problem (see Section 1.3). Hence, if GREEDY outputs a solution S^* such that $|S^*| = k$, then it is a 2-approximate solution to k -median. Moreover, we can always set f high enough so that any good solution opens at most k facilities and f low enough so that any good solution opens at least k facilities. Thus, a simple binary search can find values f and f' such that running GREEDY with opening costs f opens $k_1 \geq k$ facilities, while running it with opening costs f' opens $k_2 \leq k$ facilities. However, even though we can ensure that $|f - f'|$ is exponentially small, say $1/2^n$, it turns out that GREEDY is quite unstable, and we may have $k_1 \ll k$ and $k_2 \gg k$, which does not directly imply anything useful for k -median. In other words, this has been addressed through an additional rounding algorithm called bi-point rounding, which introduces an extra factor into the approximation guarantee. In contrast, we modify GREEDY into a similar algorithm that is “smoother” and less adaptive in its selection of facilities to open.

3.2 LOGADAPTIVE

It is natural to think of the GREEDY as increasing the α -values of active clients with a time θ . That is, we always have $\alpha_j = \theta$ for $j \in A$. A difficulty in ensuring that GREEDY opens k facilities is that it is a very adaptive algorithm. To see this, consider the case when two facilities i and i' both become paid for at the same time θ . Whether we open i or i' can dramatically change the bids to future facilities and completely change the trace and what facilities are opened by the algorithm. Our idea to overcome this difficulty is to limit the number of time steps where such decisions are made, thus lowering the adaptivity of the algorithm. Specifically, our high-level goal is to only consider the time steps of the form $(1 + \varepsilon^2)^0, (1 + \varepsilon^2)^1, (1 + \varepsilon^2)^2, \dots$. Then, as the minimum nonzero distance is 1 and the maximum distance is $\text{poly}(n)$, the standard arguments (formalized in Section 3.5) imply that the total number of time steps that we need to consider is $O(\log(n)/\varepsilon^2)$, and this is what we refer to as $O(\log n)$ adaptivity.

The difficulty in only considering those time steps is that we easily lose the property that we never overbid a facility i , which is crucial in the analysis of GREEDY. Indeed, consider a time step

$\theta = (1 + \varepsilon^2)^t$ and let A and I be the current active and inactive clients, respectively. Now, suppose that there is a facility i whose bids

$$\sum_{j \in A} [\alpha_j - d(i, j)]^+ + \sum_{j \in I} [d(j, S) - d(j, i)]^+$$

are just barely below $\hat{f}$. However, in that case, increasing the α -value of active clients by a factor $1 + \varepsilon^2$ may cause the bids to be strictly above $\hat{f}$, leading to difficulties in the analysis. (Specifically, in the proof that $\alpha^*/2$ is dual feasible.) We intuitively change GREEDY in two ways to handle this. First, we open centers if the bids pay for them in the metric scaled down by a factor of $(1 - \delta)$. That is, we consider a facility i paid for if

$$\sum_{j \in A} [\alpha_j - (1 - \delta)d(i, j)]^+ + (1 - \delta) \sum_{j \in I} [d(j, S) - d(j, i)]^+ \geq \hat{f}.$$

As we scale down the metric (i.e., the connection cost) by a factor $1 - \delta$, this loses a factor $1/(1 - \delta)$ in the approximation guarantee. However, the benefit is the following. Let $X = \{j \in A \mid \alpha_j \geq (1 - \delta)d(j, i)\}$ be the active clients that bid for i . Then

$$\sum_{j \in X} (\alpha_j - (1 - \delta)d(i, j)) \geq \sum_{j \in X} ((1 + \varepsilon^2)\alpha_j - d(i, j))$$

assuming that $\sum_{j \in X} \varepsilon^2 \alpha_j = \varepsilon^2 \theta |X|$ is smaller than $\delta \sum_{j \in X} d(i, j)$. We will select $\delta = 3\varepsilon$ and so this means that it is safe to increase the α -values of active clients by a factor $(1 + \varepsilon^2)$ as long as the average distance from bidding active clients is at least $\varepsilon^2 \theta / \delta = \varepsilon \theta / 3$. Our second change deals with the remaining case when the clients in X are close to co-located with i . We do so by allowing the clients j in the ball $B(i, \varepsilon \theta)$ to increase their bids for i up to $\min((1 + \varepsilon^2)\alpha_j, (1 - \delta)d(j, S))$. These changes are captured in the following definition of openable facilities. The bids $(\tau_j)_{j \in A}$ from active clients are only allowed to increase for nearby clients (the first two properties). A facility is considered paid for with respect to the scaled down metric (the third property). Finally, for the proof of dual feasibility even after increasing the bids of nearby clients, we need a technical fourth condition in the definition. We remark that we use (α, S, A, θ) to denote the algorithm's state, where α is the approximate feasible dual solution, S the set of open facilities, A the set of active clients (which determines $I = D \setminus A$), and θ the time. Both GREEDY and LOGADAPTIVE (that we describe below) satisfy $\alpha_j = \theta$ for all $j \in A$.

Definition 2. A facility i is openable with respect to the algorithm's state (α, S, A, θ) if there are bids $(\tau_j)_{j \in A}$ of active clients that satisfy the following conditions.

- Only nearby clients are increased:

$$\tau_j = \alpha_j \quad \text{for every } j \in A - B(i, \varepsilon \theta).$$

- Nearby clients are only increased slightly:

$$\alpha_j \leq \tau_j \leq \min\{(1 - \delta)d(j, S), (1 + \varepsilon^2)\theta\} \quad \text{for every } j \in A \cap B(i, \varepsilon \theta).$$

- Facility i is paid for:

$$\sum_{j \in A} [\tau_j - (1 - \delta)d(i, j)]^+ + (1 - \delta) \sum_{j \in I} [d(j, S) - d(i, j)]^+ \geq \hat{f}.$$

- Dual feasibility: for every facility i_0 and $\ell \in A$,

$$\sum_{j \in A} [\tau_j - d(i_0, j)]^+ + \sum_{j \in I} [\tau_\ell - 2d(j, i_0) - d(\ell, i_0)]^+ \leq \hat{f}.$$

We note that when the state of the algorithm is clear from the context, we simply say that " i is openable" without explicitly specifying the state.

The following lemma follows from that we can write the selection of the bids as a linear program. Indeed, given the state (α, S, A, θ) of the algorithm, each of the four conditions on the bids $(\tau_j)_{j \in A}$ can be formulated as linear inequalities.

Lemma 3.5. *There is a polynomial-time algorithm that, given the state (α, S, A, θ) and a facility i , either outputs bids $(\tau)_{j \in A}$ that satisfy all the conditions of Definition 2 or certifies that i is not openable.*

Proof. Fix i and (α, S, A, θ) and consider a linear program with variables $(\tau_j)_{j \in A}$. We claim that each of the four bullets in Definition 2 can be expressed as linear constraints. Indeed, the first two bullets are immediate. As $\tau_j = \alpha_j$ for every $j \in A - B(i, \varepsilon\theta)$, we can treat them as constants and only have $(\tau_j)_{j \in A \cap B(i, \varepsilon\theta)}$ as the true variables. Then, the third bullet,

$$\sum_{j \in A} [\tau_j - (1 - \delta)d(i, j)]^+ + (1 - \delta) \sum_{j \in I} [d(j, S) - d(i, j)]^+ \geq \hat{f}$$

becomes a linear inequality in the true variables, since the only terms involving the true variables are $[\tau_j - (1 - \delta)d(i, j)]^+$ for $j \in A \cap B(i, \varepsilon\theta)$, and for those j , $[\tau_j - (1 - \delta)d(i, j)]^+$ is indeed equal to $(\tau_j - (1 - \delta)d(i, j))$ without $[\cdot]^+$ because $(\tau_j - (1 - \delta)d(i, j)) \geq \theta - (1 - \delta)\varepsilon\theta > 0$.

Finally, the fourth bullet

$$\sum_{j \in A} [\tau_j - d(i_0, j)]^+ + \sum_{j \in I} [\tau_\ell - 2d(j, i_0) - d(\ell, i_0)]^+ \leq \hat{f}$$

is equivalent to introducing variables t_j for $j \in D$ and adding the following constraints: (1) $t_j \geq \tau_j - d(i_0, j)$ for $j \in A$, (2) $t_j \geq \tau_\ell - 2d(j, i_0) - d(\ell, i_0)$ for $j \in I$, (3) $t_j \geq 0$ for $j \in D$, and (4) $\sum_{j \in A} t_j + \sum_{j \in I} t_j \leq \hat{f}$. $\square$

The description of LOGADAPTIVE is given in Algorithm 3. It takes a parameter $\varepsilon \in [0, 1/6]$ and

Algorithm 3 (LOGADAPTIVE).

Initialization: Set $\hat{f} = 2f$, $S = \emptyset$, $A = D$, $\theta = 1$ and $\alpha_j = \theta$ for every $j \in A = D$.

Repeat the following while $A \neq \emptyset$:

1. While there is an unopened facility i that is openable (chosen arbitrarily if there are many),

Calculate $(\tau_j)_{j \in A}$ so that i is openable with bids $(\tau_j)_{j \in A}$. Open i (i.e., add it to S), let $\alpha_j \leftarrow \tau_j$ for every $j \in A$ (note that only the values of the clients in $A \cap B(i, \varepsilon\theta)$ change), mark all clients $j \in A$ such that $\alpha_j \geq (1 - \delta)d(j, S)$ as served (i.e., remove them from A).

2. Perform the following step and proceed to the next phase.

For every remaining $j \in A$, let $\alpha_j \leftarrow \min((1 + \varepsilon^2)\theta, (1 - \delta)d(j, S))$, remove it from A when $\alpha_j = (1 - \delta)d(j, S)$. Update $\theta = (1 + \varepsilon^2)\theta$.

$\delta = 3\varepsilon$. The time θ is now increased in steps by a multiplicative factor of $(1 + \varepsilon^2)$. This is done in each iteration of the outermost loop, "Repeat the following while $A \neq \emptyset$," which we refer to as a *phase* of the algorithm. Inside a phase, we refer to the two points as *stages*, so stage 1 corresponds to the while-loop where we open openable facilities, and stage 2 corresponds to the increase of α -values of the active clients A . At any point in time, the state of the algorithm is described, as aforementioned, by the tuple (α, S, A, θ) . The careful reader may observe that A is redundant because the active clients A are determined by S . Indeed, the same holds for the set of *inactive* clients $I = D - A = \{j \in D : \alpha_j < (1 - \delta)d(j, S)\}$. However, it will be notationally convenient to explicitly keep track of A in the algorithm's state.

The parameter $\varepsilon > 0$ regulates the number of phases the algorithm will make. Indeed, all active clients have the same α -value, and after the completion of p phases, the α -value of active clients equals $(1 + \varepsilon^2)^p$. This ensures that the number of phases until the algorithm terminates is $O(\log(n)/\varepsilon^2)$. Hence, the increase of α -values only depends on $O(\log(n)/\varepsilon^2)$ states of the algorithm, the states

before the second stage in each of the phases. This is why we refer to the algorithm to be of logarithmic adaptivity.

Apart from impacting the number of phases, the choice of ε also impacts the approximation guarantee. Specifically, we prove the following theorem in the subsequent section. (Recall that $\delta = 3\varepsilon$.)

Theorem 3.6. *The set S^* of facilities output by LOGADAPTIVE satisfies*

$$\sum_{j \in D} d(j, S^*) \leq \frac{2}{1 - \delta} \cdot (\text{opt}_{\text{LP}}(f) - f \cdot |S^*|) .$$

3.3 Analysis of LOGADAPTIVE

An *atomic step* refers to the set of operations in the indented paragraphs in LOGADAPTIVE that change the state (α, S, A, θ) . (I.e., an atomic step happens when a new facility i is opened and at the end of each phase.) By “at any point of the algorithm”, we mean any point in the algorithm’s execution that is not in the middle of an atomic step. Throughout the analysis, we let α_j^* be the final α -value of a client $j \in D$, and we let S^* be the set of opened facilities. We start by analyzing the approximation guarantee of the algorithm with respect to α^* , and we then prove that $\alpha^*/2$ is a feasible solution to the dual.

3.3.1 Approximation Guarantee.

In the following lemma, we prove that $\sum_{j \in D} \alpha_j^*$ pays for the opening costs of the facilities in S^* and the connection cost of the clients scaled by a factor $(1 - \delta)$.

Lemma 3.7. *We have $\sum_{j \in D} \alpha_j^* \geq (1 - \delta) \sum_{j \in D} d(j, S^*) + \hat{f} |S^*|$.*

In the next subsection, we prove the following lemma showing that $\alpha^*/2$ is a feasible dual solution.

Lemma 3.8. *For every facility i , $\sum_{j \in D} [\alpha_j^* - 2d(i, j)]^+ \leq \hat{f}$.*

These two lemmas imply

$$\text{opt}_{\text{LP}}(f) \geq \sum_{j \in D} \frac{\alpha_j^*}{2} \geq \frac{1 - \delta}{2} \sum_{j \in D} d(j, S^*) + f |S^*| ,$$

where we used that $\hat{f} = 2 \cdot f$. Therefore, we have an LMP $2/(1 - \delta)$ -approximation: the connection cost of the output solution S^* satisfies

$$\sum_{j \in D} d(j, S^*) \leq \frac{2}{1 - \delta} (\text{opt}_{\text{LP}}(f) - f |S^*|) .$$

In other words, Lemma 3.7 and Lemma 3.8 imply Theorem 3.6.

We proceed to prove Lemma 3.7 (and prove Lemma 3.8 in the next subsection).

Proof of Lemma 3.7. We prove by induction that at any point of the algorithm

$$\sum_{j \in I} \alpha_j \geq \sum_{j \in I} (1 - \delta) d(j, S) + \hat{f} |S| . \quad (3)$$

The equality is initially true since $A = D$ (thus $I = \emptyset$) and $S = \emptyset$. We now analyze each of the two stages separately.

Stage 1. Consider what happens when we open a facility i , i.e., add it to S . Let (α, S, A, θ) be the state right before opening i , A' be the clients that are removed from A by the opening, which means $A' = \{j \in A : \tau_j \geq (1 - \delta)d(i, j)\}$. (Recall that the opening lets $\alpha_j \leftarrow \tau_j$ for $j \in A$.) Further, let $X := \{j \in I : d(j, i) < d(j, S)\}$ be the subset of clients of I that make positive bids to i . The change of cost of the right-hand side of (3) is at most

$$\hat{f} + \sum_{j \in A'} (1 - \delta)d(i, j) + \sum_{j \in X} (1 - \delta)(d(i, j) - d(j, S)).$$

Since i is fully paid (the third bullet of Definition 2), we also have

$$\hat{f} \leq \sum_{j \in A'} (\tau_j - (1 - \delta)d(i, j)) + \sum_{j \in X} (1 - \delta)(d(j, S) - d(i, j)).$$

We thus get that the change of cost of the right-hand side is at most $\sum_{j \in A'} \tau_j$, which is the change of the left-hand side.

Stage 2. No facility is open at this stage, and every client j that becomes inactive at this stage has $\alpha_j = (1 - \delta)d(j, S)$, which implies that the left-hand side and right-hand side both increase with the same amount when a client j becomes inactive. $\square$

3.3.2 Dual Feasibility

We start by showing that we never overbid a facility i . This is rather immediate in the analysis of GREEDY that we outlined in Section 3.1. While we prove this remains true for LOGADAPTIVE, proving it requires more work, especially because of the discrete increase of α -values in the second stage of a phase.

Claim 1 (No Over-Bidding). *At any point of the algorithm, for every facility i ,*

$$\sum_{j \in A} [\alpha_j - d(i, j)]^+ + \sum_{j \in I} [(1 - \delta)d(j, S) - d(j, i)]^+ \leq \hat{f}. \quad (4)$$

Proof. In the beginning, the claim is true because every client is active and $\alpha_j = 1 \leq d(i, j)$ for every $j \in D$. Let us prove that each step of the algorithm maintains this invariant. We first consider the steps that happen during the first stage and then those during the second stage.

Stage 1. Suppose that facility i' is open with the bids $(\tau_j)_{j \in A}$. It does (1) potentially increase the α -values of clients in $B(i', \varepsilon\theta) \cap A$, and (2) make clients in $A \cap B(i', \theta/(1 - \delta))$ inactive, which includes $B(i', \varepsilon\theta) \cap A$ considered in (1).

Consider any facility i and see how (4) is impacted by this change. First, observe that a client j that was inactive before the opening of i' does not increase its contribution to (4). This is because $d(j, S)$ monotonically decreases in S . We now turn our attention to the more interesting case. For a client j becoming inactive from active, the contribution to (4) changes from $[\alpha_j - d(i, j)]^+$ (where α -values are measured right before the opening) to $[(1 - \delta)d(j, S) - d(j, i)]^+$ (where S includes i').

If α_j does not change by the opening of i' , the fact that j becomes inactive means $(1 - \delta)d(j, S) \leq \alpha_j$, so j 's contribution to (4) cannot increase. If α_j does strictly increase by the opening of i' , it means that $d(j, S) \leq d(j, i') \leq \varepsilon\theta$ after the opening, so the contribution to (4) decreases by at least $\theta - (1 - \delta)\varepsilon\theta$ or becomes 0.

We have thus proved that no client increases their contribution to (4) during the steps of the first stage, and so these steps maintain the inequality.

Stage 2. Assume towards contradiction that at the end of the phase, if we increase α_j of every $j \in A$ to $\min((1 + \varepsilon^2)\theta, (1 - \delta)d(j, S))$, the claim is violated for a non-empty subset of facilities F' . We select

a “minimal” such counter example in the following way: let $\tau' \leq (1 + \varepsilon^2)\theta$ be the smallest value so that $\alpha'_j := \min(\tau', (1 - \delta)d(j, S))$ satisfies

$$\sum_{j \in A} [\alpha'_j - d(i, j)]^+ + \sum_{j \in I} [(1 - \delta)d(j, S) - d(j, i)]^+ = \hat{f}, \quad (5)$$

for some $i \in F'$. We remark that $\tau' \geq \theta$ since (4) is satisfied for all facilities after the first stage by the previous arguments. Furthermore, $i \notin S$ is not yet open since if it was open then there would be no client $j \in A$ for which $\alpha'_j - d(i, j)$ is strictly positive. Hence the increase in α -values cannot cause i to violate (4) if it were already opened, which would contradict $i \in F'$.

We now show that this i with $\tau_j = \alpha'_j$ for $j \in A \cap B(i, \varepsilon\theta)$ and $\tau_j = \alpha_j$ for $j \in A - B(i, \varepsilon\theta)$ satisfies the conditions of Definition 2 for i . In other words, i is an openable facility (that was not yet opened), which contradicts the completion of the first stage. We verify the conditions of Definition 2 one-by-one. The first two bullets are satisfied by the definition of the bids $(\tau_j)_{j \in A}$. For the third bullet,

$$\begin{aligned} & \sum_{j \in A} [\tau_j - (1 - \delta)d(i, j)]^+ + (1 - \delta) \sum_{j \in I} [d(j, S) - d(i, j)]^+ \\ &= \sum_{j \in A \cap B(i, \varepsilon\theta)} [\alpha'_j - (1 - \delta)d(i, j)]^+ + \sum_{j \in A - B(i, \varepsilon\theta)} [\alpha_j - (1 - \delta)d(i, j)]^+ + (1 - \delta) \sum_{j \in I} [d(j, S) - d(i, j)]^+ \\ &\geq \sum_{j \in A \cap B(i, \varepsilon\theta)} [\alpha'_j - d(i, j)]^+ + \sum_{j \in A - B(i, \varepsilon\theta)} [\alpha'_j - d(i, j)]^+ + \sum_{j \in I} [(1 - \delta)d(j, S) - d(i, j)]^+. \end{aligned}$$

To upper bound the second summation we used that $\alpha_j - (1 - \delta)d(i, j) \geq (1 + \varepsilon^2)\alpha_j - d(i, j) \geq \alpha'_j - d(i, j)$ when $d(i, j) \geq \frac{\varepsilon^2 \alpha_j}{\delta} = \frac{\varepsilon\theta}{3}$ (recall that $\alpha_j = \theta$ for all $j \in A$), condition which is satisfied by the clients not in $B(i, \varepsilon\theta)$.

It remains to verify the fourth bullet of Definition 2. Suppose toward contradiction that there is a facility i_0 and $\ell \in A$ such that

$$\sum_{j \in A} [\tau_j - d(i_0, j)]^+ + \sum_{j \in I} [\tau_\ell - d(i_0, \ell) - 2d(i_0, j)]^+ > \hat{f}.$$

Note that $\ell \in A$ implies $\tau_\ell \leq (1 - \delta)d(\ell, S)$ and for every $j \in I$, $(1 - \delta)d(\ell, S) \leq (1 - \delta)(d(\ell, i_0) + d(i_0, j) + d(j, S))$. Thus

$$\tau_\ell - d(i_0, \ell) - 2d(i_0, j) \leq (1 - \delta)d(j, S) - d(i_0, j). \quad (6)$$

We conclude that

$$\begin{aligned} & \sum_{j \in A} [\alpha'_j - d(i_0, j)]^+ + \sum_{j \in I} [(1 - \delta)d(j, S) - d(i_0, j)]^+ \\ &\geq \sum_{j \in A} [\tau_j - d(i_0, j)]^+ + \sum_{j \in I} [(1 - \delta)d(j, S) - d(i_0, j)]^+ \\ &\stackrel{(6)}{\geq} \sum_{j \in A} [\tau_j - d(i_0, j)]^+ + \sum_{j \in I} [\tau_\ell - d(i_0, \ell) - 2d(i_0, j)]^+ > \hat{f} \end{aligned}$$

This implies that we could have further lowered τ' while keeping equality (5) true for i_0 . This contradicts the minimality of τ' . $\square$

Having proved that LOGADAPTIVE as GREEDY maintains the invariant that there is no overbidding, we are ready to prove the dual-feasibility of $\alpha^*/2$. (This corresponds to Lemma 3.2 in the analysis of GREEDY.)

Lemma 3.8. *For every facility i , $\sum_{j \in D} [\alpha_j^* - 2d(i, j)]^+ \leq \hat{f}$.*

We remark that this implies dual feasibility of $\alpha^*/2$ by dividing both sides by 2 (recall $\hat{f} = 2 \cdot f$).

The rest of the section is devoted to the proof of Lemma 3.8, and from now on, we focus on an arbitrary but fixed facility $i \in F$. Let $D^* = \{j \in D : \alpha_j^* > 2d(i, j)\}$ be those clients that contribute to the left-hand side of the inequality in the claim. Further, order the clients in D^* according to α^* values by the time they are removed from A (in the algorithm) and break ties according to α^* values (in increasing order). We will use j, ℓ to denote clients in D^* , and let us slightly abuse notation and use $j \leq \ell$ with respect to this ordering. In other words, we have $j \leq \ell$ if j was removed from A in an atomic step before ℓ was removed, or j and ℓ were removed during the same atomic step and $\alpha_j^* \leq \alpha_\ell^*$. We also use the terminology that a facility i_0 *freezes* i if i_0 becomes open when i is not, and $d(i, i_0) \leq (1 + 3\varepsilon)\theta/2$. (Of course, i freezes itself when it is open.) This notion is important as for a facility i , its D^* and their α^* values do not change after it is frozen.

The following lemma, analogous to Lemma 3.3 in the analysis of GREEDY, is key to proving dual feasibility.

Lemma 3.9. *For any $\ell \in D^*$ that becomes inactive strictly before i becomes frozen,*

$$\sum_{j \in D^*: j \geq \ell} [\alpha_\ell^* - d(i, j)]^+ + \sum_{j \in D^*: j < \ell} [\alpha_\ell^* - 2d(i, j) - d(i, \ell)]^+ \leq \hat{f}. \quad (7)$$

Proof. Let us do the following case analysis depending on the stage ℓ becomes inactive.

Stage 1. ℓ becomes inactive because of the opening of some i_0 . By the statement of the lemma, it suffices to handle the case that i_0 does not freeze i . So $d(i, i_0) > (1 + 3\varepsilon)\theta/2$. The opening of i_0 might have strictly increased the α -value of the clients in $B(i_0, \varepsilon\theta)$, but no such client j will be in D^* , since they immediately become inactive while

$$d(i, j) \geq d(i, i_0) - d(i_0, j) > ((1 + 3\varepsilon)/2 - \varepsilon)\theta \geq (1 + \varepsilon^2)\theta/2 \geq \alpha_j^*/2.$$

Applying this argument to every facility that is open in this phase (which did not freeze i), we can conclude that, right before i_0 is open, every $j \in D^*$ has $\alpha_j \leq \theta$ and all active ones have $\alpha_j = \theta$. (So, $\alpha_\ell^* = \theta$ as well since the opening of i_0 does not strictly increase α_ℓ .) Claim 1, applied right before i_0 is open, ensures that

$$\sum_{j \in A} [\alpha_j - d(i, j)]^+ + \sum_{j \in I} [(1 - \delta)d(j, S) - d(i, j)]^+ \leq \hat{f}.$$

This satisfies (7) as (1) every $j \in D^*$ with $j \geq \ell$ is in $j \in A$ at that point by our ordering of the clients within D^* , and (2) for every $j < \ell$ with $j \in I$,

$$\alpha_\ell^* < (1 - \delta)d(\ell, S) \leq (1 - \delta)d(j, S) + d(j, i) + d(i, \ell)$$

implies

$$\alpha_\ell^* - 2d(i, j) - d(i, \ell) \leq (1 - \delta)d(j, S) - d(i, j).$$

(Note that this S does not contain i_0 and ℓ was not connected yet.)

Stage 2. Suppose that ℓ becomes inactive by the increase of the α -values at the end of a phase. Then Claim 1 ensures that, at the end of this phase (after ℓ becomes inactive),

$$\sum_{j \in A} [\alpha_j - d(i, j)]^+ + \sum_{j \in I} [(1 - \delta)d(j, S) - d(j, i)]^+ \leq \hat{f}. \quad (8)$$

Let us see that the left-hand side of (8) is at least that of (7) by comparing the contribution of $j \in D^*$ to both. If $j \in A$, its contribution to the former is definitely at least that to the latter, since any active j has $\alpha_j = (1 + \varepsilon^2)\theta \geq \alpha_\ell^*$.

For $j \in I$, we must have

$$\alpha_\ell^* \leq (1 - \delta)d(\ell, S) \leq (1 - \delta)(d(\ell, i) + d(i, j) + d(j, S)) \leq (1 - \delta)d(j, S) + d(\ell, i) + d(i, j).$$

(since α_ℓ is never increased above $(1 - \delta)d(\ell, S)$). Thus

$$\alpha_\ell^* - 2d(i, j) - d(i, \ell) \leq (1 - \delta)d(j, S) - d(i, j)$$

which satisfies our goal when $j < \ell$.

Finally, when $j \in I$ but $j \geq \ell$, it means that j was removed at the same time as ℓ at the end of the phase. In that case $(1 - \delta)d(j, S) = \alpha_j^*$ and, as we break ties with α^* -values, $\alpha_j^* \geq \alpha_\ell^*$, which again satisfies our goal. $\square$

Equipped with the above lemma, we are ready to complete the proof of dual feasibility.

Proof of Lemma 3.8. Consider the case that i is frozen by i_0 with $(\tau_j)_{j \in A}$. We first prove the following claim that says that after i is frozen the α -values of clients in D^* are finalized.

Claim. We have $\alpha_j^* = \tau_j$ for every $j \in D^* \cap A$.

Proof of Claim. Consider $j \in A$. First, note that if $\tau_j \geq (1 - \delta)d(j, i_0)$ then j is removed from A when i_0 is opened and so $\alpha_j^* = \tau_j$.

In the other case, when $\alpha_j \leq \tau_j < (1 - \delta)d(j, i_0)$ we show that $j \notin D^*$. Consider the (future) time right after j is removed from the active clients. The α -value of j then (which is equal to the final α_j^*) cannot be strictly greater than $(1 - \delta)d(j, i_0)$; whether it is increased in stage 1 (as $\alpha \leftarrow \tau$) or stage 2, since $i_0 \in S$ (where S is the set of open facilities right before j becomes inactive), the algorithm ensures that it cannot be strictly more than $(1 - \delta)d(j, S) \leq (1 - \delta)d(j, i_0)$. By the triangle inequality

$$d(j, i) \geq d(j, i_0) - d(i_0, i) = \left(1 - \frac{d(i_0, i)}{\theta} \frac{\theta}{d(j, i_0)}\right) d(j, i_0),$$

where θ is considered at the time when i_0 was open. As i_0 freezes i , we have $d(i_0, i) \leq (1 + 3\epsilon)\theta/2$ and, by the assumption of the case we have $\theta = \alpha_j \leq (1 - \delta)d(j, i_0)$. Plugging in those bounds to the above inequality yields,

$$d(j, i) \geq \left(1 - \frac{(1 + 3\epsilon)}{2}(1 - \delta)\right) d(j, i_0).$$

We rewrite the above inequality (multiplying both sides by two and using that $\delta = 3\epsilon$) to obtain

$$2d(j, i) \geq (2 - (1 + 3\epsilon)(1 - \delta)) d(j, i_0) = (1 + 9\epsilon^2) d(j, i_0).$$

In other words we have $\alpha_j^* \leq d(j, i_0) < 2d(j, i)$, so j cannot be in D^* . $\square$

Having proved the above claim, we proceed to analyze the dual feasibility. As the α -values of clients in $A \cap D^*$ remain unchanged after the removal of i_0 , the fourth bullet of Definition 2 ensures that, for any $\ell \in A \cap D^*$,

$$\sum_{j \in A \cap D^*} [\alpha_j^* - d(i, j)]^+ + \sum_{j \in I \cap D^*} [\alpha_\ell^* - 2d(i, j) - d(\ell, i)]^+ \leq \hat{f}. \quad (9)$$

Moreover, for every client $\ell \in D^* \cap I$ that became inactive strictly before i_0 was opened, Lemma 3.9 gives

$$\sum_{j \in D^*: j \geq \ell} [\alpha_\ell^* - d(i, j)]^+ + \sum_{j \in D^*: j < \ell} [\alpha_\ell^* - 2d(i, j) - d(i, \ell)]^+ \leq \hat{f}. \quad (10)$$

Let us consider the summation of (10) for every $\ell \in I \cap D^*$ and (9) for every $\ell \in A \cap D^*$, and consider how many times each term appears. Let $a = |A \cap D^*|$, $b = |D^*|$.

- $-d(i, j')$ when $j' \in I \cap D^*$: Say j' is the q th client in D^* for some $q \leq b - a$. Then $-d(i, j')$ appears $b + (b - q)$ times in (9) ($2a$ times) and (10) ($2(b - a - q) + q$ times) as $-d(i, j)$ and $(q - 1)$ times as $-d(i, \ell)$ in (10) when $\ell = j'$, so the total is $2b - 1$.

- $-d(i, j')$ when $j' \in A \cap D^*$: It appears b times in (9) and (10) as $-d(i, j)$ and $b - a$ times as $-d(i, \ell)$ in (10) when $\ell = j'$, so the total is $2b - a$.
- $\alpha_{j'}^*$ for $j' \in I \cap D^*$: It appears b times in (10) as α_ℓ^* when $\ell = j'$.
- $\alpha_{j'}^*$ for $j' \in A \cap D^*$: It appears $b - a$ times in (9) as α_ℓ^* when $\ell = j'$ and once in (9) as α_j^* for each $\ell \in A \cap D^*$, so b times in total.

Summarizing we have,

$$b \sum_{j \in D^*} \alpha_j^* - (2b - a) \sum_{j \in D^* \cap A} d(i, j) - (2b - 1) \sum_{j \in D^* \cap I} d(i, j) \leq b\hat{f},$$

and so in particular $\sum_{j \in D^*} (\alpha_j^* - 2d(i, j)) \leq \hat{f}$, which proves the lemma in the case the opening of a facility i_0 freezes i .

To complete the proof, when no facility freezes i , Lemma 3.9 holds for every client in D^* , so we can simply sum up (10) over all clients in D^* , which corresponds to the special case of the above analysis when $D^* \cap A = \emptyset$. $\square$

3.4 Walking Between Two Solutions: Setup

Given the description of the LOGADAPTIVE, we present our final algorithm MERGESOLUTIONS, resulting in a solution that opens $k + O_\varepsilon(\log n)$ centers by maintaining two partial solutions that lead to opening at least k and at most k centers respectively and *gradually merge them* into a single solution that opens exactly $k + O_\varepsilon(\log n)$ facilities, exploiting the $O_\varepsilon(\log n)$ adaptivity of LOGADAPTIVE.

Before detailing the merging procedure, which we call MERGESOLUTIONS, it is helpful to observe that we may generalize the analysis to make it more “robust”. We introduce this more robust analysis in this subsection, where we also define the notation that will be used later to describe MERGESOLUTIONS in Section 3.5.

Starting from the two solutions obtained in the standard way whose facility costs differ by at most $\eta := 2^{-n}$, the two solutions we maintain will have an exponentially small difference η in one of their parameters. (These parameters will be either the facility cost f or some distance related to a *free facility* that will be introduced later.) For this reason, when we merge the two solutions, it may occur that some facilities in one solution are almost completely paid for, rather than fully paid for. To address this, we generalize the definition of “openable” to “ η -openable,” where the only difference is in the third condition: we now require each facility that is opened to be paid $\hat{f} - \eta n$ instead of $\hat{f}$ (as in Definition 2). We also note that being openable only depends on the time θ and the set S of open facilities so far. This is because $\alpha_j = \theta$ for $j \in A$, $I = \{j \in D : \theta \geq (1 - \delta)d(j, S)\}$ and $A = D - I$. This allows us to simplify notation and we have the following generalization of Definition 2.

Definition 3. Consider the time θ and let S be the set of opened facilities. Then set of inactive and active clients are $I = \{j \in D : \theta \geq (1 - \delta)d(j, S)\}$ and $A = D - I$, respectively. We say that a facility $i \in F$ is η -openable (with respect to θ and S) if there are bids $(\tau_j)_{j \in A}$ of active clients that satisfy the following conditions.

- Only nearby clients are increased:

$$\tau_j = \alpha_j \quad \text{for every } j \in A - B(i, \varepsilon\theta).$$

- Nearby clients are only increased slightly:

$$\alpha_j \leq \tau_j \leq \min\{(1 - \delta)d(j, S), (1 + \varepsilon^2)\theta\} \quad \text{for every } j \in A \cap B(i, \varepsilon\theta).$$

- Facility i is paid for (up to the error parameter η):

$$\sum_{j \in A} [\tau_j - (1 - \delta)d(i, j)]^+ + (1 - \delta) \sum_{j \in I} [d(j, S) - d(i, j)]^+ \geq \hat{f} - n\eta.$$

- *Dual feasibility*: for every facility i_0 and $\ell \in A$,

$$\sum_{j \in A} [\tau_j - d(i_0, j)]^+ + \sum_{j \in I} [\tau_\ell - 2d(j, i_0) - d(\ell, i_0)]^+ \leq \hat{f}.$$

We also introduce the concept of *free facilities*. During the process of merging the two solutions, we introduce free facilities, whose opening costs are not necessarily paid. While we will introduce at most $O(\log n)$ such free facilities, we do not restrict the number of free copies a regular facility may have. Let $S_{\text{free}} \subseteq S$ denote the multiset of free facilities that have been opened.

Moreover, for each free copy $\tilde{i} \in S_{\text{free}}$ of $i \in F$, we define a parameter $u(\tilde{i})$, which represents the distance between the freely opened facility $\tilde{i}$ and its original copy i . So for any point x in the metric space (either a facility or a client), the distance is given by $d(\tilde{i}, x) = u(\tilde{i}) + d(i, x)$.

We further introduce the following terminology: as mentioned, we refer to the facilities in S_{free} as *free facilities*, the facilities in F and $S_{\text{reg}} = S - S_{\text{free}}$ as *regular facilities*, and when we simply mention “facilities,” we are referring to their union.

Finally, we will use the letters i and h to denote facilities. Specifically, we use $i \in F$ to denote a regular facility and use $\tilde{i}$ to denote a free copy of i . When referring to a facility that can be either free or regular, we use the letter h .

We now introduce the definition of valid sequences, which aims to capture the sequence of facilities that are opened during the while-loop of stage 1 in LOGADAPTIVE.

Definition 4 (η -valid sequence). *Consider the time θ and let S be the set of opened facilities. A sequence $\langle h_1, \dots, h_\ell \rangle$ of facilities is η -valid (with respect to θ and S), if the following conditions hold.*

- Each h_t is either free or η -openable with respect to the time θ and a superset $S' \supseteq S \cup \{h_1, \dots, h_{t-1}\}$ of opened facilities.
- The sequence is maximal, i.e., there is no openable facility with respect to the time θ and $S \cup \{h_1, \dots, h_\ell\}$.

We remark that LOGADAPTIVE generates 0-valid sequences in each phase. Specifically, it generates sequences $\mathcal{H} = (H_1, H_2, \dots, H_L)$, where each H_p is 0-valid (with respect to $\theta = (1 + \varepsilon^2)^{p-1}$ and $S = \cup_{i \in [p-1]} H_i$) and contains only regular facilities. The set S of opened facilities is such that $A = \emptyset$ at the final time θ .

For a general sequence of sequences $\mathcal{H} = (H_1, H_2, \dots, H_L)$, let us say it consists of η -valid sequences if, for every $p \in [L]$, H_p is an η -valid sequence with respect to $\theta = (1 + \varepsilon^2)^{p-1}$ and $S = \cup_{q \in [p-1]} H_q$. (Here we abuse notation and treat H_q as a set of facilities.) Furthermore, we say that $\mathcal{H}$ is a *solution* if the set S of opened facilities ensures that $A = \emptyset$ at the end, meaning that at the time $\theta = (1 + \varepsilon^2)^L$ of the $(L + 1)$ st phase, the set of inactive clients $I = \{j \mid \theta \geq (1 - \delta)d(j, S)\}$ equals all clients in D . Otherwise, we say that $\mathcal{H}$ is a partial solution.

Theorem 3.6 can thus be stated as: if $\mathcal{H}$ is a solution of 0-valid sequences consisting of regular facilities then the set S of opened facilities satisfies

$$\sum_{j \in D} d(j, S) \leq \frac{2}{1 - \delta} \cdot (\text{opt}_{\text{LP}}(f) - f \cdot |S|).$$

We now allow to open regular facilities by paying at least $\hat{f} - \eta n$ (instead of $\hat{f}$) and incorporate the notion free facilities, which leads to the following generalization of the analysis, proved in Appendix A.

Theorem 3.10. *Consider a solution $\mathcal{H}$ of η -valid sequences and let S be the opened facilities. Further, let S_{free} and $S_{\text{reg}} = S - S_{\text{free}}$ be the free copies and regular facilities, respectively. Then*

$$\sum_{j \in D} d(j, S) \leq \frac{2}{1 - \delta} \cdot (\text{opt}_{\text{LP}}(f) - (f - \eta n) \cdot |S_{\text{reg}}|).$$

It will be useful to use similar terminology as we did in Section 3.3, let us consider the *execution* of the solution $\mathcal{H}$ where for $p = 1, \dots, L$, we run the p -th phase according to the sequence H_p ; in Stage 1, each facility in the sequence becomes open one by one with the corresponding $(\tau_j)_j$ values, and in Stage 2 at the end of the phase, $\theta \leftarrow \theta(1 + \varepsilon^2)$.

Then an *atomic step* can still be defined in the same way as in LOGADAPTIVE; it corresponds to opening a facility in Stage 1 (and updating α, S, A accordingly) or increasing α values in Stage 2 simultaneously (and updating S accordingly). By “at any point of the algorithm”, we mean any point in the algorithm’s execution that is not in the middle of an atomic step.

With this notation, we have the following generalization of Claim 1:

At any point of the algorithm, for every facility i ,

$$\sum_{j \in A} [\alpha_j - d(i, j)]^+ + \sum_{j \in I} [(1 - \delta)d(j, S) - d(j, i)]^+ \leq \hat{f}. \quad (11)$$

This is formally proved in Claim 10 in Appendix A, and the proof is essentially the same as that of Claim 1. We use it to prove Lemma 3.11, which we will use in the merging procedure to ensure maximality. At a point of the execution, we say that a facility i is *payable* if it is openable (with $\eta = 0$) except that the dual feasibility condition is not necessarily satisfied. The payable notion is thus a weaker notion than openable, and a facility i may be payable but not openable. However, we have the following lemma that says that if there is a payable facility, there is also an openable facility. We remark that it does not guarantee that a payable facility is also openable but it does guarantee that if there exists a payable facility, there is also an openable facility, which is sufficient for our arguments.

Lemma 3.11. *At any point of the algorithm, if there is a payable facility, then there is also an openable facility (that is not yet opened).*

Proof. The argument is similar to that of the “Stage 2” proof of Claim 1. Suppose there is a non-empty subset of facilities F' that are payable. To prove the lemma, we need to show that there is an openable facility.

We choose a “minimal” payable facility as follows. Let $\tau' \in [\theta, (1 + \varepsilon^2)\theta]$ be the smallest value such that there exists some $i \in F'$, for which the values τ_j , defined by

$$\tau_j := \begin{cases} \min\{\tau', (1 - \delta)d(j, S)\}, & \text{if } j \in A \cap B(i, \varepsilon\theta), \\ \alpha_j, & \text{otherwise,} \end{cases}$$

satisfy

$$\sum_{j \in A} [\tau_j - (1 - \delta)d(i, j)]^+ + (1 - \delta) \sum_{j \in I} [d(j, S) - d(j, i)]^+ \geq \hat{f}, \quad (12)$$

for some $i \in F'$. We remark that $i \notin S$ is not yet open since if it was open then there would be no client $j \in A$ for which $\tau_j - (1 - \delta)d(i, j)$ is strictly positive and the bids of inactive clients would be 0. (Strictly speaking, here we assume $\hat{f} > 0$ since otherwise all facilities are opened at the very beginning of the algorithm and there is nothing to prove.)

We now show that this τ satisfies the conditions of Definition 3 for i . In other words, i is an openable facility (that was not yet opened), which implies the lemma. We verify the conditions of Definition 3 with $\eta = 0$ one-by-one. The first two bullets are satisfied by the definition of the bids $(\tau_j)_{j \in A}$. The third bullet is satisfied by (12). Finally, we verify the dual feasibility condition. Suppose toward contradiction that there is a facility i_0 and $\ell \in A$ such that

$$\sum_{j \in A} [\tau_j - d(i_0, j)]^+ + \sum_{j \in I} [\tau_\ell - d(i_0, \ell) - 2d(i_0, j)]^+ > \hat{f}.$$

Note that $\ell \in A$ implies $\tau_\ell \leq (1 - \delta)d(\ell, S)$ and for every $j \in I$, $(1 - \delta)d(\ell, S) \leq (1 - \delta)(d(\ell, i_0) + d(i_0, j) + d(j, S))$. Thus

$$\tau_\ell - d(i_0, \ell) - 2d(i_0, j) \leq (1 - \delta)d(j, S) - d(i_0, j). \quad (13)$$

We conclude that

$$\begin{aligned} \sum_{j \in A} [\tau_j - d(i_0, j)]^+ + \sum_{j \in I} [(1 - \delta)d(j, S) - d(i_0, j)]^+ \\ \stackrel{(13)}{\geq} \sum_{j \in A} [\tau_j - d(i_0, j)]^+ + \sum_{j \in I} [\tau_\ell - d(i_0, \ell) - 2d(i_0, j)]^+ > \hat{f}. \end{aligned}$$

We remark that this implies that $\tau' > \theta$ since otherwise, as then $\tau_j = \alpha_j = \theta$ for every $j \in A$, the above inequality would contradict (11).

We proceed to prove that the above inequality implies that we could have further lowered τ' while keeping equality (12) true for i_0 , which contradicts the minimality of τ' (using that $\tau' > \theta$). (It implies that i_0 is also payable with a smaller τ' , which contradicts the minimality of τ' .) To see this, define the values $\tau_j^{(i_0)}$ for every $j \in A$ by

$$\tau_j^{(i_0)} := \begin{cases} \min\{\tau', (1 - \delta)d(j, S)\}, & \text{if } j \in A \cap B(i_0, \varepsilon\theta), \\ \alpha_j, & \text{otherwise,} \end{cases}$$

We claim

$$\tau_j^{(i_0)} - (1 - \delta)d(i_0, j) \geq \tau_j - d(i_0, j) \quad \text{for every } j \in A.$$

If $j \in B(i_0, \varepsilon\theta)$, then $\tau_j^{(i_0)} \geq \tau_j$ and the inequality is immediate. Else if $j \notin B(i_0, \varepsilon\theta)$, then $\tau_j^{(i_0)} = \alpha_j = \theta$ and $\tau_j \leq \tau' \leq (1 + \varepsilon^2)\theta$, so the inequality is also satisfied in this case because $\delta d(i_0, j) \geq \delta\varepsilon\theta = 3\varepsilon^2\theta \geq \varepsilon^2\theta$.

By the above calculations, we have

$$\begin{aligned} \sum_{j \in A} [\tau_j^{(i_0)} - (1 - \delta)d(i_0, j)]^+ + (1 - \delta) \sum_{j \in I} [d(j, S) - d(j, i_0)]^+ \\ \geq \sum_{j \in A} [\tau_j - d(i_0, j)]^+ + (1 - \delta) \sum_{j \in I} [d(j, S) - d(j, i_0)]^+ \\ \geq \sum_{j \in A} [\tau_j - d(i_0, j)]^+ + \sum_{j \in I} [(1 - \delta)d(j, S) - d(j, i_0)]^+ \\ > \hat{f}, \end{aligned}$$

which implies that τ' could have been further lowered while keeping (12) true for i_0 , contradicting the minimality of τ' . This concludes the proof that there is an openable facility, which proves the lemma. $\square$

In the next subsection we will merge two solutions into one with $|S_{\text{reg}}| = k$ and $S_{\text{free}} = O(\log n / \varepsilon^2)$. To our help, we will have two basic routines COMPLETE SOLUTION and COMPLETE SEQUENCE:

- COMPLETE SOLUTION $_{f,u}(\mathcal{H})$ takes a partial solution $\mathcal{H}$ as input and returns a solution by running the while-loop of stage 1 in LOGADAPTIVE using the opening cost f and the distances u for free facilities already opened.
- COMPLETE SEQUENCE $_{f,u}(\mathcal{H} = (H_1, \dots, H_{p-1}), H_p = \langle h_1, \dots, h_\ell \rangle)$ where $\mathcal{H}$ is partial solution and H_p is an η -valid sequence except that it may not be maximal. COMPLETE SEQUENCE then runs the while-loop of stage 1 in LOGADAPTIVE starting with H_p to return a maximal sequence whose prefix is H_p (with respect to opening cost f and distances u).

We note that all facilities added by COMPLETE SOLUTION and COMPLETE SEQUENCE are 0-openable and regular. Additionally, for intuition, observe that, with the above notation, the set of facilities opened by COMPLETE SOLUTION $_{f,u}(\emptyset)$ is equivalent to that opened by LOGADAPTIVE. (There is no free facility so u is meaningless here.) To simplify notation, we only use COMPLETE SEQUENCE when the partial solution $\mathcal{H}$ is clear from the context, so we simply write COMPLETE SEQUENCE $_{(f,u)}(\langle h_1, \dots, h_\ell \rangle)$. Furthermore, we omit (f, u) when the parameters are clear from the context.

3.5 MERGESOLUTIONS

We now present our procedure MERGESOLUTIONS that *walks* between two solutions. We will maintain two solutions $(\mathcal{H}, f, u)$ and $(\mathcal{H}', f', u')$ whose numbers of open regular facilities *sandwich* k . (Formally, let us say a and b sandwich k if $a < k < b$ or $b < k < a$.) We will gradually make them closer until one of them opens exactly k regular facilities. We start from the two initial solutions obtained using the standard method. Note that the minimum nonzero distance is 1 and the maximum distance is $M = \text{poly}(n)$, which implies that $\text{opt} \leq nM$.

Initialization. We consider executions of $\text{COMPLETESOLUTION}_{f,u}(\emptyset)$ with empty u . When $f = 1/n^2$, every client $j \in D$ satisfies $d(j, S) = 0$ if $d(j, F) = 0$ and $d(j, S) \leq d(j, F) + 2/n^2$ otherwise, as each client can open its closest facility by itself when $\alpha_j = d(j, F) + 2/n^2$. If this solution opens at most k centers, then it is already an $(1 + 2/n)$ -approximation and we return it. On the other hand, when $f = 4nM$, the algorithm will open exactly one facility, since the first facility is open when $\theta \geq 2M$, so every client becomes immediately inactive when it is open. We then perform a binary search in the interval $[1/n^2, 4nM]$ of values of f , keeping the invariant that for the current research interval $[f, f']$ the corresponding two solutions $(\mathcal{H}, f, u)$ and $(\mathcal{H}', f', u)$ are such that $(\mathcal{H}, f, u)$ opens less than k facilities and $(\mathcal{H}', f', u)$ opens more than k facilities. We halt the process when $f \leq f' \leq f + \eta$.

General Step. MERGESOLUTIONS works on phase $p = 1, 2, \dots$ iteratively, and ensures that the two solutions are identical up to the p -th phase. Formally, MERGESOLUTIONS takes two solutions $(\mathcal{H}, f, u)$ and $(\mathcal{H}', f', u')$ as well as the current phase p as input. The following promises on them will be satisfied whenever we call MERGESOLUTIONS.

- Both solutions consist of η -valid sequences.
- The numbers of open regular facilities by the two solutions sandwich k .
- Exactly one parameter, which we call the *difference parameter*, differs by at most η between the two solutions, i.e., if $f \neq f'$ then $|f - f'| \leq \eta$ and $u(\tilde{i}) = u'(\tilde{i})$ for every $\tilde{i} \in F_{\text{free}}$; otherwise if $f = f'$ there is exactly one free facility $\tilde{i} \in F_{\text{free}}$ so that $u(\tilde{i}) \neq u'(\tilde{i})$ and $|u(\tilde{i}) - u'(\tilde{i})| \leq \eta$.
- $\mathcal{H} = (H_1, H_2, \dots, H_L)$ and $\mathcal{H}' = (H'_1, H'_2, \dots, H'_L)$ have a common prefix of length $p - 1$, i.e., $H_i = H'_i$ for $i = 1, 2, \dots, p - 1$. Furthermore, the remaining sequences $H_p, \dots, H_L$ and $H'_p, \dots, H'_L$ only contain regular facilities and are 0-valid with respect to the parameters f, u and f', u' , respectively. (This implies that $\mathcal{H}$ and $\mathcal{H}'$ open the same set of free facilities.)

Given the promises, the goal of MERGESOLUTIONS is to

- (1) either find a solution of η -valid sequences that opens exactly k regular facilities and $O_\epsilon(\log n)$ free facilities, or
- (2) produce two new solutions that satisfy the above promises for phase $p + 1$.

Since the maximum distance between any two points is $M \leq \text{poly}(n)$ and $f \leq 4nM$, before θ reaches $6M$, at least one facility will open and every client will become inactive. Therefore, for some threshold $p^* = O(\log n / \epsilon^2)$, once $\mathcal{H}$ and $\mathcal{H}'$ agree on the first p^* phases, they open exactly the same set of facilities which violates the promise that the numbers of open regular facilities sandwich k . Therefore, case (1) must happen for some $p \leq p^*$, which yields a solution of η -valid sequences that opens exactly k regular facilities and $O(\log n)$ free facilities. Combined with Theorem 3.10, this proves Theorem 1.2. Indeed, we then have a solution consisting of open centers S with $|S| = k + O(\log(n)/\epsilon^2)$ such that, by Theorem 3.10,

$$\sum_{j \in D} d(j, S) \leq \frac{2}{1 - \delta} \cdot (\text{opt}_{\text{LP}}(f) - (f - \eta n) \cdot k) \leq (2 + O(\epsilon)) \text{opt},$$

where, for the last inequality, we used that $\eta = 2^{-n}$ and $\text{opt} \geq 1$ (since any distance is at least 1). Theorem 1.2 then follows by scaling ϵ appropriately.

In the remainder of this section, we present MERGESOLUTIONS for two solutions and a given phase p . It consists of three subroutines, presented in Section 3.5.1 3.5.2, and 3.5.3 respectively. Each

subroutine achieves (1) or (2) above (which ends the whole procedure or lets us move on to the next phase), or sets up the stage for the next subroutine. The final third subroutine always achieves (1) or (2).

3.5.1 Making Parameters Identical

In this subsection, we try to make the parameters of the two solutions exactly identical. By renaming, we assume that if $f \neq f'$ is the difference parameter then $f' < f$, and if the difference parameter is $u(\tilde{i}) \neq u'(\tilde{i})$ then $u(\tilde{i}) < u'(\tilde{i})$.

Claim 2. $H'_1, H'_2, \dots, H'_p$ are η -valid sequences with respect to parameters f, u .

Proof. Suppose first that $f \neq f'$ is the difference parameter. As the first $p-1$ sequences are identical, we have that $H'_1, \dots, H'_{p-1}$ are η -valid with respect to parameters f, u . We further have that H'_p is η -valid with respect to parameters f, u since it was 0-valid with respect to parameters f', u' and the only difference is that f is slightly bigger than f' by at most η . For maximality, we have by Lemma 3.11 that there is no payable facilities after opening H'_p with respect to opening cost f' and increasing the opening cost f cannot introduce any new payable facilities, so H'_p is also maximal with respect to f .

Now assume that $u(\tilde{i}) \neq u'(\tilde{i})$ is the difference parameter. Then by the same arguments as above, we have $H'_1, \dots, H'_{p-1}$ are η -valid with respect to f, u . Moreover, H'_p was 0-valid with respect to f', u' and we have that $u'(\tilde{i}) - \eta \leq u(\tilde{i}) \leq u'(\tilde{i})$. It remains to show that H'_p is η -valid with respect to f, u in this case. The maximality of H'_p with respect to f, u is implied by Lemma 3.11 by the same arguments as above when $f \neq f'$: since lowering $u(\tilde{i}) \leq u'(\tilde{i})$ only decreases the payments toward opening facilities and, hence, it does not introduce any new payable facilities. We proceed to show that the facilities remain η -openable.

Let $H'_p = \langle h_1, \dots, h_\ell \rangle$ and consider h_t for some $t \in [\ell]$, which is 0-openable with respect to $f', u', \theta = (1 + \varepsilon^2)^{p-1}$ and some superset $S' \supseteq (\cup_{q \in [p-1]} H_q) \cup \{h_1, \dots, h_{t-1}\}$. When we change from f', u' to f, u , since the only difference is $u(\tilde{i}) < u'(\tilde{i})$, the set of active clients A' for f', u' is a superset of the set of active clients A for f, u . Also, let $d'(\cdot, \cdot)$ and $d(\cdot, \cdot)$ be the distances with respect to u' and u respectively. Based on the fact that h_t was 0-openable with respect to the former setting with bids $(\tau'_j)_{j \in A'}$, let $(\tau_j)_{j \in A}$ be the new bids defined as $\tau_j = \min(\tau'_j, (1 - \delta)d(j, S))$ for $j \in A$. Let us show that h_t was η -openable with respect to the latter setting with bids $(\tau_j)_{j \in A}$ by checking the conditions in Definition 3:

- $\tau_j = \alpha_j$ for every $j \in A - B(h_t, \varepsilon\theta)$: Since it was true for $A' \supseteq A$, it remains true.
- $\alpha_j \leq \tau_j \leq \min\{(1 - \delta)d(j, S), (1 + \varepsilon^2)\theta\}$ for every $j \in A \cap B(h_t, \varepsilon\theta)$: Fix $j \in A \cap B(h_t, \varepsilon\theta)$. Since $A \subseteq A'$, $\tau'_j \leq \min\{(1 - \delta)d'(j, S), (1 + \varepsilon^2)\theta\}$. Since τ_j is defined as $\min(\tau'_j, (1 - \delta)d(j, S))$, the condition is satisfied. (Note that $j \in A$ implies $\alpha_j \leq \min(\tau'_j, (1 - \delta)d(j, S))$ as well.)
- $\sum_{j \in A} [\tau_j - (1 - \delta)d(h_t, j)]^+ + (1 - \delta) \sum_{j \in I} [d(j, S) - d(h_t, j)]^+ \geq \hat{f} - n\eta$: Let us argue that the contribution of each client j to the left-hand side changes by at most η between (A', d', τ') and (A, d, τ) . For $j \in A$, note that $\tau_j < \tau'_j$ implies $\tau_j = (1 - \delta)d(j, \tilde{i}) \geq (1 - \delta)(d'(j, \tilde{i}) - \eta) \geq \tau'_j - \eta$, so $\tau_j \geq \tau'_j - \eta$ in any case. For $j \in I'$, note that $d(j, S)$ and $d'(j, S)$ only differ by at most η . Finally for $j \in A' \cap I$, $(1 - \delta)d(j, S) \geq \tau_j \geq \tau'_j - \eta$. Therefore, the fact that this condition was satisfied for (A', d', τ') with $\hat{f}$ on the right-hand side implies that (A, d, τ) satisfies this condition with $\hat{f} - n\eta$ on the right-hand side.
- For every regular facility i_0 and $\ell \in A$, $\sum_{j \in A} [\tau_j - d(i_0, j)]^+ + \sum_{j \in I} [\tau_\ell - 2d(j, i_0) - d(\ell, i_0)]^+ \leq \hat{f}$: Since $\ell \in A$ implies $\ell \in A'$, we have

$$\sum_{j \in A'} [\tau'_j - d(i_0, j)]^+ + \sum_{j \in I'} [\tau'_\ell - 2d(j, i_0) - d(\ell, i_0)]^+ \leq \hat{f}.$$

(Since i_0 is a regular facility, $d(i_0, j)$ for a client j does not change by changing $u(\tilde{i})$.) For $j \in A' \cap I$, since $j \in I$ and $\ell \in A$, one has

$$\tau'_j \geq \alpha_j \geq d(j, S) \geq d(\ell, S) - d(\ell, j) \geq \tau_\ell - d(j, i_0) - d(\ell, i_0).$$

Thus $\tau'_j - d(i_0, j) \geq \tau_\ell - 2d(j, i_0) - d(\ell, i_0)$. Therefore we have

$$\sum_{j \in A} [\tau_j - d(i_0, j)]^+ + \sum_{j \in I} [\tau_\ell - 2d(j, i_0) - d(\ell, i_0)]^+ \leq \hat{f}.$$

□

Outcome of this section of MERGESOLUTIONS. Let

$$\mathcal{H}'' = \text{COMPLETESOLUTION}_{(f,u)}(H'_1, \dots, H'_p),$$

which is a valid call by Claim 2. If $\mathcal{H}''$ opens k regular facilities, we return this solution. Otherwise, as $(\mathcal{H}, f, u)$ and $(\mathcal{H}', f', u')$ sandwich k , we must have that either $(\mathcal{H}, f, u)$ and $(\mathcal{H}'', f, u)$, or $(\mathcal{H}'', f, u)$ and $(\mathcal{H}', f', u')$ sandwich k . On the one hand, if $(\mathcal{H}'', f, u)$ and $(\mathcal{H}', f', u')$ sandwich k , then we made progress in that we now have two solutions with a common prefix of length p (instead of $p-1$) and only one parameter differs by at most η . In this case, we are done with phase p and can continue MERGESOLUTIONS with these two solutions and phase $p+1$.

On the other hand, if $(\mathcal{H}, f, u)$ and $(\mathcal{H}'', f, u)$ sandwich k , then these two solutions have the same parameters and their first η -valid sequences that differ are H_p and H'_p . In this case, we rename $\mathcal{H}''$ to $\mathcal{H}'$ and proceed to the next step.

3.5.2 Maximize Common Prefix of H_p and H'_p

We are given two solutions $(\mathcal{H}, f, u)$ and $(\mathcal{H}', f, u)$ on the same parameters f, u that sandwich k . Moreover, if we let $\mathcal{H} = (H_1, \dots, H_L)$ and $\mathcal{H}' = (H'_1, \dots, H'_L)$, then $H_i = H'_i$ for $i = 1, \dots, p-1$ and $H_p \neq H'_p$. In addition H_p and H'_p only contain regular facilities.

In this step of MERGESOLUTIONS, we increase the common prefix of H_p and H'_p by “slowly” modifying H'_p . Recall that H_p only contains regular facilities: let $H_p = \langle i_1, i_2, \dots, i_\ell \rangle$. Moreover, as the parameters (f, u) are clear from the context, we omit them in the following when, e.g., calling the procedures COMPLETESOLUTION and COMPLETESEQUENCE.

Transforming H'_p step-by-step so that H_p becomes a prefix of H'_p .

Repeat the following until all of H_p becomes a prefix of H'_p .

- Let q be the largest index so that $H'_p = \langle i_1, i_2, \dots, i_q, h'_{q+1}, \dots, h'_{\ell_1} \rangle$, i.e., q equals the length of the common prefix $i_1, i_2, \dots, i_q$ of H_p and H'_p .
- Update H'_p by inserting a free copy of $i_{q+1}, \tilde{i}_{q+1}$, at the end of H'_p with $u(\tilde{i}_{q+1}) = 0$. That is, we get the sequence

$$H'_p = \langle i_1, i_2, \dots, i_q, h'_{q+1}, \dots, h'_{\ell_1}, \tilde{i}_{q+1} \rangle.$$

- For $t = q+1, \dots, \ell_1$,
 - Update H'_p by first removing h'_t and then completing the sequence by a call to COMPLETESEQUENCE. More explicitly, if

$$H'_p = \langle i_1, \dots, i_q, h'_t, h'_{t+1}, \dots, h'_{\ell_1}, \tilde{i}_{q+1}, h'_{\ell_1+1}, \dots, h'_{\ell_2} \rangle$$

then we delete h'_t to obtain

$$H''_p = \langle i_1, \dots, i_q, h'_{t+1}, \dots, h'_{\ell_1}, \tilde{i}_{q+1}, h'_{\ell_1+1}, \dots, h'_{\ell_2} \rangle$$

and finally update $H'_p = \text{COMPLETESEQUENCE}(H''_p)$ which will be of the form

$$\langle i_1, \dots, i_q, h'_{t+1}, \dots, h'_{\ell_1}, \tilde{i}_{q+1}, h'_{\ell_1+1}, \dots, h'_{\ell_2}, \dots, h'_{\ell_3} \rangle,$$

where $h'_{\ell_2+1}, \dots, h'_{\ell_3}$ are the facilities added by COMPLETESEQUENCE (which could be empty).

- After removing $h'_{q+1}, \dots, h'_{\ell_1}$, we have

$$H'_p = \langle i_1, i_2, \dots, i_q, \tilde{i}_{q+1}, h'_x, \dots, h'_y \rangle.$$

We update H'_p by replacing $\tilde{i}_{q+1}$ with its regular copy i_{q+1} , i.e., we obtain

$$H'_p = \langle i_1, i_2, \dots, i_q, i_{q+1}, h'_x, \dots, h'_y \rangle.$$

We have the following claims.

Claim 3. Throughout the updates to $H'_p, H'_1, \dots, H'_p$ remains an η -valid sequence, and H'_p contains at most one free facility.

Proof. As we never update f or the value $u(\cdot)$ of a free facility in $H'_1, \dots, H'_{p-1}$, these sequences remain η -valid.

We continue to argue that H'_p remains η -valid throughout its updates. By Claim 2, this is true before any updates to H'_p . We now analyze the three types of updates to H'_p .

- Updating H'_p by inserting a free copy $\tilde{i}_{q+1}$ of i_{q+1} maintains that it is η -valid as every regular facility stays η -openable in the sequence (they have the same prefix as before) and the sequence remains maximal (by Lemma 3.11 as adding a free facility does not make any new facility payable).
- Updating H'_p by removing h'_t and calling COMPLETESEQUENCE . Removing a h'_t can only decrease the prefix of regular facilities in the sequence, and so each regular facility still has a (superset) of facilities for which it is η -openable. Finally, the call COMPLETESEQUENCE ensures that the sequence is maximal.
- Updating H'_p by replacing $\tilde{i}_{q+1}$ with its regular copy i_{q+1} . First note that, since $u(\tilde{i}_{q+1}) = 0$, this does not impact on the following regular facilities being η -openable (with respect to a superset). By the same argument, the sequence stays maximal (as $\tilde{i}_{q+1}$ with $u(\tilde{i}_{q+1}) = 0$ and i_{q+1} has the same impact on following facilities). Finally, to see that i_{q+1} is η -openable (with respect to a potential superset) notice that the prefix $i_1, \dots, i_q$ is the same as that in H_p which is an η -valid sequence.

We complete the proof of the claim by arguing that H'_p contains at most one free facility at any point of time. This follows from the facts that (1) H'_p initially contains no free facility, (2) in one repetition, the free facility $\tilde{i}_{q+1}$ is inserted into H'_p , and (3) $\tilde{i}_{q+1}$ is replaced by its regular copy i_{q+1} before continuing to the next repetition. $\square$

Outcome of this section of MERGESOLUTIONS. We distinguish three cases.

Case 1. If at some point H'_p is such that $\text{COMPLETESOLUTION}(H'_1, H'_2, \dots, H'_p)$ opens exactly k regular facilities, then we return that solution.

Case 2. Otherwise, suppose there is an update to H'_p so that if we let H_p^{before} and H_p^{after} denote H'_p before and after this update, respectively, then $\text{COMPLETE_SOLUTION}(H_1, H_2, \dots, H_p^{\text{before}})$ and $\text{COMPLETE_SOLUTION}(H_1, H_2, \dots, H_p^{\text{after}})$ sandwich k . There are three kinds of updates to H'_p . However, note that the update where $\tilde{i}_{q+1}$ is replaced by its regular copy i_{q+1} cannot satisfy the conditions of this case. Indeed, as $u(\tilde{i}_{q+1}) = 0$, we have

$$\text{COMPLETE_SOLUTION}(H_1, \dots, H_p^{\text{before}}) = \text{COMPLETE_SOLUTION}(H_1, \dots, H_p^{\text{after}})$$

in this case, which contradicts that they sandwich k .

We proceed to analyze the two other kind of updates. If the update to H'_p is to insert a free facility $\tilde{i}_{q+1}$ with $u(\tilde{i}_{q+1}) = 0$. Then

$$\begin{aligned} H_p^{\text{before}} &= \langle i_1, i_2, \dots, i_q, h'_{q+1}, \dots, h'_{\ell_1} \rangle \\ H_p^{\text{after}} &= \langle i_1, i_2, \dots, i_q, h'_{q+1}, \dots, h'_{\ell_1}, \tilde{i}_{q+1} \rangle \end{aligned}$$

Note that the solution $\text{COMPLETE_SOLUTION}(H_1, H_2, \dots, H_p^{\text{before}})$ is equivalent to $\text{COMPLETE_SOLUTION}(H_1, H_2, \dots, H_p^{\text{after}})$ if we set $u(\tilde{i}_{q+1}) = 10M$ (except for the free facility $\tilde{i}_{q+1}$). We can thus do binary search on $u(\tilde{i}_{q+1})$ to find two values $u'(\tilde{i}_{q+1})$ and $u(\tilde{i}_{q+1})$ with $|u'(\tilde{i}_{q+1}) - u(\tilde{i}_{q+1})| \leq \eta$ so that the solutions $\text{COMPLETE_SOLUTION}_{(f, u')}(H_1, \dots, H_p^{\text{after}})$ and $\text{COMPLETE_SOLUTION}_{(f, u)}(H_1, \dots, H_p^{\text{after}})$ sandwich k (where u and u' are identical except for $\tilde{i}_{q+1}$). We have thus finished phase p , and we can restart MERGE_SOLUTIONS with the two solutions and phase $p+1$, since they have the first p , instead of $p-1$, η -valid sequences in common and exactly one difference parameter. Their maximality follows from that of H_p^{before} .

Finally, consider when the update to H'_p is of the second type, i.e., $u(\tilde{i}_{q+1}) = 0$ and

$$\begin{aligned} H_p^{\text{before}} &= \langle i_1, \dots, i_q, h'_t, h'_{t+1}, \dots, h'_{\ell_1}, \tilde{i}_{q+1}, h'_{\ell_1+1}, \dots, h'_{\ell_2} \rangle \\ H_p^{\text{after}} &= \langle i_1, \dots, i_q, h'_{t+1}, \dots, h'_{\ell_1}, \tilde{i}_{q+1}, h'_{\ell_1+1}, \dots, h'_{\ell_2}, \dots, h'_{\ell_3} \rangle \end{aligned}$$

In this case, $\text{COMPLETE_SOLUTION}(H_1, \dots, H_p^{\text{before}})$ and $\text{COMPLETE_SOLUTION}(H_1, \dots, H_p^{\text{after}})$ are two solutions with the same parameters f, u , that sandwich k and we proceed to the next step with the following properties of H_p^{before} and H_p^{after} :

- Both sequences contain one free facility, $\tilde{i}_{q+1}$.
- H_p^{before} and H_p^{after} are identical except that (1) H_p^{before} contains a regular facility h'_t not present in H_p^{after} , and (2) H_p^{after} contains a (potentially empty) suffix of regular facilities not present in H_p^{before} .

Case 3. None of the previous cases apply. Then, completing the solution with the initial H'_p and the final H'_p cannot sandwich k . Since then (assuming the first case does not apply) there must be an update to H'_p that results in two solutions that sandwich k : this happens because the completion of H_p and the completion of the initial H'_p sandwich k . The above implies that if we let $H_p^{\text{before}} = H_p$ and H_p^{after} equal the final H'_p then $\text{COMPLETE_SOLUTION}(H_1, \dots, H_p^{\text{before}})$ and $\text{COMPLETE_SOLUTION}(H_1, \dots, H_p^{\text{after}})$ sandwich k . Further note that, since H_p^{after} equals H'_p after the final updates, H_p^{before} appears as a prefix of H_p^{after} and both sequences only contain regular facilities. To summarize, in this case, $\text{COMPLETE_SOLUTION}(H_1, \dots, H_p^{\text{before}})$ and $\text{COMPLETE_SOLUTION}(H_1, \dots, H_p^{\text{after}})$ are two solutions with the same parameters f, u , that sandwich k and we proceed to the next step with the following properties of H_p^{before} and H_p^{after} :

- Both sequences contain no free facilities.
- H_p^{before} and H_p^{after} are identical except that H_p^{after} contains a suffix of regular facilities not present in H_p^{before} .

3.5.3 Removing Extra Facilities

The input to this step consists of two solutions $\text{COMPLETE_SOLUTION}(H_1, \dots, H_p^{\text{before}})$ and $\text{COMPLETE_SOLUTION}(H_1, \dots, H_p^{\text{after}})$ that have the same parameters f, u and sandwich k . Moreover, H_p^{before} and H_p^{after} both contain at most one free facility and they are identical except for potentially two differences:

- H_p^{before} may contain a facility not present in H_p^{after} ;
- H_p^{after} contains a suffix of regular facilities (which may be empty) that is not present in H_p^{before} .

To simplify the notation in this section, we let i_* denote the regular facility in H_p^{before} that is not present in H_p^{after} if it exists (so i_* corresponds to h'_t in the previous section). We further let $h_1, \dots, h_\ell$ denote the suffix of H_p^{after} not present in H_p^{before} .

We now transform H_p^{after} into H_p^{before} one change at a time:

Generation of η -valid sequences from H_p^{after} to H_p^{before} .

- If H_p^{before} contains the regular facility i_* not present in H_p^{after} , then let $\tilde{i}_*$ be a free copy of i_* with $u(\tilde{i}_*) = 0$ and obtain H_p^0 from H_p^{after} by adding $\tilde{i}_*$ at the end. Otherwise, we let H_p^0 equal H_p^{after} .
- For $r = 1, \dots, \ell$, obtain H_p^r from H_p^{r-1} by removing h_r .

We remark that H_p^ℓ is identical to H_p^{before} since we removed the whole suffix $h_1, \dots, h_\ell$ present in H_p^{after} , except for potentially the facility i_* . That is, the only difference in H_p^{before} and H_p^ℓ may be the placement of $\tilde{i}_*$ in the sequence and the fact that $\tilde{i}_*$ is a free copy of the regular facility i_* present in H_p^{before} . However, since $u(\tilde{i}_*) = 0$, we have that the two sequences H_p^{before} and H_p^ℓ have an identical impact on COMPLETE_SOLUTION , which is a fact we will use in the proof of Claim 5.

Claim 4. For $r = 0, 1 \dots \ell$, H_p^r is an η -valid sequence with at most two free facilities.

Proof. The sequence H_p^r has at most two free facilities because H_p^{after} has at most one free facility, and we potentially add one new free facility $\tilde{i}_*$.

We now verify that H_p^r is η -valid. For any regular facility h' in the sequence, the prefix of H_p^r before h' is a subset of the prefix before h' in H_p^{after} . Thus, as h' was openable with respect to a superset $S(h')$ in H_p^{after} , h' must be η -openable with respect to the same superset $S(h')$ in H_p^r . This ensures that the first property of Definition 4 is satisfied. To verify maximality, notice that since we added $\tilde{i}_t$ with $u(\tilde{i}_t) = 0$ in the end, we have that the set of active clients after opening the facilities in H_p^r is a subset of the active clients after opening the facilities in H_p^{before} . Hence, the maximality of H_p^{before} implies the maximality of H_p^r (by Lemma 3.11 as any payable facility in H_p^r is also payable in H_p^{before}). $\square$

Claim 5. One of the following is true.

- For one of the sequences H_p^r , $\text{COMPLETE_SOLUTION}(H_1, H_2, \dots, H_p^r)$ opens k regular facilities.
- $\text{COMPLETE_SOLUTION}(H_1, H_2, \dots, H_p^{\text{after}})$ and $\text{COMPLETE_SOLUTION}(H_1, H_2, \dots, H_p^0)$ sandwich k .
- $\text{COMPLETE_SOLUTION}(H_1, H_2, \dots, H_p^r)$ and $\text{COMPLETE_SOLUTION}(H_1, H_2, \dots, H_p^{r+1})$ sandwich k for some $r \in \{0, 1, \dots, \ell - 1\}$.

Proof. Assume the first bullet point does not hold, i.e., the completion of no H_p^r leads to the opening of k regular facilities. Now notice that the second and third bullet points ask whether consecutive solutions sandwich k , starting with H_p^{after} and ending with H_p^ℓ . By definition, H_p^ℓ equals H_p^{before} except

potentially for the placement of $\tilde{i}_*$ and that $\tilde{i}_*$ is the free copy of i_* present in H_p^{before} . However, as $u(\tilde{i}_t) = 0$, the two sequences have an identical impact when we complete the solution, which implies

$$\text{COMPLETE_SOLUTION}(H_1, \dots, H_p^\ell) = \text{COMPLETE_SOLUTION}(H_1, \dots, H_p^{\text{before}}).$$

Thus, as $\text{COMPLETE_SOLUTION}(H_1, \dots, H_p^{\text{after}})$ and $\text{COMPLETE_SOLUTION}(H_1, \dots, H_p^{\text{before}})$ sandwich k , either the second or third bullet point of the statement must be valid (since no solution opened exactly k regular facilities). $\square$

Outcome of this section of MERGESOLUTIONS. We distinguish three cases based on the first bullet point that is true in Claim 5.

Case 1. If one of the sequences H_p^r is such that $\text{COMPLETE_SOLUTION}(H_1, H_2, \dots, H_p^r)$ opens k regular facilities, then we return that solution.

Case 2. Otherwise, if the solutions $\text{COMPLETE_SOLUTION}(H_1, H_2, \dots, H_p^{\text{after}})$ and $\text{COMPLETE_SOLUTION}(H_1, H_2, \dots, H_p^0)$ sandwich k , then H_p^0 is equal to H_p^{after} except that $\tilde{i}_p$ was added in the end of H_p^{after} with $u(\tilde{i}_p) = 0$. Note that the solution $\text{COMPLETE_SOLUTION}(H_1, H_2, \dots, H_p^{\text{after}})$ is equivalent to $\text{COMPLETE_SOLUTION}(H_1, H_2, \dots, H_p^0)$ if we set $u(\tilde{i}_{q+1}) = 10M$ (except the presence of $\tilde{i}_*$). We can thus do binary search on $u(\tilde{i}_*)$ to find two values $u'(\tilde{i}_*)$ and $u(\tilde{i}_*)$ with $|u'(\tilde{i}_*) - u(\tilde{i}_*)| \leq \eta$ so that the two solutions $\text{COMPLETE_SOLUTION}_{(f,u')}(H_1, \dots, H_p^0)$ and $\text{COMPLETE_SOLUTION}_{(f,u)}(H_1, \dots, H_p^0)$ sandwich k (where u and u' are identical except for $\tilde{i}_{q+1}$). We have thus finished phase p , and we can restart MERGESOLUTIONS with the two solutions and phase $p + 1$, since they have the first p , instead of $p - 1$, η -valid sequences in common and exactly one difference parameter.

Case 3. Finally, consider the case when none of the above cases apply and thus (by Claim 5) there is an $r \in \{0, 1, \dots, \ell - 1\}$ so that $\text{COMPLETE_SOLUTION}(H_1, H_2, \dots, H_p^r)$ and $\text{COMPLETE_SOLUTION}(H_1, H_2, \dots, H_p^{r+1})$ sandwich k . The only difference between H_p^r and H_p^{r+1} is that H_p^r contains h_{r+1} which is not present in H_p^{r+1} . Let H_p be the sequence obtained by adding a free copy $\tilde{i}_{r+1}$ of h_{r+1} at the end of H_p^{r+1} (which clearly maintains that is η -valid), then

$$\text{COMPLETE_SOLUTION}(H_1, H_2, \dots, H_p) = \text{COMPLETE_SOLUTION}(H_1, H_2, \dots, H_p^r) \text{ if } u(\tilde{i}_{r+1}) = 0$$

and

$$\text{COMPLETE_SOLUTION}(H_1, H_2, \dots, H_p) = \text{COMPLETE_SOLUTION}(H_1, H_2, \dots, H_p^{r+1}) \text{ if } u(\tilde{i}_{r+1}) = 10M.$$

Hence, doing a binary search on $u(\tilde{i}_{r+1})$ as in the previous case finishes phase p . We can restart MERGESOLUTIONS with the two solutions and phase $p + 1$, since they have the first p , instead of $p - 1$, η -valid sequences in common and exactly one difference parameter.

4 $(2 + \varepsilon)$ -Approximation for $(\frac{\zeta}{\log n})$ -Stable Instances

In this section, we give a $(2 + \varepsilon)$ -approximation algorithm for $(\frac{\zeta}{\log n})$ -stable k -median instances. The proof of the theorem is presented in Section 4.7. We restate the theorem for the reader's convenience.

Theorem 1.3. *For any two constants $\varepsilon, \zeta > 0$, there exists a randomized polynomial-time algorithm that, given a $(\zeta / \log n)$ -stable k -median instance, returns a solution of cost at most $(2 + \varepsilon)\text{opt}$ with high probability.*

To simplify notation, we let ε be the minimum of ε and ζ in the above theorem and further assume that $\varepsilon < 1/12$ is sufficiently small. With this notation, we present a $(2 + O(\varepsilon))$ -approximation algorithm for $\varepsilon / \log(n)$ -stable instances. This implies the above theorem as any $\zeta / \log(n)$ -stable instance is also $\zeta' / \log(n)$ -stable with $\zeta' \leq \zeta$.

The algorithm and the proof of the above theorem follow several steps where we iteratively simplify the instance by adding more structure. Finally, we obtain enough structure to solve the problem by maximizing a submodular function subject to a matroid partition constraint. We have not optimized constants in favor of simplifying the description, and throughout this section we let $O_\varepsilon(\log n)$ denote $O(\log(n)/\varepsilon^{O(1)})$. An overview of this section and the algorithm is as follows:

Overview of 2-Approximation for k -median on $(\varepsilon/\log n)$ -Stable Instance k, D, F, d .

1. We start, in Section 4.1, by running a standard Local Search on (k, D, F, d) to obtain a set of centers S . It is well-known that S is a constant-factor approximation. Moreover, for a stable instance, we show that S “correctly” implicitly identifies all but $O_\varepsilon(\log n)$ centers of OPT. Specifically, this ensures that all but $O_\varepsilon(\log n)$ clusters have the same cost as in OPT up to a $(1 + O(\varepsilon))$ factor.
2. In Section 4.2, we use a technique inspired from the classic k -means++ algorithm and its so called D^2 -sampling procedure to sample clients proportional to their cost in solution S , and we show that if we take $O_\varepsilon(\log n)$ samples W then with good probability we “hit” all but an insignificant fraction of clusters of high cost in S .
3. In Section 4.3, we then, guess a subset of the sampled clients W so that each hit cluster has exactly one client, which we refer to as the leader of that cluster. In addition, for each client ℓ that is a leader, we guess the (approximate) distance from ℓ to the optimal center in the optimal cluster it hits. This defines a set $\mathcal{B}$ of balls with the guarantee that the optimal solution opens one center in each of these balls. Moreover, if we let $\hat{\text{OPT}}$ be the optimal centers corresponding to these balls, we prove in Section 4.3.1 that there is a swap of centers $M_O = S - S_0 \cup \hat{\text{OPT}}$ that removes S_0 and adds the optimal centers $\hat{\text{OPT}}$ so that the cost of the solution M_O is good and in particular a $2 + O(\varepsilon)$ -approximation.
4. The remaining steps are then focused on approximating this swap, i.e., intuitively to that of finding S_0 and $\hat{\text{OPT}}$:
 - (a) We first find the set $\mathcal{Q}$ of expensive centers of S_0 so that the centers $S_0 - \mathcal{Q}$ has total cost $O(\varepsilon \text{opt})$ in Section 4.4. This allows us to think of the centers in $S_0 - \mathcal{Q}$ as contracted, i.e., as single weighted points, and in particular, that all clients of these clusters are assigned to the same center when their center in S_0 is removed. We then, in Section 4.4.1, use this structure to partition $S_0 - \mathcal{Q}$ into two sets $\mathcal{U}$ and $\mathcal{R}$ where $\mathcal{U}$ are those centers that are reassigned to other centers in $S - S_0$ and $\mathcal{R}$ are those centers that are assigned to $\hat{\text{OPT}}$.
 - (b) The set $\mathcal{U}$ is then approximated with a set $\tilde{\mathcal{U}}$ (with close to identical properties) in Section 4.5.
 - (c) At this stage, we have thus “guessed” all centers $\mathcal{Q} \cup \tilde{\mathcal{U}}$ to remove from S except for those in $\mathcal{R}$. This is achieved in Section 4.6 where we reduce the problem of both finding $\mathcal{R}$ and an approximate version of $\hat{\text{OPT}}$ to that of submodular function optimization subject to a partition matroid (since we should open one center per ball in $\mathcal{B}$).

We remark that in each of Steps 3, 4a, 4b, and 4c, we will make guesses by enumerating polynomially $n^{\varepsilon^{-O(1)}}$ many potential choices. In the following, we describe the algorithm and its analysis by presenting each of these steps. When presenting the next step, we analyze the branch of the algorithm that took the successful guesses up to that point. The algorithm outputs the solution of the smallest cost among the $n^{\varepsilon^{-O(1)}}$ constructed solutions obtained by enumerating all possible guesses (see Section 4.7 for a more formal treatment where we put everything together). In particular, the final solution will have cost at most the solution that is output in the analyzed branch of “correct” guesses.

We end this section by introducing basic definitions and concepts, followed by the definition of matroids and submodular functions. We also state the known result, an $(1 - 1/e)$ -approximation algorithm for maximizing monotone submodular functions subject to a matroid constraint, that we use in the last step (Section 4.6).

Definitions of leaders. We use the following definitions throughout this section. Given a set of centers $A \subseteq F$, we let $\text{cost}(A) := \sum_{p \in D} d(p, A)$ denote the cost of the corresponding clustering (where each client p is assigned to the closest center in A and pays the corresponding distance). Sometimes we will consider a (possibly suboptimal) assignment $\mu : D \rightarrow A$ of clients to centers in A , and let $\text{cost}(A, \mu) := \sum_{p \in D} d(p, \mu(p))$ be the corresponding cost. Let OPT be a fixed optimum solution for the k -median instance, and let $\text{opt} = \text{cost}(\text{OPT})$ be its cost. For a client $p \in D$, let $\text{OPT}_p := d(p, \text{OPT})$ be the cost paid by p in OPT .

Let C^* be an OPT cluster with center $c^* \in \text{OPT}$. We let $\text{avg}_{C^*, \text{OPT}}$ be the distance from c^* to the $\varepsilon|C^*|$ th closest client of C^* to c^* . We further let C_{avg}^* be the set of clients of C^* at a distance at most $\text{avg}_{C^*, \text{OPT}}$ from c^* , we refer to these clients as *leaders* for C^* . So $\text{avg}_{C^*, \text{OPT}}$ is the *maximum distance* from a leader to its center c^* . Also note that we have, $|C_{\text{avg}}^*| \geq \varepsilon|C^*|$. We thus have that $\text{avg}_{C^*, \text{OPT}}$ is upper bounded by $\frac{1}{1-\varepsilon}$ times the average cost $\frac{1}{|C^*|} \sum_{p \in C^*} d(p, c^*)$, and at the same time C_{avg}^* contains a constant fraction of the clients.

Partition matroids. In Section 4.6, we formulate a submodular function that we then maximize over a partition matroid using known results. We define those concepts and also state the result that we will use. First, recall the definition of a matroid and partition matroid. A *matroid* is a tuple $(E, \mathcal{I})$ defined on a ground set E with a family of independent sets $\mathcal{I}$ that satisfy: (1) if $A \in \mathcal{I}$ and $A' \subseteq A$, then $A' \in \mathcal{I}$; and (2) if $A, B \in \mathcal{I}$ and $|A| < |B|$, then there exists an element $e \in B - A$ such that $A \cup \{e\} \in \mathcal{I}$. In the special case of a *partition matroid*, the ground set E is partitioned into disjoint subsets $E_1, E_2, \dots, E_k$, and there are non-negative integers $r_1, r_2, \dots, r_k$ such that:

- A subset $I \subseteq E$ is independent if and only if $|I \cap E_i| \leq r_i$ for each $i = 1, 2, \dots, k$.

In other words, each subset E_i has a capacity limit r_i , and an independent set I cannot contain more than r_i elements from E_i . To work with matroids (to get a running time that is polynomial in the size of the ground set E), we often use *independence queries*, which allow us to determine if a given subset $A \subseteq E$ is independent (i.e., whether $A \in \mathcal{I}$). Specifically, an independence query on a subset A returns true if A is independent and false otherwise.

Submodular functions. Having defined matroids, we proceed to define non-negative monotone submodular functions. For a finite ground set E , a set function $f : 2^E \rightarrow \mathbb{R}$ is *submodular* if it satisfies the diminishing returns property: for every pair of sets $A \subseteq B \subseteq E$ and any element $e \in E - B$, it holds that

$$f(A \cup \{e\}) - f(A) \geq f(B \cup \{e\}) - f(B).$$

Intuitively, this means that adding an element e to a smaller set A provides at least as much additional value as adding e to a larger set B . In addition, f is called *monotone* if for any pair of sets $A \subseteq B \subseteq E$, we have $f(A) \leq f(B)$. In other words, adding elements to a set does not decrease the function value. Finally, f is *non-negative* if $f(A) \geq 0$ for all $A \subseteq E$.

A celebrated result [CCPV11] gives a polynomial-time $(1 - 1/e)$ -approximation algorithm for the problem of maximizing a non-negative monotone submodular function over a matroid constraint. While the original result was a randomized algorithm, a recent striking result [BF24], building upon the work of [FW14], gives a deterministic algorithm with the same guarantee. We remark that polynomial time here refers to a running time that is polynomial in the size of the ground set E . To summarize, they show the following theorem.

Theorem 4.1. *Let $f : 2^E \rightarrow \mathbb{R}$ be a non-negative monotone submodular function that we can evaluate in polynomial time, and let $(E, \mathcal{I})$ be a matroid on the same groundset, for which we can answer independence queries in polynomial time. Then, for every $\zeta > 0$, there is a polynomial-time algorithm that outputs a set $X \subseteq E$ with $X \in \mathcal{I}$ so that*

$$f(X) \geq (1 - 1/e - \zeta) \cdot \max_{X^* \in \mathcal{I}} f(X^*).$$

4.1 Properties of a Locally Optimal Solution

Let S be a locally optimal solution output by the following standard local search algorithm⁴. We remark that its running time is polynomial since by Lemma 1.4 the cost of a solution is a polynomially bounded integer and, at each improving step, the cost decreases by at least one.

localSearch($\cdot$)

```

1:  $S \leftarrow$  arbitrary solution for  $k$ -Median
2: while there exists a solution  $S'$  such that  $|S \Delta S'| \leq 2$  and  $\text{cost}(S') < \text{cost}(S)$  do
3:    $S \leftarrow S'$ 
4: return  $S$ 

```

Let S be the solution returned by *localSearch*($\cdot$). Local search achieves a 5 approximation for k -median, see e.g. Chapter 9.2 in [WS11].

Lemma 4.2. $\text{cost}(S) \leq 5 \cdot \text{opt}$.

In the remaining part of the algorithm and its analysis we fix S , and recall that we fixed the reference optimal solution OPT . We distinguish different types of clusters of OPT and analyze their properties in the next two sections. We let S_p be the cost paid by p in S and recall that OPT_p denotes the cost $p \in D$ pays in OPT .

4.1.1 Pure Clusters

We say that a cluster C^* of OPT is *pure* (w.r.t. S) if there exists a cluster C' of S such that $|C' \Delta C^*| \leq 3\epsilon \min(|C^*|, |C'|)$. In which case we also say that C' is pure w.r.t. OPT . We say that C' and C^* are associated. The following lemma follows from the fact that the input is β -stable for $\beta := \epsilon / \log n$ and that the solution S is a 5-approximation by Lemma 4.2. Given a client or center a and a distance r , we let $B(a, r)$ be the set of clients and centers at distance at most r from a . This lemma is originally proved in [CS17], we include a proof for completeness. In this proof we use in a few places the assumption that $\epsilon > 0$ is upper bounded by a small enough constant.

Lemma 4.3 (Restatement from [CS17], Lemma IV.4). *The number k_{imp} of clusters of OPT that are not pure w.r.t. S (and thus the number of clusters of S which are not pure w.r.t. OPT) is at most $\frac{\log n}{\epsilon^3}$.*

Proof. We let C_o^* denote the optimal cluster for each optimal center $o \in \text{OPT}$ and we let C_c denote the cluster in S corresponding to center $c \in S$. We refine OPT in steps so that the final set only consists of centers of pure clusters.

We start by removing expensive clusters in OPT . Specifically, let

$$\widetilde{\text{OPT}} = \{o \in \text{OPT} \mid \sum_{p \in C_o^*} d(p, o) \geq \frac{5\epsilon^3}{\log n} \text{opt}\}.$$

Clearly $|\widetilde{\text{OPT}}| \leq \frac{\log n}{5\epsilon^3}$. Let $\text{OPT}_1 := \text{OPT} - \widetilde{\text{OPT}}$.

For the next step, for every $o \in \text{OPT}_1$, let $\pi(o)$ be the closest center in $\text{OPT} - \{o\}$ to o , and let $d_o = d(o, \text{OPT} - \{o\}) = d(o, \pi(o))$ be the distance. As the instance is $\beta = \frac{\epsilon}{\log n}$ stable we have for every $o \in \text{OPT}_1$:

$$|C_o^*|d_o = |C_o^*|d(o, \pi(o)) \geq \sum_{p \in C_o^*} (d(p, \pi(o)) - d(p, o)) \geq \beta \text{opt} - \frac{5\epsilon^3}{\log n} \text{opt} \geq \frac{\beta}{2} \text{opt},$$

where the first inequality is by the triangle inequality. Therefore, as the cluster C_o^* has cost less than $5\epsilon^3 \text{opt} / \log n$ for $o \in \text{OPT}_1$, we have

$$|C_o^* \cap B(o, 10\epsilon \cdot d_o)| \geq (1 - \epsilon)|C_o^*|.$$

⁴Here and in the following procedures, the input instance is considered as passed implicitly.

Indeed, otherwise, we would have the contradiction

$$\sum_{p \in C_o^*} d(p, o) \geq |C_o^* - B(o, \varepsilon \cdot d_o)| \cdot 10\varepsilon \cdot d_o > |C_o^*| 10\varepsilon^2 d_o \geq 10\varepsilon^2 \frac{\beta}{2} \text{opt} = 5\varepsilon^3 \frac{\text{opt}}{\log n}.$$

Now let $\widetilde{\text{OPT}}_1$ be the subset of OPT_1 so that for every $o \in \widetilde{\text{OPT}}_1$, we have $B(o, 20\varepsilon \cdot d_o) \cap S = \emptyset$. We claim that $|\widetilde{\text{OPT}}_1| \leq \frac{\log n}{5\varepsilon^3}$. Indeed, for each $o \in \widetilde{\text{OPT}}_1$, the cost of the points in $C_o^* \cap B(o, 10\varepsilon \cdot d_o)$ in the solution S would be at least $10\varepsilon d_o (1 - \varepsilon) |C_o^*| \geq 10(1 - \varepsilon) \varepsilon^2 \frac{\text{opt}}{\log n}$. Therefore, using that local search gives a 5-approximate solution, we must have $|\widetilde{\text{OPT}}_1| \leq \frac{5 \log n}{10(1 - \varepsilon) \varepsilon^2} \leq \frac{\log n}{5\varepsilon^3}$.

We define $\text{OPT}_2 = \text{OPT}_1 - \widetilde{\text{OPT}}_1$ and so now we have $B(o, 20\varepsilon \cdot d_o) \cap S \neq \emptyset$ for every $o \in \text{OPT}_2$. Now let $\widetilde{\text{OPT}}_2$ be the subset of OPT_2 so that every $o \in \widetilde{\text{OPT}}_2$ is such that $|B(o, d_o/3) \cap S| > 1$. By the pigeon hole principle we must have that $|\widetilde{\text{OPT}}_2| \leq |\text{OPT}| - |\text{OPT}_2| = |\text{OPT}| + |\widetilde{\text{OPT}}_1| \leq \frac{2 \log n}{5\varepsilon^3}$.

Let $\text{OPT}_3 := \text{OPT}_2 - \widetilde{\text{OPT}}_2$. For each center $o \in \text{OPT}_3$ there is thus a center $c(o) \in S$ such that $d(o, c(o)) \leq 20\varepsilon d_o$ and any other center $c' \in S$ with $c' \neq c(o)$ satisfies $d(o, c') \geq d_o/3$. It follows that all points in $B(o, d_o/8)$ are assigned to $c(o)$ in S , where we impose that ε is small enough so that $d_o(\frac{1}{8} + 20\varepsilon) < d_o(\frac{1}{3} - \frac{1}{8})$. In other words, at least a $(1 - \varepsilon)$ fraction of the points of C_o^* are assigned to the cluster with center $c(o)$ in S . Now let $\widetilde{\text{OPT}}_3$ be those centers in OPT_3 so that for every $o \in \widetilde{\text{OPT}}_3$ the associated center $c(o) \in S$ is assigned more than $\varepsilon |C^*|$ points from outside the ball $B(o, d_o/8)$. Imposing that ε is small enough so that $d_o(\frac{1}{8} - 20\varepsilon) > \frac{d_o}{9}$, the cost of the cluster of $c(o)$ in S is then at least

$$\frac{d_o}{9} \cdot \varepsilon |C^*| \geq \frac{\varepsilon^2}{9 \log n} \text{opt}.$$

It follows that $\widetilde{\text{OPT}}_3$ has cardinality at most $45 \frac{\log n}{\varepsilon^2} \leq \frac{\log n}{5\varepsilon^3}$, where the last inequality again holds for ε small enough. Our final set is $\text{OPT}_4 := \text{OPT}_3 - \widetilde{\text{OPT}}_3$.

Now for each center in $c \in \text{OPT}_4$, with associated cluster C^* , we have that it together with its associated center $c(o) \in S$, with associated cluster C' , satisfies the following:

- All clients in $B(o, d_o/8)$, i.e. at least $(1 - \varepsilon) |C^*|$ many clients, are both assigned to o and $c(o)$ in OPT and S , respectively. Notice that this implies $\min\{|C^*|, |C'|\} \geq (1 - \varepsilon) |C^*|$.
- The additional clients assigned to o and not to $c(o)$ is at most $\varepsilon |C^*|$.
- The additional clients assigned to $c(o)$ and not o is at most $\varepsilon |C^*|$.

Notice that $|C^* \Delta C'| \leq 2\varepsilon |C^*| \leq 3\varepsilon \min\{|C^*|, |C'|\}$, in particular C^* is pure. It follows that all centers in OPT_4 correspond to pure clusters. Furthermore, OPT_4 was obtained by removing at most $\frac{\log n}{\varepsilon^3}$ centers. The claim follows. $\square$

Let C^* be a pure cluster of OPT and C' be the associated cluster of S . Let c^* be the center of C^* and c' be the center of C' . We define $t(c^*) := c'$. For any client $p \in C^*$, let $r(p) := d(p, c') = d(p, t(c^*))$. We have the following lemma which is the only lemma that requires that the solution S is obtained via the local search (the proof of the previous lemma only used that it was a constant-factor approximation). In words, it says that S approximates the connection cost of clients belonging to pure clusters of the optimal solution almost perfectly.

Lemma 4.4 (Adapted from [CS17]). *Let $D_{\text{pure}}^* \subseteq D$ be the subset of clients that belong to pure clusters of the optimal solution OPT . We have*

$$\sum_{p \in D_{\text{pure}}^*} r(p) \leq \sum_{p \in D_{\text{pure}}^*} \text{OPT}_p + O(\varepsilon \cdot \text{opt}).$$

Proof. Recall that $S_p = d(p, S)$. Consider a pair of pure clusters C^* and C' with the same notation as above. We prove the following:

1. $\sum_{p \in C^* \cap C'} r(p) = \sum_{p \in C^* \cap C'} d(p, S) \leq \sum_{p \in C^* \cap C'} (1 + 4\varepsilon) \text{OPT}_p + 4\varepsilon S_p$; and

$$2. \sum_{p \in C^* - C'} r(p) = \sum_{p \in C^* - C'} d(p, c') \leq \sum_{p \in C^* - C'} \text{OPT}_p + 4\epsilon \sum_{p \in C^*} (\text{OPT}_p + S_p).$$

The statement of the lemma then follows by summing up the above bounds and using that $\text{cost}(S) \leq 5 \cdot \text{opt}$ by Lemma 4.2.

For proving the first bullet, consider the swap that swaps in c^* and swaps out the center c' . The cost change for the clients in $C^* \cap C'$ is exactly $\sum_{p \in C^* \cap C'} (\text{OPT}_p - S_p)$. For any $p \notin C^*$, if p is not served by c' in S the cost is unchanged. Otherwise the cost of $p \in C' - C^*$ in the new solution $S - \{c'\} \cup \{c^*\}$ is by triangle inequality at most $S_p + \frac{1}{|C^* \cap C'|} \sum_{p' \in C^* \cap C'} (S_{p'} + \text{OPT}_{p'})$. Summing over all clients $p \in C' - C^*$, this is at most $\sum_{p \in C' - C^*} S_p + \frac{|C' - C^*|}{|C^* \cap C'|} \sum_{p' \in C^* \cap C'} (S_{p'} + \text{OPT}_{p'})$. Thus, the cost difference $\text{cost}(S - \{c'\} \cup \{c^*\}) - \text{cost}(S)$ for these clients is at most $\frac{|C' - C^*|}{|C^* \cap C'|} \sum_{p' \in C^* \cap C'} (S_{p'} + \text{OPT}_{p'}) \leq 4\epsilon \sum_{p' \in C^* \cap C'} (S_{p'} + \text{OPT}_{p'})$, where in the inequality we used the fact that C^* is pure, hence $\frac{|C' - C^*|}{|C^* \cap C'|} \leq \frac{3\epsilon|C^*|}{(1-\epsilon)|C^*|} \leq 4\epsilon$. The first bullet follows since, by local optimality, $\text{cost}(S - \{c'\} \cup \{c^*\}) - \text{cost}(S) \geq 0$.

We turn to the second bullet. By triangle inequality, for each $p \in C^* - C'$, $d(p, c')$ is at most $d(p, c^*) + \frac{1}{|C^* \cap C'|} \sum_{p' \in C^* \cap C'} (S_{p'} + \text{OPT}_{p'})$. Summing up over all clients $p \in C^* - C'$, we have that

$$\sum_{p \in C^* - C'} d(p, c') \leq \sum_{p \in C^* - C'} \text{OPT}_p + \frac{|C^* - C'|}{|C^* \cap C'|} \sum_{p' \in C^* \cap C'} (\text{OPT}_{p'} + S_{p'}).$$

The second bullet follows from the assumption that C^* is pure, and hence $\frac{|C^* - C'|}{|C^* \cap C'|} \leq 4\epsilon$ similarly to the first bullet. $\square$

4.2 Cheap Clusters and D -Sample Process to Hit Expensive Clusters

In the previous section, we showed that the cost of pure clusters in OPT is close to to their optimal cost in our local search solution S . Our goal in this section is to “hit” those non-pure clusters of OPT that have a high cost in S . We say that a non-pure cluster C^* of OPT with center c^* is *basic-cheap* if the total cost in S of the clients in C_{avg}^* is less than $\epsilon^5 \text{opt} / \log n$ or if there exists a center of S at distance at most $(1 + \epsilon) \text{avg}_{C^*, \text{OPT}} + \epsilon \gamma_{C^*}$ from c^* , where $\gamma_{C^*} := \frac{1}{|C^*|} \sum_{p \in C^*} (\text{OPT}_p + S_p)$; in the latter case we also say that C^* is *covered* by S .

The following procedure, that aims to “hit” all clusters of OPT that are non-pure and non-basic-cheap, is inspired by the classic k -means++ algorithm and its so-called D^2 -sampling procedure to sample clients proportional to their cost in solution S . For k -median this becomes D -sampling (instead of D^2 -sampling).

D-Sample(S, s)

- 1: $W \leftarrow \emptyset$
- 2: **for** s many times **do**
- 3: Add to W one client p sampled with probability $\frac{d(p, S)}{\text{cost}(S)}$
- 4: **return** W

Next let W be the set returned by $D\text{-Sample}(S, s^*)$ with

$$s^* = 5 \cdot \frac{\log n}{\epsilon^5} \ln \left(\frac{7}{\epsilon^2} \right).$$

We say that W *hits* a cluster C^* of OPT if $C_{\text{avg}}^* \cap W \neq \emptyset$, i.e., if W contains at least one leader of C^* . We have the following lemma that says that we hit all non-pure and non-basic-cheap clusters except for a set V of clusters that have an insignificant cost in S .

For a non-pure optimal cluster C^* with center $c^* \in \text{OPT}$, we let $t(c^*)$ be the closest center in S to c^* , and for any $p \in C^*$, we let the replacement cost of p be $r(p) := d(p, t(c^*))$.

Lemma 4.5. Let V be the set of clusters C^* of OPT that are non-pure and non-basic-cheap such that $C_{\text{avg}}^* \cap W = \emptyset$. With probability at least $1 - \varepsilon$,

$$\sum_{C^* \in V} \sum_{p \in C^*} r(p) \leq \varepsilon \cdot \text{opt}. \quad (14)$$

Proof. Recall that $S_p = d(p, S)$. Consider a non-pure and non-basic-cheap cluster C^* of OPT . We say that a sampled point $p \in W$ hits C^* if $p \in C_{\text{avg}}^*$ (i.e., p is a leader of C^*). As C^* is not basic-cheap, we have $\sum_{p \in C_{\text{avg}}^*} S_p \geq \varepsilon^5 \text{opt} / \log n$. Thus the probability that a sampled $p \in W$ hits C^* is at least $\frac{\sum_{p \in C_{\text{avg}}^*} S_p}{\text{cost}(S)} \geq \frac{\varepsilon^5}{5 \log n}$, where we used that $\text{cost}(S) \leq 5 \text{opt}$ (Lemma 4.2). Thus the probability that C^* is not hit by any point in W is at most $(1 - \frac{\varepsilon^5}{5 \log n})^{s^*}$ which by the selection of s^* is at most $\varepsilon^2 / 7$. For a point $p \in C^*$ which is assigned to c' in S , one has

$$\begin{aligned} r(p) &= d(p, t(c^*)) \leq d(p, c^*) + d(c^*, t(c^*)) \leq d(p, c^*) + d(c^*, c') \\ &\leq d(p, c^*) + d(p, c^*) + d(p, c') = 2\text{OPT}_p + S_p. \end{aligned}$$

Thus, by linearity of expectation,

$$E\left[\sum_{C^* \in V} \sum_{p \in C^*} r(p)\right] \leq \sum_{p \in D} (2\text{OPT}_p + S_p) \cdot (\varepsilon^2 / 7) \leq \varepsilon^2 \cdot \text{opt},$$

where we again used that $\text{cost}(S) \leq 5 \text{opt}$ by Lemma 4.2. The total cost of the clusters in V is thus at most $\varepsilon \cdot \text{opt}$ with probability at least $1 - \varepsilon$ by Markov's inequality. $\square$

Successful W . We say that the set of clients W obtained from the D -sample process is a *successful* sample if (14) holds. We define the following two types of OPT clusters. We say that a non-pure cluster C^* of OPT with center c^* is *cheap* if it is basic-cheap or is in V ; a non-pure cluster is *expensive* otherwise. In other words, a cluster C^* of the optimal solution is expensive if it is a non-pure and non-basic-cheap cluster such that $C_{\text{avg}}^* \cap W \neq \emptyset$.

Assuming that W is successful, we have a good bound on the cost of the clients assigned to cheap centers in the optimal solution.

Lemma 4.6. Let $D_{\text{cheap}}^* \subseteq D$ be the subset of clients that belong to cheap clusters of the optimal solution OPT . If W is a successful sample,

$$\sum_{p \in D_{\text{cheap}}^*} r(p) \leq \sum_{p \in D_{\text{cheap}}^*} 2\text{OPT}_p + O(\varepsilon \cdot \text{opt}).$$

Proof. We claim that for any cluster C^* of OPT that is basic-cheap, and any $p \in C^*$:

1. If C^* is not covered, then $r(p) = d(p, t(c^*)) \leq \text{OPT}_p + \text{avg}_{C^*, \text{OPT}} + \frac{1}{\varepsilon |C^*|} \sum_{p \in C_{\text{avg}}^*} S_p$.
2. If C^* is covered, then $r(p) \leq \text{OPT}_p + (1 + \varepsilon) \text{avg}_{C^*, \text{OPT}} + \varepsilon \gamma_{C^*}$.

If C^* is covered, the bound follows immediately by the triangle inequality. Otherwise, observe that for any such cluster, we have that $d(p, t(c^*)) \leq \text{OPT}_p + d(t(c^*), c^*)$, and let us bound $d(t(c^*), c^*)$. We have that $d(t(c^*), c^*) \leq \frac{1}{|C_{\text{avg}}^*|} \sum_{p \in C_{\text{avg}}^*} (S_p + \text{OPT}_p)$. Which is at most $\text{avg}_{C^*, \text{OPT}} + \frac{1}{|C_{\text{avg}}^*|} \sum_{p \in C_{\text{avg}}^*} S_p$ by definition. So the inequality follows from the fact that $|C_{\text{avg}}^*| \geq \varepsilon |C^*|$.

Now, to prove the above lemma, we sum up the cost of all the clients of cheap clusters C^* . For a covered cluster C^* , by 2. we have

$$\begin{aligned} \sum_{p \in C^*} r(p) &\leq \sum_{p \in C^*} \left(\text{OPT}_p + (1 + \varepsilon) \text{avg}_{C^*, \text{OPT}} + \varepsilon \gamma_{C^*} \right) \\ &\leq (1 + (1 + 2\varepsilon)(1 + \varepsilon)) \sum_{p \in C^*} \text{OPT}_p + \varepsilon \sum_{p \in C^*} (\text{OPT}_p + S_p), \end{aligned}$$

where we used the definition of $\gamma_{C^*} = \frac{1}{|C^*|} \sum_{p \in C^*} (\text{OPT}_p + S_p)$ and that $|C^*| \cdot \text{avg}_{C^*, \text{OPT}} \leq (1 + 2\varepsilon) \sum_{p \in C^*} \text{OPT}_p$, which holds because C^* has at least $(1 - \varepsilon)|C^*|$ clients $p \in C^*$ so that $\text{OPT}_p \geq \text{avg}_{C^*, \text{OPT}}$.

For a basic-cheap cluster C^* which is not covered, by 1. one has

$$\sum_{p \in C^*} r(p) \leq \sum_{p \in C^*} \left(\text{OPT}_p + \text{avg}_{C^*, \text{OPT}} + \frac{1}{\varepsilon|C^*|} \sum_{p \in C_{\text{avg}}^*} S_p \right) \leq (2 + 2\varepsilon) \sum_{p \in C^*} \text{OPT}_p + \varepsilon^4 \text{opt} / \log(n),$$

where we again used that $|C^*| \cdot \text{avg}_{C^*, \text{OPT}} \leq (1 + 2\varepsilon) \sum_{p \in C^*} \text{OPT}_p$ and we additionally used that, by the definition of basic-cheap clusters, $\sum_{p \in C_{\text{avg}}^*} S_p \leq \varepsilon^5 \text{opt} / \log n$. Finally, we have that if we sum up all clusters in V , i.e., clusters that are cheap but not basic-cheap, then Lemma 4.5 says

$$\sum_{C^* \in V} \sum_{p \in C^*} r(p) \leq \varepsilon \cdot \text{opt}.$$

The statement of the lemma now follows by summing up the above bounds for all cheap clusters and using that $\text{cost}(S) \leq 5 \cdot \text{opt}$ by Lemma 4.2 and the property that there are at most $\log(n)/\varepsilon^3$ basic-cheap clusters. The latter claim is true because Lemma 4.3 says that there are at most $\log(n)/\varepsilon^3$ non-pure clusters and basic-cheap clusters is a subset of non-pure clusters. $\square$

Given S and W , the set of clusters of the optimal solution OPT is thus partitioned into pure clusters, cheap clusters, and expensive clusters. We let $\hat{\text{OPT}}$ be the centers of OPT that are expensive. We further define the set $D_{\text{exp}}^* \subseteq D$ to be the subset of clients that belong to the expensive clusters in OPT . We have thus partitioned D into sets D_{pure}^* , D_{cheap}^* , and D_{exp}^* depending on the type of optimal cluster they belong to. Lemma 4.4 bounds the cost of the clients in D_{pure}^* in S and Lemma 4.6 bounds the cost of the clients in D_{cheap}^* . For future reference, we summarize these two lemmas (by weakening the upper bound for clients in D_{pure}^*).

Lemma 4.7. *Assuming W is a successful sample,*

$$\sum_{p \in D - D_{\text{exp}}^*} r(p) \leq \sum_{p \in D - D_{\text{exp}}^*} 2\text{OPT}_p + O(\varepsilon \cdot \text{opt}).$$

It follows that the connection cost in solution S of clients in $D - D_{\text{exp}}^*$ is within a factor 2 of their cost in the optimal solution. The remaining part of the algorithm is thus devoted to modifying S to obtain a small connection cost of the clients in D_{exp}^* without significantly increasing the cost of the other clients.

4.3 Identifying Balls of Expensive Clusters

By the definition of expensive clusters, each expensive cluster C^* of OPT satisfies $C^* \cap W \neq \emptyset$ where W is the sample obtained by running the sampling procedure of the last subsection. Here, our goal is to guess a subset $\mathcal{B}$ of balls so that for each expensive cluster C^* there is a ball $B(\ell, \rho) \in \mathcal{B}$ so that ℓ , which we refer to as a leader, is in C_{avg}^* and ρ is a very close approximation to $\text{avg}_{C^*, \text{OPT}}$. This guarantees that the center $c^* \in \hat{\text{OPT}}$ of cluster C^* is in $B(\ell, \rho)$ and that no other center in $B(\ell, \rho)$ is "too" far from c^* since the radius of the ball is $\approx \text{avg}_{C^*, \text{OPT}}$. The balls $\mathcal{B}$ will then be used when we approximate the centers in $\hat{\text{OPT}}$ via submodular function maximization subject to a partition matroid constraint (the balls will correspond to the partitions of the matroid). As the algorithm does not know the set of expensive clusters, it enumerates all subsets of W , and for each guessed point, it enumerates a constant number of radii.

ballGuess(S, W)

- 1: $\mathcal{L}_{\text{ball}} \leftarrow \emptyset$
- 2: **for** all subsets $\{p_1, p_2, \dots, p_q\} \subseteq W$ **do**
- 3: **for** all integers $i_1, i_2, \dots, i_q$ such that $\varepsilon^3 S_{p_j} \leq (1 + \varepsilon^3)^{i_j} \leq S_{p_j} / \varepsilon^3$ for $j = 1, 2, \dots, q$ **do**

4: add $\mathcal{B} = \{B(p_1, (1 + \varepsilon^3)^{i_1}), B(p_2, (1 + \varepsilon^3)^{i_2}), \dots, B(p_q, (1 + \varepsilon^3)^{i_q})\}$ to $\mathcal{L}_{\text{ball}}$
5: return $\mathcal{L}_{\text{ball}}$

We show that the *ballGuess()* runs in polynomial time and that one of the subsets $\mathcal{B}$ of balls that the procedure constructs is a correct guess: every expensive cluster C^* has a leader, and $\text{avg}_{C^*, \text{OPT}}$ is guessed approximately correctly. We say that a set of balls $\mathcal{B}$ is a *valid* set of balls if it satisfies the following: For each expensive cluster C^* of OPT, we have a ball $B(\ell, \rho) \in \mathcal{B}$ such that $\ell \in C_{\text{avg}}^*$ is a leader and

$$\text{avg}_{C^*, \text{OPT}} \leq \rho \leq \text{avg}_{C^*, \text{OPT}} + \varepsilon \gamma_{C^*},$$

where we recall that $\gamma_{C^*} = \frac{1}{|C^*|} \sum_{p \in C^*} (\text{OPT}_p + S_p)$.

Lemma 4.8. *ballGuess() runs in polynomial time $n^{\varepsilon^{-O(1)}}$ and produces a collection $\mathcal{L}_{\text{ball}}$ of sets of balls such that at least one set $\mathcal{B} \in \mathcal{L}_{\text{ball}}$ is a valid set of balls.*

Proof. We start by showing that the running time is bounded by $n^{\varepsilon^{-O(1)}}$. Indeed, we have $|W| = s^*$, and so the number of subsets of W is $2^{s^*} = n^{\varepsilon^{-O(1)}}$ since $s^* = 5 \cdot \frac{\log n}{\varepsilon^3} \ln \left(\frac{7}{\varepsilon^2} \right)$. Moreover, for a fixed subset $\{p_1, p_2, \dots, p_q\}$, we consider a constant number $\alpha = \log_{1+\varepsilon^3}(1/\varepsilon^6)$ of balls (i.e., values of i_j) for each p_j . So the total number of balls considered added for each subset $\{p_1, p_2, \dots, p_q\}$ is α^q which is $n^{\varepsilon^{-O(1)}}$ since $q \leq |W| \leq s^*$.

We now turn our attention to showing that there is a set of balls $\mathcal{B} \in \mathcal{L}_{\text{ball}}$ that satisfies the properties of the lemma. Let $C^*(1), \dots, C^*(|\hat{\text{OPT}}|)$ be the expensive clusters of OPT. By definition, we have $C_{\text{avg}}^*(j) \cap W \neq \emptyset$ for each such cluster. Now arbitrarily fix one client $\ell_j \in C_{\text{avg}}^*(j) \cap W$ for $j = 1, 2, \dots, |\hat{\text{OPT}}|$, which we refer to the leader of $C^*(j)$. Since the algorithm enumerates over all subsets of W , there exists one such subset $\{\ell_1, \dots, \ell_{|\hat{\text{OPT}}|}\}$ that is a valid set of leaders for all the expensive clusters. We consider the set

$$\mathcal{B} = \{B(\ell_1, (1 + \varepsilon^3)^{i_1}), B(\ell_2, (1 + \varepsilon^3)^{i_2}), \dots, B(\ell_{|\hat{\text{OPT}}|}, (1 + \varepsilon^3)^{i_{|\hat{\text{OPT}}|}})\}$$

of balls, where i_j is the smallest integer so that $(1 + \varepsilon^3)^{i_j} \geq \max\{\text{avg}_{C^*(j), \text{OPT}}, \varepsilon^3 S_{\ell_j}\}$.

It remains to prove that, for each leader ℓ_j , we have $(1 + \varepsilon^3)^{i_j} \leq S_{\ell_j}/\varepsilon^3$ and $\text{avg}_{C^*(j), \text{OPT}} \leq (1 + \varepsilon^3)^{i_j} \leq \text{avg}_{C^*(j), \text{OPT}} + \varepsilon \gamma_{C^*(j)}$. The first property guarantees that $\mathcal{B} \in \mathcal{L}_{\text{ball}}$ and the second property is the desired bound on each radius ρ for $\mathcal{B}$ to be a valid set of balls.

We now verify these two properties for each leader ℓ_j . To simplify notation, we let $\ell = \ell_j$, $C^* = C^*(j)$ and $\rho = (1 + \varepsilon^3)^{i_j}$. We have $d(\ell, S) = S_\ell > \varepsilon \text{avg}_{C^*, \text{OPT}}$ since otherwise C^* would be covered by S , which would contradict that C^* is an expensive cluster. By the selection of i_j , $\rho = (1 + \varepsilon^3)^{i_j} \leq (1 + \varepsilon^3) \max\{\text{avg}_{C^*, \text{OPT}}, \varepsilon^3 S_\ell\} \leq (1 + \varepsilon^3) S_\ell / \varepsilon$ and thus $(1 + \varepsilon^3)^{i_j} \leq S_\ell / \varepsilon^3$ as required.

We further claim that $\rho \in [\text{avg}_{C^*, \text{OPT}}, \text{avg}_{C^*, \text{OPT}} + \varepsilon^2 S_\ell]$. Assume first that $\text{avg}_{C^*, \text{OPT}} \geq \varepsilon^3 S_\ell$. Then $\rho = (1 + \varepsilon^3)^{i_j} \geq \text{avg}_{C^*, \text{OPT}}$, for the smallest possible integer i_j which satisfies this condition. The claim then follows since $\rho \leq (1 + \varepsilon^3) \text{avg}_{C^*, \text{OPT}} \leq \text{avg}_{C^*, \text{OPT}} + \varepsilon^2 S_\ell$ as required (where we used that $\text{avg}_{C^*, \text{OPT}} \leq S_\ell / \varepsilon$). In the complementary case, namely $\text{avg}_{C^*, \text{OPT}} < \varepsilon^3 S_\ell$, one has that i_j was selected to be the smallest integer so that $(1 + \varepsilon^3)^{i_j} \geq \varepsilon^3 S_\ell$. Moreover, the interval $[\text{avg}_{C^*, \text{OPT}}, \text{avg}_{C^*, \text{OPT}} + \varepsilon^2 S_\ell]$ contains the interval $[\varepsilon^3 S_\ell, \varepsilon^2 S_\ell]$, and the latter interval contains at least one power of $(1 + \varepsilon^3)$. We thus have

$$\text{avg}_{C^*, \text{OPT}} \leq \rho \leq \text{avg}_{C^*, \text{OPT}} + \varepsilon^2 S_\ell.$$

We conclude the proof by showing that $S_\ell \leq (2 + 2\varepsilon) \gamma_{C^*}$. Let ℓ be assigned to c' in S , and consider any $p \in C^*$ which is assigned to c'' in S . Then one has $S_\ell = d(\ell, c') \leq d(\ell, c'') \leq d(\ell, c^*) + d(c^*, p) + d(p, c'') = d(\ell, c^*) + \text{OPT}_p + S_p$. Hence, averaging over C^* , one gets $S_\ell \leq d(\ell, c^*) + \frac{1}{|C^*|} \sum_{p \in C^*} (S_p + \text{OPT}_p)$. Since ℓ is a leader, i.e., $\ell \in C_{\text{avg}}^*$, $d(\ell, c^*) \leq \text{avg}_{C^*, \text{OPT}} \leq \frac{1+2\varepsilon}{|C^*|} \sum_{p \in C^*} \text{OPT}_p$, where we used that there is at least $(1 - \varepsilon)|C^*|$ clients $p \in C^*$ so that $d(p, c^*) \geq \text{avg}_{C^*, \text{OPT}}$. Therefore, $S_\ell \leq (2 + 2\varepsilon) \gamma_{C^*}$, which, as aforementioned, concludes the proof of the lemma. $\square$

We thus have that the ball procedure produces a family $\mathcal{L}_{\text{ball}}$ such that a set $\mathcal{B} \in \mathcal{L}_{\text{ball}}$ satisfies the conditions of the lemma. While our algorithm will try all possible sets in $\mathcal{L}_{\text{ball}}$ (since it does not know which one is valid), we do our analysis by considering the run of the algorithm when it selects this set $\mathcal{B}$ of valid balls.

4.3.1 Dummy Centers and Mixed Solutions M_O and M_D

Next, the algorithm creates, for each ball $B(\ell, \rho) \in \mathcal{B}$ with center ℓ and radius ρ , a *dummy* center δ at distance ρ from ℓ and at distance $\rho + d(\ell, p)$ from any other input point p . Let Λ be the set of dummy centers, so $|\Lambda| = |\mathcal{B}|$. Recall that D_{exp}^* is the set of clients that are in an expensive cluster of OPT, i.e., served by a center in $\hat{\text{OPT}}$ in the optimal solution.

Lemma 4.9. *Suppose the sample W is successful, and that the set of balls $\mathcal{B}$ is valid, then there exists a set of centers S_0 of S that satisfies the following properties:*

1. $|S_0| = |\hat{\text{OPT}}| \leq \frac{\log n}{\epsilon^3}$, and;
2. $\forall c \in S_0$, there is no $c^* \in \text{OPT} - \hat{\text{OPT}}$ such that $t(c^*) = c$, and;
3. $\text{cost}(S - S_0 \cup \Lambda) \leq 3 \sum_{p \in D_{\text{exp}}^*} \text{OPT}_p + \sum_{p \in D - D_{\text{exp}}^*} r(p) + O(\epsilon \cdot \text{opt}) \leq (3 + O(\epsilon)) \text{opt}$.

Proof. By definition, the centers in $\hat{\text{OPT}}$ are not pure, hence $|\hat{\text{OPT}}| \leq \frac{\log n}{\epsilon^3}$ by Lemma 4.3. Furthermore, since there are $|\text{OPT} - \hat{\text{OPT}}|$ centers of OPT not in $\hat{\text{OPT}}$, there are (at least) $|\hat{\text{OPT}}|$ centers c of S such that there is no $c^* \in \text{OPT} - \hat{\text{OPT}}$ such that $t(c^*) = c$, and we let S_0 refer to these centers: 1. and 2. follow immediately.

Let us turn to 3. The bound on the cost of the clients that are not in D_{exp}^* follows from 2.: for each client p in a cluster of OPT whose center is $c^* \in \text{OPT} - \hat{\text{OPT}}$, we have that $t(c^*) \in S - S_0$ and so its cost is at most $r(p)$. We finally consider the points in D_{exp}^* . Since the balls are valid, Lemma 4.8 implies that for each expensive cluster C^* of OPT, there is a ball $B(\ell, \rho)$ where $\rho \in [\text{avg}_{C^*, \text{OPT}}, \text{avg}_{C^*, \text{OPT}} + \epsilon \gamma_{C^*})$ and $\ell \in C_{\text{avg}}^*$. Therefore, if we let c^* be the center of C^* ,

$$\begin{aligned}
\sum_{p \in C^*} d(p, \Lambda) &\leq \sum_{p \in C^*} (d(p, c^*) + d(c^*, \ell) + d(\ell, \Lambda)) \\
&\leq \sum_{p \in C^*} (\text{OPT}_p + \text{avg}_{C^*, \text{OPT}} + \rho) \\
&\leq \sum_{p \in C^*} (\text{OPT}_p + 2(\text{avg}_{C^*, \text{OPT}} + \epsilon \gamma_{C^*})) \\
&= \sum_{p \in C^*} \text{OPT}_p + |C^*| 2(\text{avg}_{C^*, \text{OPT}} + \epsilon \gamma_{C^*}) \\
&\leq (3 + O(\epsilon)) \sum_{p \in C^*} \text{OPT}_p + O(\epsilon \sum_{p \in C^*} S_p).
\end{aligned}$$

The lemma follows from $\text{cost}(S) \leq 5 \cdot \text{opt}$ (Lemma 4.2) and by invoking Lemma 4.7 to bound the sum of the $r(p)$ values (where we use the assumption that the sample W is successful). $\square$

Definition of the mixed solutions M_O and M_D . Next, let us analyze the cost of the swap $(\hat{\text{OPT}}, S_0)$. Namely, of the solution $M_O = S - S_0 \cup \hat{\text{OPT}}$, where M in M_O stands for “mixed solution” and the subscript O stands for that the mixed solution is obtained by swapping in some elements from OPT (and removing S_0). We will also analyze the “dummy” version of M_O where instead of swapping in $\hat{\text{OPT}}$, we swap in the dummy centers Λ . We denote that solution by $M_D = S - S_0 \cup \Lambda$, where the subscript D stands for “dummy”.

Lemma 4.10. *We have $d(p, \hat{\text{OPT}}) = \text{OPT}_p$ if $p \in D_{\text{exp}}^*$ and $d(p, S - S_0) \leq r(p)$ if $p \in D - D_{\text{exp}}^*$.*

Proof. We have $d(p, \hat{\text{OPT}}) = \text{OPT}_p$ when $p \in D_{\text{exp}}^*$ because the center serving p in the optimal solution is in $\hat{\text{OPT}}$. Consider the remaining case and let $p \in D - D_{\text{exp}}^*$ be a client whose serving center in OPT is c^* . By property 2. of Lemma 4.9, the center $t(c^*)$ is in $S - S_0$ and so $d(p, S - S_0) \leq d(p, t(c^*)) = r(p)$. $\square$

Notice that M_O contains $S - S_0$ and $\hat{\text{OPT}}$. Therefore, if we sum up the above bounds for all clients we get

$$\text{cost}(M_O) \leq \sum_{p \in D_{\text{exp}}^*} \text{OPT}_p + \sum_{p \in D - D_{\text{exp}}^*} r(p),$$

which by Lemma 4.7 is at most $\sum_{p \in D_{\text{exp}}^*} \text{OPT}_p + \sum_{p \in D - D_{\text{exp}}^*} 2\text{OPT}_p + O(\varepsilon \cdot \text{opt})$ (assuming that the sample W was successful). By swapping S_0 with $\hat{\text{OPT}}$ we thus get a 1-approximation on the clients D_{exp}^* and a 2-approximation of the remaining clients (plus a small error term). Our goal in the next sections will thus be to approximate this swap. The next few steps will be aimed at simplifying (or guessing parts of) S_0 . The last step will then approximate $\hat{\text{OPT}}$ and the remaining part of S_0 by submodular function maximization subject to a partition matroid constraint. For that part, it will be helpful to have a bound on the dummy solution as well; specifically, a modified version of it (see Lemma 4.12 and Claim 7). For intuition of those statements, let us here say that one can show

$$\text{cost}(M_D) \leq \sum_{p \in D_{\text{exp}}^*} 3\text{OPT}_p + \sum_{p \in D - D_{\text{exp}}^*} r(p) + O(\varepsilon \cdot \text{opt})$$

by observing that only the clients D_{exp}^* have a different connection cost in M_D than in M_O . Moreover, each such client $p \in D_{\text{exp}}^*$ that was previously assigned to a center $c^* \in \hat{\text{OPT}}$, and belongs to cluster C^* of OPT, has an associated dummy center δ at distance at most $d(p, c^*) + 2\text{avg}_{C^*, \text{OPT}} + \varepsilon\gamma_{C^*}$ (assuming the set $\mathcal{B}$ of balls is valid). Simplifications then give the stated inequality as $|C^*|\text{avg}_{C^*, \text{OPT}} \leq (1 + 2\varepsilon) \sum_{p \in C^*} \text{OPT}_p$ and $|C^*|\varepsilon\gamma_{C^*} = \varepsilon \sum_{p \in C^*} (\text{OPT}_p + S_p)$.

4.4 Removal of Expensive Centers of S_0

Thus, it remains to show that our algorithm can yield a good approximation to the gain that the swap $(\hat{\text{OPT}}, S_0)$ would provide. Of course, it will not necessarily be optimum, but we can show it is enough to conclude the proof of our theorem. We focus on the iteration of the procedure that produces valid ball guesses. In particular, at this point given that W is successful and the ball guesses are valid, the number of balls in our ball guess is equal to $|S_0| = |\hat{\text{OPT}}| = |\Lambda| \leq \frac{\log n}{\varepsilon^3}$. Our algorithm then makes use of the following procedure that takes as input the local search solution S and the set Λ of dummy centers.

expRem(S, Λ)

```

1:  $\mathcal{L}_{\text{exp}} \leftarrow \emptyset$ 
2: for  $(\frac{8}{\varepsilon})^{|\Lambda|+1} \ln n$  many iterations do
3:    $\mathcal{Q} \leftarrow \emptyset$ 
4:   for  $|\Lambda| + 1$  many iterations do
5:     With probability 1/2 do the following. Consider the clustering induced by  $\Lambda \cup S - \mathcal{Q}$ 
     and, for each  $c \in S - \mathcal{Q}$ , let  $\text{cost}(c)$  be the cost of the cluster associated with  $c$  in the considered
     solution
6:     Sample one  $c \in S - \mathcal{Q}$  with probability  $\frac{\text{cost}(c)}{\sum_{c' \in S - \mathcal{Q}} \text{cost}(c')}$ 
7:     Set  $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{c\}$ 
8:    $\mathcal{L}_{\text{exp}} \leftarrow \mathcal{L}_{\text{exp}} \cup \{\mathcal{Q}\}$ 
9: return  $\mathcal{L}_{\text{exp}}$ 

```

The goal of the above procedure is to guess a subset $\mathcal{Q}$ of all “costly” centers in S_0 (the set $\mathcal{L}_{\text{exp}}$ contains all guesses). Specifically, if we consider the solution $(S - \mathcal{Q}) \cup \Lambda$, then we wish that the cost of the clusters corresponding to the remaining centers in $S_0 - \mathcal{Q}$ is at most $\varepsilon \cdot \text{opt}$. Formally,

the clusters corresponding to $S_0 - Q$ in $S - Q \cup \Lambda$ consists of all the clients whose closest center in $S - Q \cup \Lambda$ is from $S_0 - Q$. If we let $D' \subseteq D$ be those clients then the total cost in solution $S - Q \cup \Lambda$ of the clusters with centers in $S_0 - Q$ is defined as

$$\sum_{p \in D'} d(p, S - Q \cup \Lambda) = \sum_{p \in D'} d(p, S_0 - Q),$$

where the equality holds because of the definition of D' .

Lemma 4.11. *expRem() runs in $n^{1/\varepsilon^{O(1)}}$ time and produces a collection $\mathcal{L}_{\text{exp}}$ of at most $n^{1/\varepsilon^{O(1)}}$ subsets of S such that with probability at least $1 - 1/n$ there exists $Q \in \mathcal{L}_{\text{exp}}$ satisfying*

1. $Q \subseteq S_0$, and
2. The total cost in the solution $S - Q \cup \Lambda$ of the clusters with centers in $S_0 - Q$ is at most $\varepsilon \cdot \text{opt}$.

Proof. The claim on the running time and on the size of $\mathcal{L}_{\text{exp}}$ is trivially satisfied since $|\Lambda| = |S_0| \leq \frac{\log n}{\varepsilon^3}$.

It remains to show that with probability at least $1 - \frac{1}{n}$ at least one set $Q \in \mathcal{L}_{\text{exp}}$ satisfies the desired properties. Let us consider some iteration of the external for loop, and let Q be the corresponding set. We say that the j -th iteration of the corresponding inner for loop is successful if the following happens:

1. If condition (2) of the lemma is satisfied considering the current value of Q , then the event from line 5 does not happen (hence in particular Q is not updated in this j -th iteration).
2. Otherwise, the event from line 5 happens and furthermore the sampled c belongs to S_0 .

Let A_j denote the event that the considered j -th iteration is successful. Let us condition on the event $A_{<j}$ that the previous iterations $A_1, \dots, A_{j-1}$ are successful. In particular, one has $Q \subseteq S_0$ at the beginning of the j -th iteration. Let B_j denote the event that condition (2) is satisfied at the beginning of the iteration. Then trivially $\Pr[A_j | B_j, A_{<j}] \geq 1/2$. Suppose next that B_j does not hold. In that case, with probability $1/2$, the procedure samples a center c . Since by assumption $\sum_{c \in S_0 - Q} \text{cost}(c) > \varepsilon \text{opt}$, the probability that the procedure samples some $c \in S_0 - Q$ is at least

$$\frac{\varepsilon \text{opt}}{\sum_{c' \in S_0 - Q} \text{cost}(c')} \geq \frac{\varepsilon \text{opt}}{\text{cost}(\Lambda \cup S - Q)} \geq \frac{\varepsilon \text{opt}}{\text{cost}(\Lambda \cup S - S_0)} \geq \frac{\varepsilon}{4},$$

where in the last inequality we used the fact that $\text{cost}(\Lambda \cup S - S_0) \leq 4 \cdot \text{opt}$ by Lemma 4.9.

Altogether $\Pr[A_j | \bar{B}_j, A_{<j}] \geq \varepsilon/8$. Thus $\Pr[A_j | A_{<j}] \geq \varepsilon/8$. Chaining the obtained inequalities we obtain that all the events $A_1, \dots, A_{|S_0|+1}$ are simultaneously true with probability at least $(\varepsilon/8)^{|S_0|+1}$. Notice that when the latter event happens, the corresponding Q satisfies all the properties. Indeed, $Q \subseteq S_0$ and furthermore the event B_j must happen for some $j \leq |S_0| + 1$ (hence for the next iterations if any) since otherwise Q would contain more than $|S_0|$ elements.

Thus, the probability that the overall procedure fails is at most $(1 - (\varepsilon/8)^{|S_0|+1})^{(8/\varepsilon)^{|S_0|+1} \ln n} \leq \frac{1}{n}$. $\square$

4.4.1 Consistently Assigning Clients in Mixed Solutions and the sets $\mathcal{U}$ and $\mathcal{R}$

While our algorithm tries all possible $Q \in \mathcal{L}_{\text{exp}}$, we focus on the execution path of a set Q of centers satisfying the properties of Lemma 4.11, i.e.,

1. $Q \subseteq S_0$, and
2. the cost in the solution $S - Q \cup \Lambda$ of the clusters with centers in $S_0 - Q$ is at most $\varepsilon \cdot \text{opt}$.

We define the solution $S_Q := S - Q \cup \Lambda$ and for a center $c \in S_Q$ we let $S_Q(c)$ be the subset of clients closest to c (i.e., assigned to c) in the solution S_Q . The second property above allows us to consider each cluster $S_Q(c)$, $c \in S_0 - Q$, as contracted by paying a small extra cost of $\varepsilon \cdot \text{opt}$. In particular, this allows us to give a “consistent” assignment μ_Q of clients in the mixed solutions. Here, consistent means that all clients in $S_Q(c)$, for $c \in S_0 - Q$, are assigned to the same center in the mixed solutions where all of S_0 is removed.

Definition of μ_O and μ_D . We modify how the clients are assigned in the solution M_O to obtain the assignment μ_O . For every $c \in S_0 - Q$, all the clients in $S_Q(c)$ are reassigned to the same center of M_O as follows. Let c_1 be the center of $\hat{OPT}$ that is the closest to c and let c_2 be the center of $M_O - \hat{OPT} = S - S_0$ that is the closest to c . The clients of $S_Q(c)$ are all assigned to either c_1 or c_2 :

- If $d(c, c_1) \leq d(c, c_2)/2$, assign $S_Q(c)$ to c_1 ; $S_Q(c)$ is called a *type-1* cluster.
- Otherwise ($d(c, c_1) > d(c, c_2)/2$), assign $S_Q(c)$ to c_2 ; $S_Q(c)$ is called a *type-2* cluster.

Let T_1 be the set of clients in a type-1 cluster and T_2 the set of clients in a type-2 cluster. The remaining clients in $D_{\text{exp}}^* - (T_1 \cup T_2)$ are assigned to their closest centers in $\hat{OPT}$, and the remaining clients in $(D - D_{\text{exp}}^*) - (T_1 \cup T_2)$ are assigned to their closest centers in $S - S_0$. Notice that, even neglecting the extra cost due to the consistent assignment, the above assignment of clients in T_2 is suboptimal when $d(c, c_2) > d(c, c_1) > d(c, c_2)/2$. The motivation for this reassignment is technical. More specifically, it will be used in Lemma 4.12 to have a better upper bound on the cost of clients in $C^* \cap T_1$ for a non-expensive cluster C^* (case b2). This leads to a larger upper bound on the cost of clients in $C^* \cap T_2$ for an expensive cluster C^* (case a3), which is however tolerable.

We further obtain an assignment μ_D of clients in M_D by modifying μ_O as follows: each client p with $\mu_O(p) \in \hat{OPT}$ is assigned to the associated dummy center by μ_D . So for each client p we have $\mu_D(p) = \mu_O(p)$ unless $\mu_O(p) \in \hat{OPT}$ in which case $\mu_D(p)$ equals the dummy center associated with $\mu_O(p)$.

Sets $\mathcal{R}$ and $\mathcal{U}$. By the definition of type-1 and type-2 cluster, we can split the set of centers of $S_0 - Q$ into two groups $\mathcal{R}$ and $\mathcal{U}$. Let $\mathcal{R}$ be the set of centers of $S_0 - Q$ whose set of clients is completely assigned to a center of $\hat{OPT}$ in the assignment μ_O , and let $\mathcal{U} = S_0 - Q - \mathcal{R}$ be the remaining ones that are assigned to centers in $S - S_0$.

We end this section by analyzing the costs of the assignments μ_O and μ_D . Recall that we use the notation $\text{cost}(M_O, \mu_O)$ to denote the cost of M_O equipped with assignment μ_O and, similarly $\text{cost}(M_D, \mu_D)$ for the cost of M_D equipped with assignment μ_D . We say that the selection of $(W, \mathcal{B}, Q)$ is successful, if the sample W selected in 4.2 is successful, the set of balls $\mathcal{B}$ from Section 4.3 is valid, and Q selected in this section satisfies the properties of Lemma 4.11.

Lemma 4.12. *If the selection of $(W, \mathcal{B}, Q)$ is successful,*

$$\text{cost}(M_O, \mu_O) + \text{cost}(M_D, \mu_D) \leq 4 \cdot \text{opt} + O(\varepsilon \cdot \text{opt}).$$

Proof. Throughout the proof of the lemma, we will repeatedly upper bound $d(p, \mu_D(p))$ in terms of $d(p, \mu_O(p))$ using the following bounds:

Claim 6. *Consider a client $p \in S_Q(c)$ with $\mu_O(p) \neq \mu_D(p)$ and so $\mu_O(p) \in \hat{OPT}$ is the center of an expensive cluster C^* of $\hat{OPT}$. Then*

1. $d(p, \mu_D(p)) \leq d(p, \mu_O(p)) + 2(\text{avg}_{C^*, \hat{OPT}} + \varepsilon \gamma_{C^*})$, and
2. $d(p, \mu_D(p)) \leq d(p, S_Q) + 3d(c, \mu_O(p))$.

Proof of Claim 6. Let $\mu_O(p) = c^*$. By the assumption that $\mathcal{B}$ is a valid set of balls, we have by Lemma 4.8 that there is a ball $B(\ell, \rho) \in \mathcal{B}$ such that $c^* \in B(\ell, \rho)$ and

$$\text{avg}_{C^*, \hat{OPT}} \leq \rho \leq \text{avg}_{C^*, \hat{OPT}} + \varepsilon \gamma_{C^*}.$$

Thus, if we let $\delta \in \Lambda$ be the dummy center associated with this ball (at a distance ρ from ℓ), then $\mu_D(p) = \delta$ and, by the triangle inequality,

$$d(p, \mu_D(p)) \leq d(p, c^*) + d(c^*, \ell) + d(\ell, \delta) \leq d(p, \mu_O(p)) + 2(\text{avg}_{C^*, \hat{OPT}} + \varepsilon \gamma_{C^*}).$$

For the second bound, we again use the triangle inequality:

$$\begin{aligned} d(p, \mu_D(p)) &\leq d(p, c) + d(c, c^*) + d(c^*, \ell) + d(\ell, \delta) \\ &\leq d(p, S_Q) + d(c, c^*) + 2(\text{avg}_{C^*, \hat{OPT}} + \varepsilon \gamma_{C^*}) \\ &\leq d(p, S_Q) + 3d(c, c^*), \end{aligned}$$

where the last inequality follows from the fact that we must have $d(c, c^*) > (1 + \varepsilon)\text{avg}_{C^*, \text{OPT}} + \varepsilon\gamma_{C^*}$ since otherwise we would have the contradiction that $c^* \in \hat{\text{OPT}}$ is covered by S ; recall that $\hat{\text{OPT}}$ only contains expensive centers of OPT . $\square$

Having proven Claim 6, we prove the lemma by distinguishing between expensive and non-expensive clusters C^* of OPT .

Case a: C^* is an expensive cluster of OPT . Let $c^* \in \hat{\text{OPT}}$ be the center of C^* . Let $p \in C^*$ be a client in C^* and let c be its closest center in S_Q , i.e., $p \in S_Q(c)$. We bound $d(p, \mu_O(p)) + d(p, \mu_D(p))$ by considering three cases:

Case a1: $p \in C^* - (T_1 \cup T_2)$: By definition, we have $\mu_O(p)$ is the closest center in $\hat{\text{OPT}}$. Moreover, $\mu_O(p)$ equals the cluster center c^* of C^* since p is closest to c^* among all centers in $\text{OPT} \supseteq \hat{\text{OPT}}$. Claim 6.1 together with Lemma 4.10 thus gives us,

$$d(p, \mu_O(p)) + d(p, \mu_D(p)) \leq 2\text{OPT}_p + 2(\text{avg}_{C^*, \text{OPT}} + \varepsilon\gamma_{C^*}).$$

Case a2: $p \in C^* \cap T_1$: By definition of type-1 clusters, $\mu_O(p) = c_1$ is the closest center in $\hat{\text{OPT}}$ to c . Now, by Claim 6.2 and triangle inequality,

$$\begin{aligned} d(p, \mu_O(p)) + d(p, \mu_D(p)) &\leq d(p, \mu_O(p)) + d(p, S_Q) + 3d(c, c_1) \leq 2d(p, S_Q) + 4d(c, c_1) \\ &\leq 2d(p, S_Q) + 4d(c, c^*) \leq 2d(p, S_Q) + 4(d(c, p) + \text{OPT}_p) \\ &= 6d(p, S_Q) + 4\text{OPT}_p. \end{aligned}$$

Case a3: $p \in C^* \cap T_2$. By the definition of type-2 clusters, $\mu_O(p) = c_2$ is the closest center to c in $S - S_0$, and $d(c, c_2) \leq 2d(c, \hat{\text{OPT}})$. So, by the triangle inequality, $d(c, c_2) \leq 2(d(c, p) + \text{OPT}_p)$ where we additionally used that $d(p, \hat{\text{OPT}}) \leq \text{OPT}_p$ by Lemma 4.10. As $\mu_O(p) = \mu_D(p)$ in this case,

$$\begin{aligned} d(p, \mu_O(p)) + d(p, \mu_D(p)) &= 2d(p, \mu_O(p)) \leq 2(d(p, c) + d(c, c_2)) \\ &\leq 2(d(p, c) + 2d(c, \hat{\text{OPT}})) \leq 2d(p, c) + 4(d(c, p) + \text{OPT}_p) \\ &= 6d(p, S_Q) + 4\text{OPT}_p. \end{aligned}$$

By the above bounds, we have that

$$\begin{aligned} &\sum_{p \in C^*} (d(p, \mu_O(p)) + d(p, \mu_D(p))) \\ &\leq \sum_{p \in C^* - (T_1 \cup T_2)} (2\text{OPT}_p + 2(\text{avg}_{C^*, \text{OPT}} + \varepsilon\gamma_{C^*})) + \sum_{p \in C^* \cap (T_1 \cup T_2)} (4\text{OPT}_p + 6d(p, S_Q)). \end{aligned}$$

We complete the analysis of the expensive clusters by upper bounding the terms $2\varepsilon\gamma_{C^*}$ and $2\text{avg}_{C^*, \text{OPT}}$. By definition we have $|C^* - (T_1 \cup T_2)| \cdot 2\varepsilon\gamma_{C^*} \leq 2\varepsilon \sum_{p \in C^*} (\text{OPT}_p + S_p)$. For the other term, we claim that

$$2|C^* - (T_1 \cup T_2)| \cdot \text{avg}_{C^*, \text{OPT}} \leq \sum_{p \in C^* - (T_1 \cup T_2)} 2\text{OPT}_p + 3\varepsilon \cdot \sum_{p \in C^*} \text{OPT}_p.$$

This holds because there are at least $(1 - \varepsilon)|C^*|$ clients in C^* such that $\text{OPT}_p \geq \text{avg}_{C^*, \text{OPT}}$, which implies

$$\begin{aligned} \sum_{p \in C^* - (T_1 \cup T_2)} 2\text{OPT}_p + 3\varepsilon \cdot \sum_{p \in C^*} \text{OPT}_p &\geq 2(|C^* - (T_1 \cup T_2)| - \varepsilon|C^*|) \cdot \text{avg}_{C^*, \text{OPT}} + 3\varepsilon(1 - \varepsilon)|C^*| \cdot \text{avg}_{C^*, \text{OPT}} \\ &\geq 2|C^* - (T_1 \cup T_2)| \cdot \text{avg}_{C^*, \text{OPT}}. \end{aligned}$$

In summary, for an expensive cluster C^* we have that

$$\begin{aligned} &\sum_{p \in C^*} (d(p, \mu_O(p)) + d(p, \mu_D(p))) \\ &\leq \sum_{p \in C^* - (T_1 \cup T_2)} 4\text{OPT}_p + \sum_{p \in C^* \cap (T_1 \cup T_2)} (4\text{OPT}_p + 6d(p, S_Q)) + \sum_{p \in C^*} (5\varepsilon\text{OPT}_p + 2\varepsilon S_p). \end{aligned} \quad (15)$$

Case b: C^* is a non-expensive cluster of OPT. We proceed with a similar analysis. Indeed, consider a client $p \in C^*$ and let c be its closest center in S_Q , i.e., $p \in S_Q(c)$. We again bound $d(p, \mu_O(p)) + d(p, \mu_D(p))$ by considering three cases:

Case b1: $p \in C^* - (T_1 \cup T_2)$. By the definition of μ_O , we have that $\mu_O(p)$ is the closest center in $S - S_0$ to p . So $\mu_D(p) = \mu_O(p)$ and $d(p, \mu_O(p)) + d(p, \mu_D(p)) \leq 2r(p)$, by Lemma 4.10.

Case b2: $p \in C^* \cap T_1$. By the definition of type-1 clusters, $\mu_O(p) = c_1$ is the closest center in OPT to c . Now, by Claim 6.2 and the triangle inequality,

$$\begin{aligned} d(p, \mu_O(p)) + d(p, \mu_D(p)) &\leq d(p, \mu_O(p)) + d(p, S_Q) + 3d(c, c_1) \\ &\leq 2d(p, S_Q) + 4d(c, c_1). \end{aligned}$$

We proceed by upper bounding $d(c, c_1)$. Here we critically use the fact that, by the definition of T_1 , one has $d(c, c_1) \leq d(c, c_2)/2$ where c_2 is the closest center to c in $S - S_0$. Let c_3 be the closest center to p in $S - S_0$. Then

$$d(c, c_1) \leq d(c, c_2)/2 \leq d(c, c_3)/2 \leq (d(c, p) + d(p, c_3))/2 = (d(p, S_Q) + d(p, c_3))/2.$$

Moreover, we have $d(p, c_3) = d(p, S - S_0) \leq r(p)$ by Lemma 4.10 and so $d(c, c_1) \leq (d(p, S_Q) + r(p))/2$, which gives us the bound

$$d(p, \mu_O(p)) + d(p, \mu_D(p)) \leq 4d(p, S_Q) + 2r(p).$$

Case b3: $p \in C^* \cap T_2$. By the definition of type-2 clusters, $\mu_O(p) = c_2$ is the closest center to c in $S - S_0$, and so $\mu_O(p) = \mu_D(p)$. Now by Lemma 4.10 and the triangle inequality,

$$d(p, \mu_O(p)) \leq d(p, c) + d(c, c_2) \leq 2d(p, c) + d(p, S - S_0) \leq 2d(p, c) + r(p) = 2d(p, S_Q) + r(p).$$

So we get the bound

$$d(p, \mu_O(p)) + d(p, \mu_D(p)) = 2d(p, \mu_O(p)) \leq 4d(p, S_Q) + 2r(p).$$

Summing up the above bounds, yields

$$\sum_{p \in C^*} (d(p, \mu_O(p)) + d(p, \mu_D(p))) \leq \sum_{p \in C^*} 2r(p) + \sum_{p \in C^* \cap (T_1 \cup T_2)} 4d(p, S_Q)$$

for a non-expensive cluster C^* of OPT. If we sum up the above inequality for all non-expensive clusters and the bound (15) for expensive clusters of OPT we thus get

$$\text{cost}(M_O, \mu_O) + \text{cost}(M_D, \mu_D) \leq \sum_{p \in D_{\text{exp}}^*} 4\text{OPT}_p + \sum_{p \in D - D_{\text{exp}}^*} 2r(p) + O(\varepsilon \cdot \text{opt}),$$

where we used that $\sum_{c \in S_0 - Q} \sum_{p \in S_Q(c)} d(p, S_Q) = \sum_{p \in T_1 \cup T_2} d(p, S_Q) \leq \varepsilon \cdot \text{opt}$ (Lemma 4.11) and $\text{cost}(S) \leq 5\text{opt}$ (Lemma 4.2) to bound the error term $O(\varepsilon \cdot \text{opt})$.

The proof is now concluded by Lemma 4.7, which says that $\sum_{p \in D - D_{\text{exp}}^*} r(p) \leq \sum_{p \in D - D_{\text{exp}}^*} 2\text{OPT}_p + O(\varepsilon \cdot \text{opt})$ if the sample W is successful. □

4.5 Removal of Cheap Centers in S_0

At this stage we have defined a clustering $S_Q = S - Q \cup \Lambda$, where Λ is the set of dummy centers. Recall that the mixed solution M_O equals $S - S_0 \cup \text{OPT}$. So $S_Q - \Lambda$ represents progress compared to S in that we have already removed a subset Q of S_0 . Furthermore, with respect to the assignment of clients μ_O , we have that the remaining centers $S_0 - Q$, which we want to remove, are divided into two sets $\mathcal{R}$ and $\mathcal{U}$. The clusters with centers in $\mathcal{R}$ are completely assigned (by μ_O) to centers in OPT, and every cluster with center in $\mathcal{U}$ is completely assigned to a center in $S - S_0$. The task of this section is to make further progress by guessing the centers in $\mathcal{U}$ and their reassignment to centers in $S - S_0$. This

turns out to be a difficult task, and we will instead “approximately” guess a set $\tilde{\mathcal{U}}$ and reassignment $\tilde{\mu}$ of the clients of those clusters. Specifically, we will output a solution $S_{\mathcal{Q} \cup \tilde{\mathcal{U}}} = S_{\mathcal{Q}} - \tilde{\mathcal{U}}$ (obtained by removing $\tilde{\mathcal{U}}$ from $S_{\mathcal{Q}}$) and an assignment $\tilde{\mu}$ of clients to centers in $S_{\mathcal{Q}} - \tilde{\mathcal{U}}$. This is done through the following lemma. Recall that for $c \in S_{\mathcal{Q}}$, $S_{\mathcal{Q}}(c)$ is the set of clients assigned to c in $S_{\mathcal{Q}}$, i.e., closest to c among all centers in $S_{\mathcal{Q}}$.

Lemma 4.13. *Given S and $\mathcal{Q}$ as described above, there is a polynomial-time algorithm that produces a collection $\mathcal{L}_{cheap}$ of subsets $\tilde{\mathcal{U}}$ of $S - \mathcal{Q}$, and for each such $\tilde{\mathcal{U}}$ and assignment $\tilde{\mu}$ of clients to centers in $S_{\mathcal{Q} \cup \tilde{\mathcal{U}}} = S_{\mathcal{Q}} - \tilde{\mathcal{U}}$, such that at least one such pair $(\tilde{\mathcal{U}}, \tilde{\mu})$ satisfies the following properties:*

1. $|\tilde{\mathcal{U}}| = |\mathcal{U}|$.
2. $\tilde{\mathcal{U}} \cap \mathcal{R} = \emptyset$, i.e., $\tilde{\mathcal{U}}$ does not contain any center of $S_0 - \mathcal{Q}$ whose set of clients is completely assigned to a center of OPT in the assignment μ_O .
3. $\tilde{\mu}$ satisfies:
 - (a) For every center $c \in S_{\mathcal{Q}} - \tilde{\mathcal{U}}$, we have $\tilde{\mu}(p) = c$ for every $p \in S_{\mathcal{Q}}(c)$.
 - (b) For a center $c \in \tilde{\mathcal{U}}$, the clients in $S_{\mathcal{Q}}(c)$ are reassigned to a center $c' \in S - \mathcal{Q} - \tilde{\mathcal{U}} - \mathcal{R}$, i.e., $\tilde{\mu}(p) = c'$ for every $p \in S_{\mathcal{Q}}(c)$.
 - (c) The cost increase of the reassignment $\tilde{\mu}$ of clients previously assigned to $\tilde{\mathcal{U}}$ compared to that of the reassignment μ_O of clients previously assigned to $\mathcal{U}$ is bounded by $O(\epsilon \cdot \text{opt})$:

$$\sum_{c \in \mathcal{U} \cup \tilde{\mathcal{U}}} \sum_{p \in S_{\mathcal{Q}}(c)} (d(p, \tilde{\mu}(p)) - d(p, \mu_O(p))) = O(\epsilon \cdot \text{opt}).$$

In words, the above properties of $\tilde{\mu}$ say that we maintain the assignment of clients whose closest center remains the same, and the clients associated to the removed centers $\tilde{\mathcal{U}}$ are reassigned to centers not in $\mathcal{R}$ at no higher cost (up to $O(\epsilon \cdot \text{opt})$) than the cost of the reassignment of $\mathcal{U}$ in the modified solution M_O .

Let $\mathcal{X}$ be the centers in $S - S_0$ to which the points $p \in S_{\mathcal{Q}}(c)$ with $c \in \mathcal{U}$ are assigned according to μ_O . Notice that $|\mathcal{X}| \leq |\mathcal{U}|$ and $\mathcal{X} \cap (\mathcal{Q} \cup \mathcal{U} \cup \mathcal{R}) = \emptyset$. We assume that the values of $|\mathcal{U}|$, $|\mathcal{R}|$, and $|\mathcal{X}|$ are known. This can be achieved by trying, for each such value, all the integers between 0 and $|S_0| \leq \frac{\log n}{\epsilon}$. Let $\ell := |\mathcal{U}| + |\mathcal{R}| + |\mathcal{X}| \leq 2|\mathcal{U}| + |\mathcal{R}| \leq 2|S_0 - \mathcal{Q}|$. Then we run the recursive procedure $\text{cheapRem}()$ described in the box with input $(\mathcal{U}', \mathcal{R}', \mathcal{X}', \mathcal{N}, \tilde{\mathcal{U}}) = (\emptyset, \emptyset, \emptyset, \emptyset, \emptyset)$. We assume that $\text{cheapRem}()$ has access to the quantities S , $\mathcal{Q}$, $|\mathcal{U}|$, $|\mathcal{R}|$, $|\mathcal{X}|$ and ℓ , as well as to a global variable $\mathcal{L}_{cheap}$, which is initialized to $\emptyset$. The intuition for the parameters is as follows. Intuitively $\mathcal{U}'$, $\mathcal{R}'$ and $\mathcal{X}'$ are subsets of $\mathcal{U}$, $\mathcal{R}$ and $\mathcal{X}$, resp., that we have already identified, while $\tilde{\mathcal{U}}$ is the current value of the set $\tilde{\mathcal{U}}$ under construction. Intuitively, $\mathcal{N}$ are the centers to which the points $p \in S_{\mathcal{Q}}(c)$ with $c \in \tilde{\mathcal{U}}$ are reassigned, namely $\tilde{\mu}(p) \in \mathcal{N}$. At the end of the root call, the global variable $\mathcal{L}_{cheap}$ contains the desired collection of sets $\tilde{\mathcal{U}}$ as in Lemma 4.13. Whenever some $\tilde{\mathcal{U}}$ is added to $\mathcal{L}_{cheap}$, we define a corresponding $\tilde{\mu}$ as follows: For each $p \in S_{\mathcal{Q}}(c)$, $\tilde{\mu}(p) = c$ if $c \in S_{\mathcal{Q}} - \tilde{\mathcal{U}}$. Otherwise, i.e. if $c \in \tilde{\mathcal{U}}$, $\tilde{\mu}(p) = \text{next}(c) \in \mathcal{N}$, where $\text{next}(c)$ is defined in the recursive call when c is added to $\tilde{\mathcal{U}}$.

$\text{cheapRem}(\mathcal{U}', \mathcal{R}', \mathcal{X}', \mathcal{N}, \tilde{\mathcal{U}})$

- 1: **if** $|S - \mathcal{Q}| \leq 4\ell$ **then**
- 2: **for** All $\tilde{\mathcal{U}} \subseteq S - \mathcal{Q}$ of size $|\mathcal{U}|$ and $\tilde{\mathcal{R}} \subseteq S - \mathcal{Q} - \tilde{\mathcal{U}}$ of size $|\mathcal{R}|$ **do**
- 3: Add $\tilde{\mathcal{U}}$ to $\mathcal{L}_{cheap}$ and, for each $c \in \tilde{\mathcal{U}}$, set $\text{next}(c)$ to the closest center to c in $S - \mathcal{Q} - \tilde{\mathcal{U}} - \tilde{\mathcal{R}}$
- 4: **halt**
- 5: **if** $|\tilde{\mathcal{U}}| = |\mathcal{U}|$ **then**
- 6: Add $\tilde{\mathcal{U}}$ to $\mathcal{L}_{cheap}$ and **halt**
- 7: Let $c \in S - \mathcal{Q} - \mathcal{R}' - \mathcal{X}' - \mathcal{N} - \tilde{\mathcal{U}}$ minimize $R(c) := \sum_{p \in S_{\mathcal{Q}}(c)} d(p, \text{next}(c))$, where $\text{next}(c)$ is the closest center to c in $S - \mathcal{Q} - \mathcal{U}' - \mathcal{R}' - \tilde{\mathcal{U}} - \{c\}$.

```

8: if  $c \notin \mathcal{U}'$  and  $|\mathcal{X}'| < |\mathcal{X}|$  then  $\text{cheapRem}(\mathcal{U}', \mathcal{R}', \mathcal{X}' \cup \{c\}, \mathcal{N}, \tilde{\mathcal{U}})$ 
9: if  $c \notin \mathcal{U}'$  and  $|\mathcal{R}'| < |\mathcal{R}|$  then  $\text{cheapRem}(\mathcal{U}', \mathcal{R}' \cup \{c\}, \mathcal{X}', \mathcal{N}, \tilde{\mathcal{U}})$ 
10: if  $|\mathcal{U}'| + |\mathcal{R}'| = |\mathcal{U}| + |\mathcal{R}|$  or  $\text{next}(c) \in \mathcal{N}$  then  $\text{cheapRem}(\mathcal{U}', \mathcal{R}', \mathcal{X}', \mathcal{N} \cup \{\text{next}(c)\}, \tilde{\mathcal{U}} \cup \{c\})$ 
11: else
12:   if  $|\mathcal{R}'| < |\mathcal{R}|$  then  $\text{cheapRem}(\mathcal{U}', \mathcal{R}' \cup \{\text{next}(c)\}, \mathcal{X}', \mathcal{N}, \tilde{\mathcal{U}})$ 
13:   if  $|\mathcal{U}'| < |\mathcal{U}|$  then  $\text{cheapRem}(\mathcal{U}' \cup \{\text{next}(c)\}, \mathcal{R}', \mathcal{X}', \mathcal{N}, \tilde{\mathcal{U}})$ 
14:    $\text{cheapRem}(\mathcal{U}', \mathcal{R}', \mathcal{X}', \mathcal{N} \cup \{\text{next}(c)\}, \tilde{\mathcal{U}} \cup \{c\})$ 

```

Proof of Lemma 4.13. Consider the execution of the above procedure for the correct guess of the values $|\mathcal{R}|$, $|\mathcal{X}|$ and $|\mathcal{U}|$. We remark that, when line 7 is executed, the set $S - \mathcal{Q} - \mathcal{R}' - \mathcal{X}' - \mathcal{N} - \tilde{\mathcal{U}}$ is not empty (so that we can choose an appropriate c). Indeed, at each recursive call we add at most two elements to $\mathcal{R}' \cup \mathcal{X}' \cup \mathcal{N} \cup \tilde{\mathcal{U}}$ and the value of $|\mathcal{U}'| + |\mathcal{R}'| + |\mathcal{X}'| + |\tilde{\mathcal{U}}|$ grows by at least 1. The latter value cannot grow more than $2|\mathcal{U}| + |\mathcal{R}| + |\mathcal{X}| \leq 2\ell$ times since at that point we would have necessarily $|\tilde{\mathcal{U}}| = |\mathcal{U}|$, which makes the condition of line 5 true. Since the condition of line 1 cannot hold if line 7 is executed at least once, we have that $|S - \mathcal{Q}| > 4\ell$. Thus $S - \mathcal{Q}$ contains sufficiently many elements to remove up to 2 elements for up to 2ℓ times.

Concerning the running time of the procedure, each recursive step involves a branching on at most 4 subproblems, and in each one of them the value of $|\mathcal{U}'| + |\mathcal{R}'| + |\mathcal{X}'| + |\tilde{\mathcal{U}}|$ grows by at least 1. Thus, by the same argument as before, the depth of the recursion is at most 2ℓ . So the number of recursive calls is at most $4^{2\ell} = n^{1/\epsilon^{O(1)}}$, implying a polynomial running time.

It remains to show that at least one set $\tilde{\mathcal{U}} \in \mathcal{L}_{\text{cheap}}$ (with the associated $\tilde{\mu}$) at the end of the procedure satisfies the claim. If the condition of line 1 holds, it must happen that in one of the executions of line 3 one has $\tilde{\mathcal{U}} = \mathcal{U}$ and $\tilde{\mathcal{R}} = \mathcal{R}$. In that case $\tilde{\mathcal{U}}$ and the corresponding $\tilde{\mu}$ trivially satisfy the claim. In particular we remark that for each $p \in S_{\mathcal{Q}}(c)$ with $c \in \tilde{\mathcal{U}} = \mathcal{U}$, $\tilde{\mu}(p) = \text{next}(c) = \mu_{\mathcal{O}}(p)$.

Otherwise, it is sufficient to show that there exists a chain of recursive calls that leads to a solution $\tilde{\mathcal{U}}$ with the desired properties. In more detail, let us focus on the solution $\tilde{\mathcal{U}} \in \mathcal{L}_{\text{cheap}}$ which is generated by the following chain of recursive calls starting from the root call with $(\mathcal{U}', \mathcal{R}', \mathcal{X}', \mathcal{N}, \tilde{\mathcal{U}}) = (\emptyset, \emptyset, \emptyset, \emptyset, \emptyset)$. Consider the current input parameters $(\mathcal{U}', \mathcal{R}', \mathcal{X}', \mathcal{N}, \tilde{\mathcal{U}})$ and let c be selected in line 7. During the process, we will maintain the following invariants:

$$\mathcal{U}' \subseteq \mathcal{U}; \mathcal{R}' \subseteq \mathcal{R}; \mathcal{X}' \subseteq \mathcal{X}. \quad (16)$$

$$\mathcal{N} \cap (\mathcal{U} \cup \mathcal{R}) = \emptyset. \quad (17)$$

$$\tilde{\mathcal{U}} \cap (\mathcal{X} \cup \mathcal{R} \cup \mathcal{N}) = \emptyset. \quad (18)$$

The root call satisfies the mentioned invariants trivially since $\tilde{\mathcal{U}} = \mathcal{U}' = \mathcal{R}' = \mathcal{X}' = \mathcal{N} = \emptyset$.

If $c \in \mathcal{X}$, we continue with the recursive call of line 8. The invariants are maintained since $\mathcal{X}' \cup \{c\} \subseteq \mathcal{X}$ and $c \notin \tilde{\mathcal{U}}$. Observe that this choice is excluded if $c \in \mathcal{U}'$ or $|\mathcal{X}'| = |\mathcal{X}|$, however by Invariant (16) the latter conditions cannot happen when $c \in \mathcal{X}$.

Similarly, if $c \in \mathcal{R}$, we continue with the recursive call of line 9. The invariants are maintained since $\mathcal{R}' \cup \{c\} \subseteq \mathcal{R}$, $c \notin \mathcal{N}$, and $c \notin \tilde{\mathcal{U}}$. Observe that this choice is excluded if $c \in \mathcal{U}'$ or $|\mathcal{R}'| = |\mathcal{R}|$, however by Invariant (16) the latter conditions cannot happen when $c \in \mathcal{R}$.

Otherwise, namely if $c \notin \mathcal{R} \cup \mathcal{X}$, if the condition of line 10 is true, we continue with the recursive call of the same line. Let us show that invariants are preserved. Suppose first that $\text{next}(c) \in \mathcal{N}$. In this case the first two invariants are not affected, and the third one is maintained since $c \notin \mathcal{N}$ (and by assumption $c \notin \mathcal{R} \cup \mathcal{X}$). Otherwise one has $|\mathcal{U}'| + |\mathcal{R}'| = |\mathcal{U}| + |\mathcal{R}|$, which by Invariant (16) implies $\mathcal{U}' = \mathcal{U}$ and $\mathcal{R}' = \mathcal{R}$. Thus one has $\text{next}(c) \notin \mathcal{U} \cup \mathcal{R}$ and $c \notin \mathcal{R} \cup \mathcal{X} \cup \mathcal{N}$.

Otherwise, i.e., if the condition of line 10 is false, depending on whether $\text{next}(c)$ belongs to $\mathcal{R}$, $\mathcal{U}$, or none of the previous cases, we continue with the recursive calls of lines 12, 13, and 14, resp. Suppose first that $\text{next}(c) \in \mathcal{R}$. Observe that Invariant (16) guarantees that in this case $|\mathcal{R}'| < |\mathcal{R}|$ as required by the condition of line 12. Invariant (16) is maintained since $\mathcal{R}' \cup \{\text{next}(c)\} \subseteq \mathcal{R}$, while Invariants (17) and (18) are maintained since $\mathcal{N}$ and $\tilde{\mathcal{U}}$ are not modified. Suppose next that $\text{next}(c) \in \mathcal{U}$. Observe that Invariant (16) guarantees that in this case $|\mathcal{U}'| < |\mathcal{U}|$ as required by the condition of line 13. Similarly to the previous case, Invariant (16) is maintained since $\mathcal{U}' \cup \{\text{next}(c)\} \subseteq \mathcal{U}$, while

Invariants (17) and (18) are maintained since $\mathcal{N}$ and $\tilde{\mathcal{U}}$ are not modified. The remaining case is that $next(c) \notin \mathcal{R} \cup \mathcal{U}$. Invariant (17) is maintained since $next(c) \notin \mathcal{R} \cup \mathcal{U}$. Invariant (18) is maintained since $c \notin \mathcal{X} \cup \mathcal{R}$ by the assumptions of this case, $c \notin \mathcal{N} \cup \{next(c)\}$ by construction, and $c \notin \tilde{\mathcal{U}}$ by construction.

Let us show that the final $\tilde{\mathcal{U}}$ obtained with the above procedure with the associated $\tilde{\mu}$ satisfies the claim. Condition (1) is trivially satisfied. Invariant (18) directly implies property (2). We next focus on property (3). Property (a) is satisfied by definition. When a center c is added to $\tilde{\mathcal{U}}$, the respective $c' := next(c) = \tilde{\mu}(p)$ for all $p \in S_Q(c)$ is added to $\mathcal{N}$ (if not already there). Invariant (17) guarantees that $c' \notin \mathcal{R}$, while Invariant (18) guarantees that $c' \notin \tilde{\mathcal{U}}$. Property (b) follows.

It remains to prove property (c). Consider first any $c \in \mathcal{U} \cap \tilde{\mathcal{U}}$. We claim that for the corresponding $p \in S_Q(c)$ one has $\tilde{\mu}(p) = \mu_O(p)$. Indeed, let $c_1, c_2, \dots$ be the centers in $S - \mathcal{Q} - \{c\}$ in non-decreasing order of distance from c . Let also c_q be the first such center belonging to $S - S_0$. In particular $\mu_O(p) = c_q$. By construction all the centers $c_1, \dots, c_{q-1}$ must have been already added to $\mathcal{U}' \cup \mathcal{R}'$ in previous recursive steps when c is added to $\tilde{\mathcal{U}}$. Furthermore, when the latter event happens either $c_q \in \mathcal{N}$ already or c_q is added to $\mathcal{N}$. In both cases one has that $\tilde{\mu}(p) = c_q$ as desired. As a consequence

$$\sum_{c \in \mathcal{U} \cap \tilde{\mathcal{U}}} \sum_{p \in S_Q(c)} (d(p, \tilde{\mu}(p)) - d(p, \mu_O(p))) = 0. \quad (19)$$

Consider next any $c \in \mathcal{U} - \tilde{\mathcal{U}}$. For any $p \in S_Q(c)$ by construction one has $\tilde{\mu}(p) = c$. Thus

$$\begin{aligned} \sum_{c \in \mathcal{U} - \tilde{\mathcal{U}}} \sum_{p \in S_Q(c)} (d(p, \tilde{\mu}(p)) - d(p, \mu_O(p))) &\leq \sum_{c \in S_0 - \mathcal{Q}} \sum_{p \in S_Q(c)} d(p, c) - \sum_{c \in \mathcal{U} - \tilde{\mathcal{U}}} \sum_{p \in S_Q(c)} d(p, \mu_O(p)) \\ &\leq O(\varepsilon)opt - \sum_{c \in \mathcal{U} - \tilde{\mathcal{U}}} \sum_{p \in S_Q(c)} d(p, \mu_O(p)), \end{aligned} \quad (20)$$

where in the last inequality above we used the assumption that $\mathcal{Q}$ satisfies property (2) of Lemma 4.11, and the fact that $\mathcal{U} - \tilde{\mathcal{U}} \subseteq S_0 - \mathcal{Q}$.

Finally consider any $c \in \tilde{\mathcal{U}} - \mathcal{U}$. Define any bijection between each such c and some distinct $c' \in \mathcal{U} - \tilde{\mathcal{U}}$ (this is possible since $|\tilde{\mathcal{U}}| = |\mathcal{U}|$). Consider the recursive call when c is added to $\tilde{\mathcal{U}}$. Let $\mathcal{U}', \mathcal{R}', \mathcal{X}', \mathcal{N}, next(c), R(c), next(c'), R(c')$ be the associated quantities in that call. Notice that at that time c' was an available candidate to be added to $\tilde{\mathcal{U}}$ since $c' \in S - \mathcal{Q} - \mathcal{R} - \mathcal{X} - \tilde{\mathcal{U}} \subseteq S - \mathcal{Q} - \mathcal{R}' - \mathcal{X}' - \tilde{\mathcal{U}}$ (using Invariant (16)) and $\mathcal{N} \cap \mathcal{U} = \emptyset$ by Invariant (17), hence $c' \in S - \mathcal{Q} - \mathcal{R}' - \mathcal{X}' - \mathcal{N} - \tilde{\mathcal{U}}$. Since we added c instead of c' , it must be the case that $R(c) \leq R(c')$. For every $p \in S_Q(c')$, $\mu_O(p) \in \mathcal{X}$, hence $\mu_O(p) \notin \tilde{\mathcal{U}}$ by Invariant (18). It follows that $\mu_O(p) \in S - \mathcal{Q} - \mathcal{U} - \mathcal{R} - \tilde{\mathcal{U}} - \{c'\} \subseteq S - \mathcal{Q} - \mathcal{U}' - \mathcal{R}' - \tilde{\mathcal{U}} - \{c'\}$, where we used again Invariant (16). Thus $d(c', next(c')) \leq d(c', \mu_O(p))$. Hence for every $p \in S_Q(c')$,

$$d(p, next(c')) \leq d(p, c') + d(c', next(c')) \leq d(p, c') + d(c', \mu_O(p)) \leq 2d(p, c') + d(p, \mu_O(p)).$$

Summarizing,

$$\begin{aligned} \sum_{c \in \tilde{\mathcal{U}} - \mathcal{U}} \sum_{p \in S_Q(c)} (d(p, \tilde{\mu}(p)) - d(p, \mu_O(p))) &\leq \sum_{c \in \tilde{\mathcal{U}} - \mathcal{U}} \sum_{p \in S_Q(c)} d(p, \tilde{\mu}(p)) = \sum_{c \in \tilde{\mathcal{U}} - \mathcal{U}} R(c) \leq \sum_{c' \in \mathcal{U} - \tilde{\mathcal{U}}} R(c') \\ &= \sum_{c' \in \mathcal{U} - \tilde{\mathcal{U}}} \sum_{p \in S_Q(c')} d(p, next(c')) \leq \sum_{c' \in \mathcal{U} - \tilde{\mathcal{U}}} \sum_{p \in S_Q(c')} (2d(p, c') + d(p, \mu_O(p))) \\ &\leq O(\varepsilon)opt + \sum_{c \in \mathcal{U} - \tilde{\mathcal{U}}} \sum_{p \in S_Q(c)} d(p, \mu_O(p)), \end{aligned} \quad (21)$$

where in the last inequality above we used again the assumption on $\mathcal{Q}$, i.e., property (2) of Lemma 4.11. Property (c) follows by summing (19), (20), and (21). $\square$

4.5.1 Mixed Solutions after the Removal of $\tilde{\mathcal{U}}$

We let $(S_{\mathcal{Q} \cup \tilde{\mathcal{U}}}, \tilde{\mu})$ be the output of a successful run of the *cheapRem()* procedure, i.e., where $\tilde{\mathcal{U}} \in \mathcal{L}_{cheap}$ and its associated assignment $\tilde{\mu}$ satisfies the properties of Lemma 4.13. Recall that $S_{\mathcal{Q} \cup \tilde{\mathcal{U}}}$ consists of

the centers $S' = S - \mathcal{Q} - \tilde{\mathcal{U}}$ and the dummy centers Λ . Further recall that M_O consists of the centers $S - S_0 = S - \mathcal{Q} - \mathcal{U} - \mathcal{R}$ and $\hat{\text{OPT}}$. M_D is the same set of centers except that $\hat{\text{OPT}}$ is replaced by Λ . Notice that some centers of M_O and M_D are now removed if $\tilde{\mathcal{U}} - \mathcal{U} \neq \emptyset$. To take care of this, we modify these mixed solutions to obtain M'_O and M'_D .

We first define M'_O ; the definition of M'_D is then very similar. The centers of M'_O are $S - \mathcal{Q} - \tilde{\mathcal{U}} - \mathcal{R}$ and $\hat{\text{OPT}}$. Hence, the difference between M_O and M'_O is that in M_O we remove $\mathcal{U}$ from S and in M'_O we remove $\tilde{\mathcal{U}}$. In other words, $M'_O = M_O \cup \mathcal{U} - \tilde{\mathcal{U}}$. Similarly, we let $M'_D = M_D \cup \mathcal{U} - \tilde{\mathcal{U}}$. We update the assignment μ_O of M_O to an assignment μ'_O of M'_O (and the assignment μ_D to μ'_D). Recall that μ_O assigns each client p to its closest center in $\hat{\text{OPT}}$ (if $p \in D_{\text{exp}}^*$) or its closest center in $S - S_0$ (if $p \in D - D_{\text{exp}}^*$) except for those clients that belong to a cluster $S_{\mathcal{Q}}(c)$ with $c \in \mathcal{U} \cup \mathcal{R}$. Indeed, the clients in $S_{\mathcal{Q}}(c)$ are all assigned to the center of $\hat{\text{OPT}}$ that is closest to c if $c \in \mathcal{R}$, and if $c \in \mathcal{U}$ they are all assigned to the center in $S - S_0$ that is closest to c . We also recall that μ_D is the same as μ_O except when $\mu_O(p) \in \hat{\text{OPT}}$ in which case $\mu_O(p)$ is replaced by its corresponding dummy center.

Definitions of μ'_O and μ'_D . For clients $p \in S_{\mathcal{Q}}(c)$ with $c \notin \mathcal{U} \cup \tilde{\mathcal{U}} \cup \Lambda$, we define $\mu'_O(p) = \mu_O(p)$. For $p \in S_{\mathcal{Q}}(c)$ with $c \in \Lambda$, we let $\mu'_O(p)$ be the closest center in $\hat{\text{OPT}}$. Finally, for $p \in S_{\mathcal{Q}}(c)$ with $c \in \mathcal{U} \cup \tilde{\mathcal{U}}$, we define $\mu'_O(p) = \tilde{\mu}(p)$.

The assignment μ'_D is obtained in the same way from μ_D with the difference that we use Λ instead of $\hat{\text{OPT}}$: For clients $p \in S_{\mathcal{Q}}(c)$ with $c \notin \mathcal{U} \cup \tilde{\mathcal{U}} \cup \Lambda$, we define $\mu'_D(p) = \mu_D(p)$. For $p \in S_{\mathcal{Q}}(c)$ with $c \in \Lambda$, we let $\mu'_D(p)$ be the closest center in Λ . Finally, for $p \in S_{\mathcal{Q}}(c)$ with $c \in \mathcal{U} \cup \tilde{\mathcal{U}}$, we define $\mu'_D(p) = \tilde{\mu}(p)$.

This completes the definition of μ'_O and μ'_D . We remark that we have $\mu'_O(p) = \mu_O(p)$ and $\mu'_D(p) = \mu_D(p)$ for all $p \in S_{\mathcal{Q}}(c)$ with $c \notin \mathcal{U} \cup \tilde{\mathcal{U}} \cup \Lambda$.

We continue by arguing that μ'_O is well-defined, i.e., that $\mu'_O(p) \in M'_O$ for every $p \in D$ (the proof for μ'_D is the same). This is immediate for a client $p \in S_{\mathcal{Q}}(c)$ with $c \in \Lambda$. For a client $p \in S_{\mathcal{Q}}(c)$ with $c \in \mathcal{U} \cup \tilde{\mathcal{U}}$, it holds because, by Lemma 4.13, $\tilde{\mu}(p)$ equals $c \in M'_O$ if $c \in \mathcal{U} - \tilde{\mathcal{U}}$ and otherwise $\tilde{\mu}(p) \in S - \mathcal{Q} - \tilde{\mathcal{U}} - \mathcal{R} \subseteq M'_O$. It remains to verify that $\mu_O(p) \in M'_O$ for a client $p \in S_{\mathcal{Q}}(c)$ with $c \notin \mathcal{U} \cup \tilde{\mathcal{U}} \cup \Lambda$. If $c \in \mathcal{R}$, we have $\mu_O(p) \in \hat{\text{OPT}} \subseteq M'_O$. Similarly, if $c \notin \mathcal{U} \cup \tilde{\mathcal{U}} \cup \Lambda \cup \mathcal{R}$ and $p \in D_{\text{exp}}^*$ we have $\mu_O(p) \in \hat{\text{OPT}} \subseteq M'_O$. In the remaining case when $c \notin \mathcal{U} \cup \tilde{\mathcal{U}} \cup \Lambda \cup \mathcal{R}$ and $p \in D - D_{\text{exp}}^*$, $\mu_O(p)$ equals p 's closest center in $S - S_0$. As $p \in S_{\mathcal{Q}}(c)$, we thus have $\mu_O(p) = c \in S - (S_0 \cup \tilde{\mathcal{U}}) \subseteq M'_O$.

The following upper bound on $\text{cost}(M'_O, \mu'_O) + \text{cost}(M'_D, \mu'_D)$ is a fairly immediate consequence of the assumption that the cheap-removal process is successful (Property 3c of Lemma 4.13). Similarly to before, we say that the selection of $(W, \mathcal{B}, \mathcal{Q}, \tilde{\mathcal{U}}, \tilde{\mu})$ is successful, if the sample W selected in 4.2 is successful, the set of balls $\mathcal{B}$ from Section 4.3 is valid, $\mathcal{Q}$ selected in Section 4.4 satisfies the properties of Lemma 4.11, and $\tilde{\mathcal{U}}, \tilde{\mu}$ selected in this section satisfies the properties of Lemma 4.13.

Claim 7. *If $(W, \mathcal{B}, \mathcal{Q}, \tilde{\mathcal{U}}, \tilde{\mu})$ is successful,*

$$\text{cost}(M'_O, \mu'_O) + \text{cost}(M'_D, \mu'_D) \leq \sum_{p \in D} 4\text{OPT}_p + O(\varepsilon \cdot \text{opt}).$$

Proof. For $x \in \{O, D\}$ the only differences between μ'_x and μ_x are clients in $S_{\mathcal{Q}}(c)$ with $c \in \Lambda \cup \mathcal{U} \cup \tilde{\mathcal{U}}$. Consider first a client $p \in S_{\mathcal{Q}}(c)$ with $c \in \Lambda$. Then, as c is the closest center among $(S - \mathcal{Q}) \cup \Lambda$, which both contains $M_O - \hat{\text{OPT}}$ and $M'_O - \hat{\text{OPT}}$, we have that the closest center to p in both M'_O and M_O is in $\hat{\text{OPT}}$ (because by the definition of dummy centers, $d(p, \hat{\text{OPT}}) \leq d(p, \Lambda)$). Similarly, the closest center to p in both M'_D and M_D is in Λ . Hence, the definitions of μ'_O and μ'_D to assign p to its closest center in $\hat{\text{OPT}}$ and Λ , respectively, cannot increase the cost, i.e., $d(p, \mu'_O(p)) \leq d(p, \mu_O(p))$ and $d(p, \mu'_D(p)) \leq d(p, \mu_D(p))$ for such a client p .

Finally, for those clients $p \in S_{\mathcal{Q}}(c)$ with $c \in \mathcal{U} \cup \tilde{\mathcal{U}}$ we have $\mu'_x = \tilde{\mu}$ by definition and $d(p, \mu_O(p)) \leq d(p, \mu_D(p))$ since $\mu_O(p) = \mu_D(p)$ unless $\mu_O(p) \in \hat{\text{OPT}}$ in which case $\mu_D(p)$ is the dummy center associated with $\mu_O(p)$ that can only be farther away from p than $\mu_O(p)$. Hence:

$$\sum_{c \in \mathcal{U} \cup \tilde{\mathcal{U}}} \sum_{p \in S_{\mathcal{Q}}(c)} (d(p, \tilde{\mu}(p)) - d(p, \mu_D(p))) \leq \sum_{c \in \mathcal{U} \cup \tilde{\mathcal{U}}} \sum_{p \in S_{\mathcal{Q}}(c)} (d(p, \tilde{\mu}(p)) - d(p, \mu_O(p))) = O(\varepsilon \cdot \text{opt}),$$

where in the equality we used Property (c) of Lemma 4.13. The claim follows since

$$\text{cost}(M'_O, \mu'_O) + \text{cost}(M'_D, \mu'_D) \leq \text{cost}(M_O, \mu_O) + \text{cost}(M_D, \mu_D) + O(\varepsilon \cdot \text{opt}) \stackrel{\text{Lem. 4.12}}{\leq} 4\text{OPT}_p + O(\varepsilon \cdot \text{opt}).$$

□

To better understand $\text{cost}(M'_O, \mu'_O)$, let us consider the cost $d(p, \mu'_O(p))$ of a single client p :

- If $p \in S_Q(c)$ with $c \notin \tilde{\mathcal{U}} \cup \mathcal{R}$ then $\tilde{\mu}(p) = c$ by Property 3a of Lemma 4.13. Moreover, $\mu'_O(p)$ assigns p either to a center in OPT or to one in $S - \mathcal{Q} - \tilde{\mathcal{U}} - \mathcal{R}$ and we have $d(p, c) \leq d(p, S - \mathcal{Q} - \tilde{\mathcal{U}} - \mathcal{R})$. Hence, in either case, we have

$$d(p, \mu'_O(p)) \geq d(p, \{\tilde{\mu}(p)\} \cup \text{OPT}) = d(p, \{\tilde{\mu}(p)\} \cup \text{OPT} \cup \Lambda).$$

- If $p \in S_Q(c)$ with $c \in \mathcal{R}$ then $\mu'_O(p) = \mu_O(p) = c^* \in \text{OPT}$ where c^* is the center of OPT that is closest to c . So

$$d(p, \mu'_O(p)) = d(p, c^*).$$

- If $p \in S_Q(c)$ with $c \in \tilde{\mathcal{U}}$ then $\mu'_O(p) = \tilde{\mu}(p)$ and so

$$d(p, \mu'_O(p)) \geq d(p, \{\tilde{\mu}(p)\} \cup \text{OPT} \cup \Lambda).$$

Similarly, we can analyze $d(p, \mu'_D(p))$ by replacing OPT with the set Λ of dummy centers (and c^* by its associated dummy center). Summarizing, we have

$$\text{cost}(M'_O, \mu'_O) \geq \sum_{c \in S_Q - \mathcal{R}} \sum_{p \in S_Q(c)} d(p, \{c\} \cup \text{OPT} \cup \Lambda) + \sum_{c \in \mathcal{R}} \min_{c' \in \text{OPT}} \sum_{p \in S_Q(c)} d(p, c')$$

and

$$\text{cost}(M'_D, \mu'_D) \geq \sum_{c \in S_Q - \mathcal{R}} \sum_{p \in S_Q(c)} d(p, \{c\} \cup \Lambda) + \sum_{c \in \mathcal{R}} \min_{c' \in \Lambda} \sum_{p \in S_Q(c)} d(p, c')$$

Furthermore, by Properties 3a and 3b of Lemma 4.13, we have that, for every $c \in \mathcal{R}$, the set $S_Q(c)$ of clients assigned to c in S_Q equals the set $\tilde{\mu}^{-1}(c)$ of clients assigned by $\tilde{\mu}$. So, if we let $\{C_c\}_{c \in S_Q \cup \tilde{\mathcal{U}}}$ be the partitioning of the set D of clients according to $\tilde{\mu}$, i.e., $C_c = \{p \in D \mid \tilde{\mu}(p) = c\}$, we can thus rewrite the above bounds on the cost as

$$\text{cost}(M'_O, \mu'_O) \geq \sum_{c \notin \mathcal{R}} \sum_{p \in C_c} d(p, \{c\} \cup \text{OPT} \cup \Lambda) + \sum_{c \in \mathcal{R}} \min_{c' \in \text{OPT}} \sum_{p \in C_c} d(p, c') \quad (22)$$

and

$$\text{cost}(M'_D, \mu'_D) \geq \sum_{c \notin \mathcal{R}} \sum_{p \in C_c} d(p, \{c\} \cup \Lambda) + \sum_{c \in \mathcal{R}} \min_{c' \in \Lambda} \sum_{p \in C_c} d(p, c'). \quad (23)$$

In the next section, we use these bounds to give a polynomial-time algorithm that outputs a solution S^* with k centers so that (see Lemma 4.14)

$$\text{cost}(S^*) \leq \frac{(1 + 2\varepsilon)}{2} (\text{cost}(M'_O, \mu'_O) + \text{cost}(M'_D, \mu'_D)) + O(\varepsilon \text{opt}) \leq \sum_{p \in D} 2\text{OPT}_p + O(\varepsilon \cdot \text{opt})$$

where the second inequality is by Claim 7. So the proof of Lemma 4.14 in the next subsection is the final step in the proof of Theorem 1.3 (see also Section 4.7 where we put everything together to prove Theorem 1.3).

4.6 Finding S^* via Submodular Optimization

In this section, we give a polynomial-time algorithm for finding the solution S^* by reducing the problem to maximizing a submodular function subject to a partition matroid constraint. Specifically, we prove the following lemma:

Lemma 4.14. *We can in polynomial-time find a clustering S^* with k centers of cost at most*

$$\frac{(1+2\varepsilon)}{2} (\text{cost}(M'_O, \mu'_O) + \text{cost}(M'_D, \mu'_D)) + O(\varepsilon \text{opt}).$$

Recall that at this point the algorithm calculated the local search solution S , sampled W , guessed $\mathcal{B}$, $\mathcal{Q}$, and $\tilde{\mathcal{U}}$ with the assignment $\tilde{\mu}$ of the clustering $S_{\mathcal{Q} \cup \tilde{\mathcal{U}}}$. We assume that all these choices were successful guesses, i.e., that $(W, \mathcal{B}, \mathcal{Q}, \tilde{\mathcal{U}}, \tilde{\mu})$ is successful. Further recall the notation that we partition the clients into the clusters $\{C_c\}_{c \in S_{\mathcal{Q} \cup \tilde{\mathcal{U}}}}$ where $C_c = \{p \in D : \tilde{\mu}(p) = c\}$.

Our goal is to give a polynomial-time algorithm that finds approximations of the sets OPT and $\mathcal{R}$ that only have slightly worse cost. We start by defining the feasible set of candidates for OPT as a partition matroid. Recall that OPT contains one center from each ball in $\mathcal{B}$.

Definition of partition matroid $\mathcal{M}_{\mathcal{B}}$. We define the partition matroid that captures the constraint that we wish to open a center in each ball in $\mathcal{B}$. Let $F_{\mathcal{B}}$ be the (multi) subset of facility/center locations containing $B \cap F$ for each ball $B \in \mathcal{B}$. If a center c is in multiple balls in $\mathcal{B}$, then $F_{\mathcal{B}}$ contains one distinct copy of c for each ball. We let $F_{\mathcal{B}}(B) \subseteq F_{\mathcal{B}}$ denote the facilities associated with $B \in \mathcal{B}$. These sets satisfy the following two properties:

- The sets $F_{\mathcal{B}}(B)$ partition $F_{\mathcal{B}}$.
- For $B \in \mathcal{B}$, $F_{\mathcal{B}}(B)$ contains (a copy of) every facility in $F \cap B$.

The first property holds since we took a unique copy of each center for each ball, and they are thus disjoint: $F_{\mathcal{B}}(B) \cap F_{\mathcal{B}}(B') = \emptyset$ for distinct $B, B' \in \mathcal{B}$. Indeed, while it is not better for the cost to open multiple copies of a center, we make the copies to ensure the above two properties. This allows us to define the partition matroid $\mathcal{M}_{\mathcal{B}} = (F_{\mathcal{B}}, \mathcal{I})$ where

$$\mathcal{I} = \{X \subseteq F_{\mathcal{B}} : |X \cap F_{\mathcal{B}}(B)| \leq 1 \text{ for every } B \in \mathcal{B}\}.$$

Moreover, since there is exactly one ball in $\mathcal{B}$ for each center in OPT , we have $\text{OPT} \in \mathcal{I}$ (where we slightly abuse notation as we should take the copy of center $c^* \in \text{OPT}$ that belongs to its associated ball).

Core, concentrated and hit clusters. Our goal is now to define a submodular function f so that we obtain a good approximation to OPT by maximizing f over the matroid constraint $\mathcal{M}_{\mathcal{B}}$. In particular, the domain of f is every subset of $F_{\mathcal{B}}$. However, we need some additional steps before defining f . In particular, we introduce the concept of the core of a cluster C_c and the notions of concentrated and hit clusters, which allow us to simplify the structure of centers in $\mathcal{R}$. For a cluster C_c , we define the *core* of C_c as

$$C_c^{\text{core}} = \left\{ p \in C_c : d(p, c) \leq \varepsilon \cdot \frac{\text{cost}(S)}{|\mathcal{R}| \cdot |C_c|} \right\}.$$

We further say that cluster C_c is *concentrated* if

$$|C_c^{\text{core}}| \geq (1 - \varepsilon)|C_c|$$

and it is *hit* by a set X of centers if there is a point $p \in C_c^{\text{core}}$ such that

$$d(p, X) < d(p, c).$$

For shortness we will sometimes say that a center is concentrated (resp., hit), if the corresponding cluster is so.

Guessing the centers of $\mathcal{R}$ that are not concentrated. We further simplify the task of finding the set of centers $\mathcal{R}$ by guessing the centers of $\mathcal{R}$ that are not concentrated. Specifically, partition the set $\mathcal{R}$ into $\mathcal{R}_0$ and $\mathcal{R}_1$, where $\mathcal{R}_1$ contain those centers of $\mathcal{R}$ that are concentrated and $\mathcal{R}_0$ contains those that are not. We can correctly guess $\mathcal{R}_0$ in polynomial time since the following simple claim shows that it is a subset of the centers whose cluster costs at least $\varepsilon^2 \cdot \text{opt}/|\mathcal{R}|$, of which there are only $O(|\mathcal{R}|/\varepsilon^2)$ many, and so guessing $\mathcal{R}_0$ can thus be done in time $2^{O(|\mathcal{R}|/\varepsilon^2)}$.

Claim 8. Suppose that C_c is not concentrated. Then the cost of C_c is at least $\varepsilon^2 \cdot \text{opt}/|\mathcal{R}|$.

Proof. Since C_c is not concentrated we have $|C_c - C_c^{\text{core}}| \geq \varepsilon|C_c|$. Moreover, each point $p \in C_c - C_c^{\text{core}}$ has $d(p, c) > \varepsilon \frac{\text{cost}(S)}{|\mathcal{R}| \cdot |C_c|}$ by definition. Hence, as $\text{cost}(S) \geq \text{opt}$,

$$\sum_{p \in C_c} d(p, c) \geq \sum_{p \in C_c - C_c^{\text{core}}} d(p, c) \geq |C_c - C_c^{\text{core}}| \cdot \varepsilon \frac{\text{opt}}{|\mathcal{R}| \cdot |C_c|} \geq \varepsilon^2 \frac{\text{opt}}{|\mathcal{R}|}.$$

□

Now let $\mathcal{P}$ be the potential centers that can be in $\mathcal{R}$. Specifically, we let $\mathcal{P}$ be the set that contains a center c in $S - \mathcal{Q} - \tilde{\mathcal{U}}$ if C_c equals the set of clients in $S_{\mathcal{Q}}(c)$, where we recall that $S_{\mathcal{Q}}(c)$ is the set of clients that are closest to c in the clustering $S_{\mathcal{Q}}$. Notice that the algorithm has all the information $S, \mathcal{B}, \mathcal{Q}, \tilde{\mathcal{U}}$ and $\tilde{\mu}$ to calculate $\mathcal{P}$. Furthermore, by Lemma 4.13, we have that no client from $\tilde{\mathcal{U}}$ was reassigned by $\tilde{\mu}$ to a center in $\mathcal{R}$. So for $c \in \mathcal{R}$ we have $C_c = S_{\mathcal{Q}}(c)$. Hence, $\mathcal{R} \subseteq \mathcal{P}$ and we can guess $\mathcal{R}_0$ as follows:

Guessing $\mathcal{R}_0$

1. Let $\mathcal{C}$ be the clusters of $\mathcal{P}$ whose cost is at least $\varepsilon^2 \cdot \text{opt}/|\mathcal{R}|$.
2. Output each subset of $\mathcal{C}$.

By the above claim and the definition of $\mathcal{P}$, one of the outputs is $\mathcal{R}_0$. Furthermore, the above guessing procedure outputs a family of polynomial many subsets. Indeed, we have $|\mathcal{R}| \leq \log(n)/\varepsilon^3$ by Lemma 4.3. Moreover, the total cost of the clusters in $\mathcal{P}$ is $O(\text{opt})$. To see this notice that the clusters corresponding to $\mathcal{P}$ is a subset of the clusters of $S_{\mathcal{Q}}$ and the cost of $S_{\mathcal{Q}}$ is at most $\text{cost}(S - S_0 \cup \Lambda)$ (since $S_{\mathcal{Q}} \supseteq S - S_0 \cup \Lambda$), which has cost at most $O(\text{opt})$ by Lemma 4.9. This implies that $|\mathcal{C}| = O(\log(n)/\varepsilon^5)$, so the total number of subsets is $n^{1/\varepsilon^{O(1)}}$. The algorithm proceeds by trying all possible subsets in the output, and we analyze the algorithm when it takes the correct guess of $\mathcal{R}_0$.

Definition of the submodular function f . We first define another function g on the same domain as f , i.e., on all subsets X of $F_{\mathcal{B}}$. We will then define f by $f(X) = g(\emptyset) - g(X)$. Let the *closed cost* of a cluster C_c be defined as

$$\text{closedcost}_c(X) := \begin{cases} \sum_{p \in C_c} d(p, \{c\} \cup X \cup \Lambda) & \text{if } C_c \text{ is hit by } X, \\ \min_{c' \in X \cup \Lambda} \sum_{p \in C_c^{\text{core}}} d(p, c') + \sum_{p \in C_c - C_c^{\text{core}}} d(p, \{c\} \cup X \cup \Lambda) & \text{otherwise.} \end{cases}$$

Further, let $\mathcal{P}_1$ be the potential centers for $\mathcal{R}_1$: it contains each center $c \in \mathcal{P} - \mathcal{R}_0$ so that c is concentrated, i.e., $|C_c^{\text{core}}| \geq (1 - \varepsilon)|C_c|$. We remark that the algorithm can calculate this set $\mathcal{P}_1$ as it only depends on $\mathcal{P}$, the guessed set $\mathcal{R}_0$, the value $\text{cost}(S)$, and the clusters C_c defined by $\tilde{\mu}$. Moreover, by definition, we have $\mathcal{R}_1 \subseteq \mathcal{P}_1$. For a subset $X \subseteq F_{\mathcal{B}}$, we then define $g(X)$ to be the minimum value of

$$\sum_{c \notin \mathcal{R}_0 \cup \mathcal{R}'_1} \sum_{p \in C_c} d(p, \{c\} \cup X \cup \Lambda) + \sum_{c \in \mathcal{R}_0} \sum_{p \in C_c} d(p, X \cup \Lambda) + \sum_{c \in \mathcal{R}'_1} \text{closedcost}_c(X)$$

over all subsets $\mathcal{R}'_1 \subseteq \mathcal{P}_1$ with $|\mathcal{R}'_1| = |\mathcal{R}_1| = |\mathcal{R}| - |\mathcal{R}_0|$. In words, over the best $\mathcal{R}'_1$, $g(X)$ is the cost of the solution obtained by removing the centers $\mathcal{R}_0 \cup \mathcal{R}'_1$ and assigning clients as follows:

- If $p \in C_c$ for a remaining center $c \notin \mathcal{R}_0 \cup \mathcal{R}'_1$ or $p \notin C_c^{\text{core}}$ with $c \in \mathcal{R}'_1$, p is assigned to its closest center in $\{c\} \cup X \cup \Lambda$.

- If $p \in C_c$ for a removed center $c \in \mathcal{R}_0$, p is assigned to its closest center in $X \cup \Lambda$.
- If $p \in C_c^{\text{core}}$ for a removed center $c \in \mathcal{R}'_1$, p is assigned to its closest center in $\{c\} \cup X \cup \Lambda$ if X hits C_c , and otherwise all clients in C_c^{core} are assigned to the same center $c' \in X \cup \Lambda$ that minimizes the cost.

We remark that this assignment is infeasible in the sense that it may assign clients to removed centers in $\mathcal{R}'_1$. Nevertheless, we relate the values of $g(\emptyset)$ and $g(\text{OPT})$ to $\text{cost}(M'_D, \mu'_D)$ and $\text{cost}(M'_O, \mu'_O)$, respectively; and, we show that given an $X \subseteq F_B$ that is independent in M_B , i.e., $X \in \mathcal{I}$, we can in polynomial-time output k centers whose cost is at most $(1 + 2\varepsilon)g(X) + 15\varepsilon\text{opt}$. Finally, the definition of g allows us to prove that f (defined by $f(X) = g(\emptyset) - g(X)$) is a monotone submodular function.

Lemma 4.15. *We have that g and f satisfy the following properties:*

1. We can evaluate $g(X)$ in polynomial time for every $X \subseteq F_B$, and we can thus evaluate $f(X)$ in polynomial time.
2. f is a non-negative monotone submodular function.
3. The value $g(\emptyset)$ is at most $\text{cost}(M'_D, \mu'_D)$.
4. The value $g(\text{OPT})$ is at most $\text{cost}(M'_O, \mu'_O)$.
5. Given an $X \subseteq F_B$ that is independent in M_B , i.e., $X \in \mathcal{I}$, we can in polynomial-time output k centers S^* whose associated cost is at most $(1 + 2\varepsilon)g(X) + 15\varepsilon\text{opt}$.

We give the proof of the lemma in the next subsection. We explain here how it implies Lemma 4.14.

Proof of Lemma 4.14. By the first property of Lemma 4.15, we can evaluate f in polynomial time. Moreover, it is easy to see that we can answer independence queries in polynomial time for any partition matroid, particularly for M_B . We can thus apply Theorem 4.1 on f and M_B with $\zeta = (1 - 1/e - 1/2)$ to find a solution $X \in \mathcal{I}$ such that⁵

$$g(\emptyset) - g(X) = f(X) \geq (1 - 1/e - \zeta)f(\text{OPT}) = (1 - 1/e - \zeta)(g(\emptyset) - g(\text{OPT})) = \frac{1}{2}(g(\emptyset) - g(\text{OPT})),$$

where we used that OPT is one feasible solution, i.e., $\text{OPT} \in \mathcal{I}$. This in turn implies that

$$\frac{1}{2}(g(\text{OPT}) + g(\emptyset)) \geq g(X).$$

Using the upper bounds on $g(\emptyset)$ and $g(\text{OPT})$ of Lemma 4.15 we thus have found a set X such that

$$\frac{1}{2}(\text{cost}(M'_O, \mu'_O) + \text{cost}(M'_D, \mu'_D)) \geq g(X).$$

Now using the last property of Lemma 4.15 we can output a solution S^* whose cost is at most

$$\frac{(1 + 2\varepsilon)}{2}(\text{cost}(M'_O, \mu'_O) + \text{cost}(M'_D, \mu'_D)) + O(\varepsilon\text{opt}).$$

as required.

Finally, each of the above steps runs in polynomial time: the algorithm of Theorem 4.1 is polynomial time, and the last property of Lemma 4.15 used to obtain S^* is polynomial-time. Moreover, the number of guesses of $\mathcal{R}_0$ is at most $n^{1/\varepsilon^{O(1)}}$, as argued after the description of that procedure. So we can, in polynomial time, try all possibilities and, among all solutions found (one for each guess of $\mathcal{R}_0$), return one that minimizes the cost and, in particular, has cost at most that of S^* (which was analyzed assuming the guess of $\mathcal{R}_0$ was correct). We thus have a polynomial time algorithm that returns a solution that satisfies the guarantee of the lemma. $\square$

⁵We remark that we could select ζ to be arbitrarily small and get a better guarantee in the statement of Lemma 4.14. We have chosen this value to simplify the calculations as improving the guarantee of Lemma 4.14 does not improve the overall result.

4.6.1 Proof of Lemma 4.15

Proof of Property 1. Given $X \subseteq F_B$, we argue that we can evaluate $g(X)$ in polynomial time. For a center $c \in \mathcal{P}_1$, define

$$\text{cost-inc}_c(X) = \text{closedcost}_c(X) - \sum_{p \in C_c} d(p, \{c\} \cup X \cup \Lambda).$$

In other words, using that $d(p, c) \leq d(p, X)$ for $p \in C_c^{\text{core}}$ if C_c is not hit by X ,

$$\text{cost-inc}_c(X) = \begin{cases} 0 & \text{if } C_c \text{ is hit by } X, \\ \min_{c' \in X \cup \Lambda} \sum_{p \in C_c^{\text{core}}} (d(p, c') - d(p, \{c\} \cup \Lambda)) & \text{otherwise.} \end{cases} \quad (24)$$

We remark that $\text{cost-inc}_c(X) \geq 0$ by definition. Let D_0 denote the clients belonging to clusters C_c with $c \in \mathcal{R}_0$. With this notation, $g(X)$ is the minimum value

$$\sum_{p \in D-D_0} d(p, \{\tilde{\mu}(p)\} \cup X \cup \Lambda) + \sum_{p \in D_0} d(p, X \cup \Lambda) + \sum_{c \in \mathcal{R}'_1} \text{cost-inc}_c(X) \quad (25)$$

over all subsets $\mathcal{R}'_1 \subseteq \mathcal{P}_1$ with $|\mathcal{R}'_1| = |\mathcal{R}_1|$. We further have that the value of $\text{cost-inc}_c(X)$ for $c \in \mathcal{R}'_1$ is independent of other centers in $\mathcal{R}'_1$. We can thus obtain the best choice of $\mathcal{R}'_1$ by evaluating $\text{cost-inc}_c(X)$ for every $c \in \mathcal{P}_1$ and choose the $|\mathcal{R}_1|$ ones of smallest value. Notice that the algorithm knows $|\mathcal{R}_1|$ since it equals $|S_0| - |\mathcal{Q}| - |\tilde{\mathcal{U}}| - |\mathcal{R}_0| = |\mathcal{B}| - |\mathcal{Q}| - |\tilde{\mathcal{U}}| - |\mathcal{R}_0|$. After we have obtained $\mathcal{R}'_1$ in polynomial time, we can evaluate $g(X)$ in polynomial time by simply calculating the sum.

Proof of Property 2. In the proof of this property it will be convenient to use definition (25) of g . We start by verifying that f is monotone. I.e., that $f(Y) \geq f(X)$ when $X \subseteq Y$, which is equivalent to verifying that $g(Y) \leq g(X)$. Let $g(X)$ equal

$$\sum_{p \in D-D_0} d(p, \{\tilde{\mu}(p)\} \cup X \cup \Lambda) + \sum_{p \in D_0} d(p, X \cup \Lambda) + \sum_{c \in \mathcal{R}'_1} \text{cost-inc}_c(X)$$

for some set $\mathcal{R}'_1$. Then $g(Y)$ is at most

$$\sum_{p \in D-D_0} d(p, \{\tilde{\mu}(p)\} \cup Y \cup \Lambda) + \sum_{p \in D_0} d(p, Y \cup \Lambda) + \sum_{c \in \mathcal{R}'_1} \text{cost-inc}_c(Y)$$

As trivially $d(p, \{\tilde{\mu}(p)\} \cup Y \cup \Lambda) \leq d(p, \{\tilde{\mu}(p)\} \cup X \cup \Lambda)$, $d(p, Y \cup \Lambda) \leq d(p, X \cup \Lambda)$ and $\text{cost-inc}_c(Y) \leq \text{cost-inc}_c(X)$ (see (24)), we have $g(Y) \leq g(X)$ as required. Moreover, as $f(\emptyset) = 0$ by definition, non-negativity follows.

We proceed to verify that f is submodular, i.e., that for every $X \subseteq Y \subseteq F_B$ and $c_0 \in F_B - Y$, $f(X \cup \{c_0\}) - f(X) \geq f(Y \cup \{c_0\}) - f(Y)$, or equivalently

$$g(X \cup \{c_0\}) - g(X) \leq g(Y \cup \{c_0\}) - g(Y). \quad (26)$$

Let $\mathcal{R}_1^X, \mathcal{R}_1^Y$, and $\mathcal{R}_1^{Y+c_0}$ be subsets of $\mathcal{P}_1$ of cardinality $|\mathcal{R}_1|$ such that

$$\begin{aligned} g(X) &= \sum_{p \in D-D_0} d(p, \{\tilde{\mu}(p)\} \cup X \cup \Lambda) + \sum_{p \in D_0} d(p, X \cup \Lambda) + \sum_{c \in \mathcal{R}_1^X} \text{cost-inc}_c(X) \\ g(Y) &= \sum_{p \in D-D_0} d(p, \{\tilde{\mu}(p)\} \cup Y \cup \Lambda) + \sum_{p \in D_0} d(p, Y \cup \Lambda) + \sum_{c \in \mathcal{R}_1^Y} \text{cost-inc}_c(Y) \\ g(Y \cup \{c_0\}) &= \sum_{p \in D-D_0} d(p, \{\tilde{\mu}(p)\} \cup (Y \cup \{c_0\}) \cup \Lambda) + \sum_{p \in D_0} d(p, (Y \cup \{c_0\}) \cup \Lambda) + \sum_{c \in \mathcal{R}_1^{Y+c_0}} \text{cost-inc}_c(Y \cup \{c_0\}) \end{aligned}$$

We shall define $\mathcal{R}_1^{X+c_0}$ to be a subset of $\mathcal{P}_1$ of cardinality $|\mathcal{R}_1|$ so that the upper bound

$$g(X \cup \{c_0\}) \leq \sum_{p \in D-D_0} d(p, \{\tilde{\mu}(p)\} \cup (X \cup \{c_0\}) \cup \Lambda) + \sum_{p \in D_0} d(p, (X \cup \{c_0\}) \cup \Lambda) + \sum_{c \in \mathcal{R}_1^{X+c_0}} \text{cost-inc}_c(X \cup \{c_0\})$$

allows us to verify Inequality (26). First notice that no matter the definition of $\mathcal{R}_1^{X+c_0}$, we have that

$$\sum_{p \in D-D_0} (d(p, \{\tilde{\mu}(p)\} \cup (X \cup \{c_0\}) \cup \Lambda) - d(p, \{\tilde{\mu}(p)\} \cup X \cup \Lambda)) + \sum_{p \in D_0} (d(p, (X \cup \{c_0\}) \cup \Lambda) - d(p, X \cup \Lambda)) \quad (27)$$

is upper bounded by

$$\sum_{p \in D-D_0} (d(p, \{\tilde{\mu}(p)\} \cup (Y \cup \{c_0\}) \cup \Lambda) - d(p, \{\tilde{\mu}(p)\} \cup Y \cup \Lambda)) + \sum_{p \in D_0} (d(p, (Y \cup \{c_0\}) \cup \Lambda) - d(p, Y \cup \Lambda)). \quad (28)$$

Indeed, if we let $D' \subseteq D - D_0$ be the subset of clients for which $d(p, c_0) < d(p, \{\tilde{\mu}(p)\} \cup Y \cup \Lambda)$ and $D'_0 \subseteq D_0$ be the subset of clients for which $d(p, c_0) < d(p, Y \cup \Lambda)$, then (28) equals

$$\sum_{p \in D'} (d(p, c_0) - d(p, \{\tilde{\mu}(p)\} \cup Y \cup \Lambda)) + \sum_{p \in D'_0} (d(p, c_0) - d(p, Y \cup \Lambda))$$

and we have

$$(27) \leq \sum_{p \in D'} (d(p, c_0) - d(p, \{\tilde{\mu}(p)\} \cup X \cup \Lambda)) + \sum_{p \in D'_0} (d(p, c_0) - d(p, X \cup \Lambda)).$$

It follows that (27) is at most (28) because

$$\sum_{p \in D'} (d(p, \{\tilde{\mu}(p)\} \cup Y \cup \Lambda) - d(p, \{\tilde{\mu}(p)\} \cup X \cup \Lambda)) + \sum_{p \in D'_0} (d(p, Y \cup \Lambda) - d(p, X \cup \Lambda)) \leq 0.$$

To prove that f is submodular, it is thus sufficient to define $\mathcal{R}_1^{X+c_0}$ so that

$$\sum_{c \in \mathcal{R}_1^{X+c_0}} \text{cost-inc}_c(X \cup \{c_0\}) - \sum_{c \in \mathcal{R}_1^X} \text{cost-inc}_c(X) \leq \sum_{c \in \mathcal{R}_1^{Y+c_0}} \text{cost-inc}_c(Y \cup \{c_0\}) - \sum_{c \in \mathcal{R}_1^Y} \text{cost-inc}_c(Y). \quad (29)$$

To this end, let Δ_{new} contain the centers in $\mathcal{R}_1^{Y+c_0} - \mathcal{R}_1^Y$ and those centers $c \in \mathcal{R}_1^{Y+c_0} \cap \mathcal{R}_1^Y$ with $\text{cost-inc}_c(Y \cup \{c_0\}) < \text{cost-inc}_c(Y)$. Notice that we may assume that, for each $c \in \Delta_{\text{new}}$,

$$\text{cost-inc}_c(Y \cup \{c_0\}) = \text{cost-inc}_c(\{c_0\}) = \begin{cases} 0 & \text{if } C_c \text{ is hit by } c_0, \\ \sum_{p \in C_c^{\text{core}}} (d(p, c_0) - d(p, \{c\} \cup \Lambda)) & \text{otherwise.} \end{cases}$$

because the addition of c_0 caused a decrease in $\text{cost-inc}(\cdot)$ for these clusters. Further, let Δ_{old}^Y contain $\Delta_{\text{new}} \cap \mathcal{R}_1^Y$ and the centers $\mathcal{R}_1^Y - \mathcal{R}_1^{Y+c_0}$. So $\mathcal{R}_1^{Y+c_0} = (\mathcal{R}_1^Y - \Delta_{\text{old}}^Y) \cup \Delta_{\text{new}}$ and $|\Delta_{\text{new}}| = |\Delta_{\text{old}}^Y|$. We now define $\mathcal{R}_1^{X+c_0} = (\mathcal{R}_1^X - \Delta_{\text{old}}^X) \cup \Delta_{\text{new}}$ where Δ_{old}^X is obtained as follows:

- Initialize $\Delta_{\text{old}}^X = \mathcal{R}_1^X \cap \Delta_{\text{new}}$.
- While $|\Delta_{\text{old}}^X| < |\Delta_{\text{new}}|$, add a center according to the following priorities:
 - If there is a center $c \in \mathcal{R}_1^X - \Delta_{\text{old}}^X$ so that $c \notin \mathcal{R}_1^Y$, add c to Δ_{old}^X .
 - Else add a center $c \in \mathcal{R}_1^X - \Delta_{\text{old}}^X$ with $c \in \Delta_{\text{old}}^Y$.

We claim that the above is well-defined, i.e., that if $|\Delta_{\text{old}}^X| < |\Delta_{\text{new}}|$ and there is no center $c \in \mathcal{R}_1^X - \Delta_{\text{old}}^X$ so that $c \notin \mathcal{R}_1^Y$, then there must be a center $c \in \mathcal{R}_1^X - \Delta_{\text{old}}^X$ with $c \in \Delta_{\text{old}}^Y$. This is because $|\Delta_{\text{old}}^X| < |\Delta_{\text{new}}| = |\Delta_{\text{old}}^Y|$ and all remaining centers $\mathcal{R}_1^X - \Delta_{\text{old}}^X$ are in $\mathcal{R}_1^Y$ in this case, so at least one of them must be in Δ_{old}^Y (recall that $|\mathcal{R}_1^X| = |\mathcal{R}_1^Y|$).

With this notation we have

$$\begin{aligned} \sum_{c \in \mathcal{R}_1^{Y+c_0}} \text{cost-inc}_c(Y \cup \{c_0\}) - \sum_{c \in \mathcal{R}_1^Y} \text{cost-inc}_c(Y) &= \sum_{c \in \Delta_{\text{new}}^Y} \text{cost-inc}_c(Y \cup \{c_0\}) - \sum_{c \in \Delta_{\text{old}}^Y} \text{cost-inc}_c(Y) \\ &= \sum_{c \in \Delta_{\text{new}}^Y} \text{cost-inc}_c(\{c_0\}) - \sum_{c \in \Delta_{\text{old}}^Y} \text{cost-inc}_c(Y). \end{aligned} \quad (30)$$

Similarly,

$$\begin{aligned} \sum_{c \in \mathcal{R}_1^{X+c_0}} \text{cost-inc}_c(X \cup \{c_0\}) - \sum_{c \in \mathcal{R}_1^X} \text{cost-inc}_c(X) &\leq \sum_{c \in \Delta_{\text{new}}^X} \text{cost-inc}_c(X \cup \{c_0\}) - \sum_{c \in \Delta_{\text{old}}^X} \text{cost-inc}_c(X) \\ &= \sum_{c \in \Delta_{\text{new}}^X} \text{cost-inc}_c(\{c_0\}) - \sum_{c \in \Delta_{\text{old}}^X} \text{cost-inc}_c(X), \end{aligned} \quad (31)$$

where we used that $\text{cost-inc}_c(X \cup \{c_0\}) \leq \text{cost-inc}_c(X)$ for any center c .

To analyze this, let π be a bijection from Δ_{old}^Y to Δ_{old}^X so that $\pi(c) = c$ for all $c \in \Delta_{\text{old}}^Y \cap \Delta_{\text{old}}^X$.

Claim 9. For every $c \in \Delta_{\text{old}}^Y$ we have

$$\text{cost-inc}_c(Y) \leq \text{cost-inc}_{\pi(c)}(X).$$

Proof. If $c \in \Delta_{\text{old}}^Y \cap \Delta_{\text{old}}^X$, the claim holds because $\text{cost-inc}_c(Y) \leq \text{cost-inc}_c(X)$ for every $c \in \mathcal{P}_1$. Otherwise, we have $\pi(c) \notin \mathcal{R}_1^Y$. This is because $(\Delta_{\text{old}}^X - \Delta_{\text{new}}^Y) \cap \mathcal{R}_1^Y = \emptyset$. Indeed, in the construction of Δ_{old}^X , we initialized with $\mathcal{R}_1^X \cap \Delta_{\text{new}}$ and, as Δ_{old}^Y contains $\Delta_{\text{new}} \cap \mathcal{R}_1^Y$, all elements in $\mathcal{R}_1^X \cap \Delta_{\text{new}}$ are either in Δ_{old}^Y or not in $\mathcal{R}_1^Y$. The property is then maintained by definition of the while-loop in the construction of Δ_{old}^X .

It follows that $\mathcal{R}_1^Y$ where we remove c and add $\pi(c)$ is another subset of $\mathcal{P}_1$ of cardinality $|\mathcal{R}_1|$. Hence, by the selection of $\mathcal{R}_1^Y$ (to be the collection of centers with smallest $\text{cost-inc}(\cdot)$) we have

$$\text{cost-inc}_c(Y) \leq \text{cost-inc}_{\pi(c)}(Y) \leq \text{cost-inc}_{\pi(c)}(X).$$

□

By the above claim,

$$\sum_{c \in \Delta_{\text{old}}^Y} \text{cost-inc}_c(Y) - \sum_{c \in \Delta_{\text{old}}^X} \text{cost-inc}_c(X) = \sum_{c \in \Delta_{\text{old}}^Y} (\text{cost-inc}_c(Y) - \text{cost-inc}_{\pi(c)}(X)) \leq 0$$

and thus by the above arguments (see (30) and (31))

$$\sum_{c \in \mathcal{R}_1^{X+c_0}} \text{cost-inc}_c(X \cup \{c_0\}) - \sum_{c \in \mathcal{R}_1^X} \text{cost-inc}_c(X) \leq \sum_{c \in \mathcal{R}_1^{Y+c_0}} \text{cost-inc}_c(Y \cup \{c_0\}) - \sum_{c \in \mathcal{R}_1^Y} \text{cost-inc}_c(Y).$$

We have thus verified (29), which concludes the proof that f is submodular.

Proof of Property 3. For every $c \in \mathcal{R}_1$, we have

$$\begin{aligned} \text{closedcost}_c(\emptyset) &= \min_{c' \in \Lambda} \sum_{p \in C_c^{\text{core}}} d(p, c') + \sum_{p \in C_c - C_c^{\text{core}}} d(p, \{c\} \cup \Lambda) \\ &\leq \min_{c' \in \Lambda} \left(\sum_{p \in C_c^{\text{core}}} d(p, c') + \sum_{p \in C_c - C_c^{\text{core}}} d(p, c') \right) = \min_{c' \in \Lambda} \sum_{p \in C_c} d(p, c'). \end{aligned}$$

Thus

$$\begin{aligned} g(\emptyset) &\leq \sum_{c \notin \mathcal{R}} \sum_{p \in C_c} d(p, \{c\} \cup \Lambda) + \sum_{p \in D_0} d(p, \Lambda) + \sum_{c \in \mathcal{R}_1} \text{closedcost}_c(\emptyset) \\ &\leq \sum_{c \notin \mathcal{R}} \sum_{p \in C_c} d(p, \{c\} \cup \Lambda) + \sum_{p \in D_0} d(p, \Lambda) + \sum_{c \in \mathcal{R}_1} \min_{c' \in \Lambda} \sum_{p \in C_c} d(p, c') \\ &\leq \sum_{c \notin \mathcal{R}} \sum_{p \in C_c} d(p, \{c\} \cup \Lambda) + \sum_{c \in \mathcal{R}} \min_{c' \in \Lambda} \sum_{p \in C_c} d(p, c') \stackrel{(23)}{\leq} \text{cost}(M'_D, \mu'_D). \end{aligned}$$

Proof of Property 4. Consider any $c \in \mathcal{R}_1$. If C_c is hit by $\hat{\text{OPT}}$ one has

$$\text{closedcost}_c(\hat{\text{OPT}}) = \sum_{p \in C_c} d(p, \{c\} \cup \hat{\text{OPT}} \cup \Lambda) \leq \min_{c' \in \hat{\text{OPT}}} \sum_{p \in C_c} d(p, c').$$

Otherwise

$$\begin{aligned} \text{closedcost}_c(\hat{\text{OPT}}) &= \min_{c' \in \hat{\text{OPT}} \cup \Lambda} \sum_{p \in C_c^{\text{core}}} d(p, c') + \sum_{p \in C_c - C_c^{\text{core}}} d(p, \{c\} \cup \hat{\text{OPT}} \cup \Lambda) \\ &\leq \min_{c' \in \hat{\text{OPT}}} \left(\sum_{p \in C_c^{\text{core}}} d(p, c') + \sum_{p \in C_c - C_c^{\text{core}}} d(p, c') \right) = \min_{c' \in \hat{\text{OPT}}} \sum_{p \in C_c} d(p, c'). \end{aligned}$$

Hence in both cases $\text{closedcost}_c(\hat{\text{OPT}}) \leq \min_{c' \in \hat{\text{OPT}}} \sum_{p \in C_c} d(p, c')$. Thus

$$\begin{aligned} g(\hat{\text{OPT}}) &\leq \sum_{c \notin \mathcal{R}} \sum_{p \in C_c} d(p, \{c\} \cup \hat{\text{OPT}} \cup \Lambda) + \sum_{p \in D_0} d(p, \hat{\text{OPT}} \cup \Lambda) + \sum_{c \in \mathcal{R}_1} \text{closedcost}_c(\hat{\text{OPT}}) \\ &\leq \sum_{c \notin \mathcal{R}} \sum_{p \in C_c} d(p, \{c\} \cup \hat{\text{OPT}} \cup \Lambda) + \sum_{p \in D_0} d(p, \hat{\text{OPT}} \cup \Lambda) + \sum_{c \in \mathcal{R}_1} \min_{c' \in \hat{\text{OPT}}} \sum_{p \in C_c} d(p, c') \\ &\leq \sum_{c \notin \mathcal{R}} \sum_{p \in C_c} d(p, \{c\} \cup \hat{\text{OPT}} \cup \Lambda) + \sum_{c \in \mathcal{R}} \min_{c' \in \hat{\text{OPT}}} \sum_{p \in C_c} d(p, c') \stackrel{(22)}{\leq} \text{cost}(M'_O, \mu'_O). \end{aligned}$$

Proof of Property 5. Let $X \subseteq F_B$ be independent in M_B , i.e., $X \in \mathcal{I}$. We give a polynomial-time algorithm that outputs k centers S^* whose cost is at most $(1 + 2\varepsilon)g(X) + 15\varepsilon\text{opt}$. Let $\mathcal{R}'_1$ be a subset of $\mathcal{P}_1$ of cardinality $|\mathcal{R}'_1|$ so that

$$g(X) = \sum_{c \notin \mathcal{R}_0 \cup \mathcal{R}'_1} \sum_{p \in C_c} d(p, \{c\} \cup X \cup \Lambda) + \sum_{p \in D_0} d(p, X \cup \Lambda) + \sum_{c \in \mathcal{R}'_1} \text{closedcost}_c(X)$$

By the arguments in the proof of the first property, we can calculate $\mathcal{R}'_1$ in polynomial time. Now we obtain S^* from $S_{Q \cup \tilde{U}}$ as follows:

- Initialize $S^* = S_{Q \cup \tilde{U}} = S \cup \Lambda - Q - \tilde{U}$.
- Remove centers $\mathcal{R}_0$ and $\mathcal{R}'_1$ from S^* to obtain a set of cardinality k .
- Finally, for each ball in $\mathcal{B}$, if X contains a center c in that ball, replace the associated dummy center by c ; otherwise, replace the dummy center with an arbitrary center in the ball.

By the definition of the dummy centers Λ , the cost of S^* is at most

$$\sum_{c \notin \mathcal{R}_0 \cup \mathcal{R}'_1} \sum_{p \in C_c} d(p, \{c\} \cup X \cup \Lambda) + \sum_{p \in D_0} d(p, X \cup \Lambda) + \sum_{c \in \mathcal{R}'_1} \sum_{p \in C_c} d(p, X \cup \Lambda).$$

To relate this to $g(X)$ we need to compare $\text{closedcost}_c(X)$ to $\sum_{p \in C_c} d(p, X \cup \Lambda)$ for every $c \in \mathcal{R}'_1$. Recall that

$$\text{closedcost}_c(X) = \begin{cases} \sum_{p \in C_c} d(p, \{c\} \cup X \cup \Lambda) & \text{if } C_c \text{ is hit by } X, \\ \min_{c' \in X \cup \Lambda} \sum_{p \in C_c^{\text{core}}} d(p, c') + \sum_{p \in C_c - C_c^{\text{core}}} d(p, \{c\} \cup X \cup \Lambda) & \text{otherwise.} \end{cases}$$

Consider a cluster C_c with $c \in \mathcal{R}'_1$ and partition $C_c - C_c^{\text{core}}$ into sets A and B where A contain those clients p so that $d(p, \{c\} \cup X \cup \Lambda) = d(p, c)$ and B contain the remaining clients p with $d(p, \{c\} \cup X \cup \Lambda) = d(p, X \cup \Lambda)$. Suppose first that there is a client $p_0 \in C_c^{\text{core}}$ so that $d(p_0, X \cup \Lambda) < d(p_0, c)$, i.e., C_c is hit by $X \cup \Lambda$. By the definition of C_c^{core} , we have $d(p_0, c) \leq \varepsilon \cdot \frac{\text{cost}(S)}{|\mathcal{R}| \cdot |C_c|} \leq 5\varepsilon \cdot \frac{\text{opt}}{|\mathcal{R}| \cdot |C_c|}$ where we used that $\text{cost}(S) \leq 5 \cdot \text{opt}$ by Lemma 4.2. So, by the triangle inequality,

$$\begin{aligned} \sum_{p \in C_c} d(p, X \cup \Lambda) &\leq \sum_{p \in C_c - B} (d(p, c) + d(c, p_0) + d(p_0, X \cup \Lambda)) + \sum_{p \in B} d(p, X \cup \Lambda) \\ &\leq \sum_{p \in A} d(p, c) + \sum_{p \in B} d(p, X \cup \Lambda) + 15\varepsilon \cdot \frac{\text{opt}}{|\mathcal{R}|}, \end{aligned}$$

where we used $d(p_0, X \cup \Lambda) \leq d(c, p_0) \leq 5\varepsilon \cdot \frac{\text{opt}}{|\mathcal{R}| \cdot |C_c|}$ and $d(p, c) \leq 5\varepsilon \cdot \frac{\text{opt}}{|\mathcal{R}| \cdot |C_c|}$ for any $p \in C_c^{\text{core}}$ for the last inequality. Moreover, by definition of A and B , we have $d(p, c) = d(p, \{c\} \cup X \cup \Lambda)$ for any $p \in A$ and $d(p, X \cup \Lambda) = d(p, \{c\} \cup X \cup \Lambda)$ for any $p \in B$. Hence, in this case,

$$\sum_{p \in C_c} d(p, X \cup \Lambda) \leq \sum_{p \in C_c - C_c^{\text{core}}} d(p, \{c\} \cup X \cup \Lambda) + 15\varepsilon \cdot \frac{\text{opt}}{|\mathcal{R}|} \leq \text{closedcost}_c(X) + 15\varepsilon \cdot \frac{\text{opt}}{|\mathcal{R}|}.$$

Let us now consider a cluster C_c with $c \in \mathcal{R}'_1$ that is not hit by $X \cup \Lambda$, particularly not by X . Then

$$\begin{aligned} \text{closedcost}_c(X) &= \min_{c' \in X \cup \Lambda} \sum_{p \in C_c^{\text{core}}} d(p, c') + \sum_{p \in C_c - C_c^{\text{core}}} d(p, \{c\} \cup X \cup \Lambda) \\ &\geq \sum_{p \in C_c^{\text{core}} \cup B} d(p, X \cup \Lambda) + \sum_{q \in A} d(q, c) \end{aligned}$$

We further have

$$\begin{aligned} \sum_{p \in C_c} d(p, X \cup \Lambda) &= \sum_{p \in C_c^{\text{core}} \cup B} d(p, X \cup \Lambda) + \sum_{q \in A} d(q, X \cup \Lambda) \\ &\leq \sum_{p \in C_c^{\text{core}} \cup B} d(p, X \cup \Lambda) + \sum_{q \in A} \frac{1}{|C_c^{\text{core}}|} \sum_{p \in C_c^{\text{core}}} (d(q, c) + d(c, p) + d(p, X \cup \Lambda)) \end{aligned}$$

by the triangle inequality. Now using that $d(c, p) \leq 5\varepsilon \cdot \frac{\text{opt}}{|\mathcal{R}| \cdot |C_c|}$ we obtain the upper bound

$$\begin{aligned} \sum_{p \in C_c} d(p, X \cup \Lambda) &\leq \left(1 + \frac{|A|}{|C_c^{\text{core}}|}\right) \sum_{p \in C_c^{\text{core}} \cup B} d(p, X \cup \Lambda) + \sum_{q \in A} d(q, c) + 5\varepsilon \frac{\text{opt}}{|\mathcal{R}|} \\ &\leq \left(1 + \frac{\varepsilon}{(1 - \varepsilon)}\right) \sum_{p \in C_c^{\text{core}} \cup B} d(p, X \cup \Lambda) + \sum_{q \in A} d(q, c) + 5\varepsilon \frac{\text{opt}}{|\mathcal{R}|} \\ &\leq (1 + 2\varepsilon) \text{closedcost}_c(X) + 5\varepsilon \frac{\text{opt}}{|\mathcal{R}|} \end{aligned}$$

where for the second inequality we used that every cluster in $\mathcal{R}'_1 \subseteq \mathcal{P}_1$ is concentrated. In more detail $|C_c^{\text{core}}| \geq (1 - \varepsilon)|C_c|$, hence $|A| \leq |C_c - C_c^{\text{core}}| \leq \varepsilon|C_c|$.

Hence, for any $c \in \mathcal{R}'_1$ we proved that $\sum_{p \in C_c} d(p, X \cup \Lambda) \leq (1 + 2\varepsilon) \text{closedcost}_c(X) + 15\varepsilon \frac{\text{opt}}{|\mathcal{R}|}$. We thus get that the cost of S^* is at most

$$\begin{aligned} &\sum_{c \notin \mathcal{R}_0 \cup \mathcal{R}'_1} \sum_{p \in C_c} d(p, \{c\} \cup X \cup \Lambda) + \sum_{p \in D_0} d(p, X \cup \Lambda) + \sum_{c \in \mathcal{R}'_1} \left((1 + 2\varepsilon) \text{closedcost}_c(X) + 15\varepsilon \frac{\text{opt}}{|\mathcal{R}|} \right) \\ &\leq (1 + 2\varepsilon) \left(\sum_{c \notin \mathcal{R}_0 \cup \mathcal{R}'_1} \sum_{p \in C_c} d(p, \{c\} \cup X \cup \Lambda) + \sum_{p \in D_0} d(p, X \cup \Lambda) + \sum_{c \in \mathcal{R}'_1} \text{closedcost}_c(X) \right) + 15\varepsilon \text{opt} \end{aligned}$$

which equals $(1 + 2\varepsilon)g(X) + 15\varepsilon \text{opt}$.

4.7 Putting Everything Together – Proof of Theorem 1.3

We now turn to proving Theorem 1.3. We first describe the algorithm we analyse.

A $(2 + \varepsilon)$ -Approximation for k -Median on an $\varepsilon / \log n$ -Stable Instance k, D, F, d .

- | | |
|---|---------------|
| 1: $S \leftarrow \text{localSearch}()$ | ▷ Section 4.1 |
| 2: $W \leftarrow D\text{-Sample}(S, s^*)$ | ▷ Section 4.2 |
| 3: $\mathcal{L}_{\text{ball}} \leftarrow \text{ballGuess}(S, W)$ | ▷ Section 4.3 |
| 4: for all $\mathcal{B} \in \mathcal{L}_{\text{ball}}$ do | |
| 5: Compute the dummy centers Λ according to $\mathcal{B}$ | |
| 6: $\mathcal{L}_{\text{exp}} \leftarrow \text{epxRem}(S, \Lambda)$ | ▷ Section 4.4 |

```

7:   for all  $\mathcal{Q} \in \mathcal{L}_{\text{exp}}$  and all valid values of  $|\mathcal{U}|$ ,  $|\mathcal{R}|$  and  $|\mathcal{X}|$  do
8:      $\mathcal{L}_{\text{cheap}} \leftarrow \emptyset$ 
9:      $\text{cheapRem}(\emptyset, \emptyset, \emptyset, \emptyset, \emptyset)$  ▷ Section 4.5
10:    for all  $\tilde{\mathcal{U}} \in \mathcal{L}_{\text{cheap}}$  with the associated  $\tilde{\mu}$  do
11:      Approximately solve the submodular maximization problem subject to a partition
      matroid induced by  $(S, W, \mathcal{B}, \mathcal{Q}, \tilde{\mathcal{U}}, \tilde{\mu})$  and turn the obtained solution into a  $k$ -median solution
       $S_{S, W, \mathcal{B}, \mathcal{Q}, \tilde{\mathcal{U}}, \tilde{\mu}}$  ▷ Section 4.6
12: return the cheapest solution of type  $S_{S, W, \mathcal{B}, \mathcal{Q}, \tilde{\mathcal{U}}, \tilde{\mu}}$ 

```

Proof of Theorem 1.3. Our approach consists of running the above algorithm $\log n$ times and taking the minimum cost solution output. In the remaining, we argue that the above algorithm yields a $2 + O(\varepsilon)$ -approximation with probability at least $(1 - \varepsilon)(1 - 1/n)$. The claim then follows as the probability that all $\log n$ executions of the above algorithm would fail is at most $(1 - (1 - \varepsilon)(1 - 1/n))^{\log n} < 1/n$, and so with high probability we output a $2 + O(\varepsilon)$ -approximation.

We first discuss the algorithm's running time. The local search algorithm runs in polynomial time. The submodular optimization step also runs in polynomial time by Lemma 4.14. The output of $D\text{-Sample}()$ (which is clearly a polynomial time procedure) is a subset of size s^* . By Lemma 4.11, we construct $\mathcal{L}_{\text{ball}}$ in polynomial time and $|\mathcal{L}_{\text{ball}}| \leq n^{1/\varepsilon^{O(1)}}$. Similarly, by Lemma 4.11, we construct $\mathcal{L}_{\text{exp}}$ in polynomial time and $|\mathcal{L}_{\text{exp}}| \leq n^{1/\varepsilon^{O(1)}}$. Finally, by Lemma 4.13, we construct in polynomial-time $n^{1/\varepsilon^{O(1)}}$ candidate pairs $\tilde{\mathcal{U}}, \tilde{\mu}$. It follows that we solve $n^{1/\varepsilon^{O(1)}}$ submodular function optimization problems, each in polynomial time, and thus the total running time is polynomial.

We turn to proving the approximation guarantee. We aim to show that there exists a $(W, \mathcal{B}, \mathcal{Q}, \tilde{\mathcal{U}}, \tilde{\mu})$ that is successful with probability at least $(1 - \varepsilon)(1 - 1/n)$. Assuming this, the $(2 + O(\varepsilon))$ -approximation follows by combining Claim 7 and Lemma 4.14.

The local search step provides a 5 approximation S that satisfies Lemma 4.4. Our algorithm next applies $D\text{-Sample}()$ and by Lemma 4.5 finds a successful W with probability at least $1 - \varepsilon$. Condition on having a successful W , we then have that Lemma 4.6 holds, and so does Lemma 4.7 by combining Lemma 4.4 and Lemma 4.6.

Next, Lemma 4.8 implies that one set of balls $\mathcal{B}$ is a valid set of balls. This set of balls induces a set of dummy centers Λ . From there, by Lemma 4.11 we have that with probability at least $1 - 1/n$, the $\text{expRem}()$ procedure produces a set of center $\mathcal{Q}$ such that

1. $\mathcal{Q} \subseteq S_0$, and
2. The total cost in solution $S - \mathcal{Q} \cup \Lambda$ of the clusters in $S_0 - \mathcal{Q}$ is at most $\varepsilon \cdot \text{opt}$.

and so we have obtained a successful $(W, \mathcal{B}, \mathcal{Q})$ and Lemma 4.12 applies. Condition on the event that we obtain a successful $(W, \mathcal{B}, \mathcal{Q})$, then, applying Lemma 4.13 we have that the $\text{cheapRem}()$ procedure outputs a pair $(\tilde{\mathcal{U}}, \tilde{\mu})$ satisfying the properties of Lemma 4.13. We thus have a $(W, \mathcal{B}, \mathcal{Q}, \tilde{\mathcal{U}}, \tilde{\mu})$ such that W is successful, the set of balls $\mathcal{B}$ from Section 4.3 is valid, $\mathcal{Q}$ selected in Section 4.4 satisfies the properties of Lemma 4.11, and $\tilde{\mathcal{U}}, \tilde{\mu}$ selected in this section satisfies the properties of Lemma 4.13 and so $(W, \mathcal{B}, \mathcal{Q}, \tilde{\mathcal{U}}, \tilde{\mu})$ is successful. Finally, note that the only probabilistic steps were the sampling of W and of $\mathcal{Q}$, and both steps are successful with probability at least $(1 - \varepsilon) \cdot (1 - 1/n)$, as required. $\square$

A Analysis of Robust LOGADAPTIVE

In this section, we prove Theorem 3.10, showing that a solution $\mathcal{H}$ of η -valid sequences yields an almost 2-LMP approximation. The proof is almost identical to that for LOGADAPTIVE presented in Section 3.3. There are only two minor changes:

1. Definition 3 of η -openability allows a facility to be paid up to an $n\eta$ -additive difference. This difference only appears in Lemma A.1 below, which is an analog of Lemma 3.7.

Lemma A.1. *We have $\sum_{j \in D} \alpha_j^* \geq (1 - \delta) \sum_{j \in D} d(j, S^*) + \sum_{i \in S_{\text{reg}}^*} (f_i - n\eta)$.*

2. Definition 4 of a η -valid sequence allows h_t is η -openable with respect to a super set $S' \supseteq S \cup \{h_1, \dots, h_{t-1}\}$. This difference only appears in (the last part of) the proof of Lemma A.2, an analog of Lemma 3.8.

Lemma A.2. For a facility i , $\sum_{j \in D} [\alpha_j^* - 2d(i, j)]^+ \leq \hat{f}$.

These two lemmas immediately imply Theorem 3.10. The other supporting claims and lemmas (Claim 10 and Lemma A.3) remain identical.

In order to use mostly the same analysis, one syntactic difference between the setting of Section 3.3 and the current setting that we have to reconcile is that the former analyzes the outcome of LOGADAPTIVE while the latter analyzes a solution $\mathcal{H} = \{H_1, \dots, H_L\}$ of η -valid sequences. To use the same terminology from Section 3.3, let us consider the *execution* of the solution $\mathcal{H}$ where for $p = 1, \dots, L$, we run the p -th phase according to the sequence H_p ; in Stage 1, each facility in the sequence becomes open one by one with the corresponding $(\tau_j)_j$ values, and in Stage 2 at the end of the phase, $\theta \leftarrow \theta(1 + \varepsilon^2)$.

Then an *atomic step* can still be defined in the same way as in LOGADAPTIVE; it corresponds to opening a facility in Stage 1 (and updating α, S, A accordingly) or increasing α values in Stage 2 simultaneously (and updating S accordingly). By “at any point of the algorithm”, we mean any point in the algorithm’s execution that is not in the middle of an atomic step. The set of open facilities S can be partitioned to be the set of the open free facilities S_{free} and the set of the open regular facilities S_{reg} .

Throughout the analysis, we let α_j^* be the final α -value of a client $j \in D$, and we let S^* be the set of opened facilities. (Also let S_{free}^* and S_{reg}^* be the set of finally open free and regular facilities respectively.) We start by analyzing the approximation guarantee of the algorithm with respect to α^* , and we then prove that $\alpha^*/2$ is a feasible solution to the dual.

A.1 Approximation Guarantee.

We prove Lemma A.1 restated below. The only difference from Lemma 3.7 is that opening a facility only requires $\hat{f} - n\eta$ instead of $\hat{f}$.

Lemma A.1. We have $\sum_{j \in D} \alpha_j^* \geq (1 - \delta) \sum_{j \in D} d(j, S^*) + \sum_{i \in S_{\text{reg}}^*} (\hat{f} - n\eta)$.

Proof of Lemma A.1. The proof is by induction on the algorithm that at any point of the algorithm

$$\sum_{j \in I} \alpha_j \geq \sum_{j \in I} (1 - \delta) d(j, S) + \sum_{i \in S_{\text{reg}}} (\hat{f} - n\eta). \quad (32)$$

The equality is initially true since $A = D$ (thus $I = \emptyset$) and $S = \emptyset$. We now analyze each of the two stages separately.

Stage 1. Consider what happens when we open a facility h , i.e., add it to S .

If h is a free facility, the α values and S_{reg} do not change, and the only change is some client j with $\alpha_j \geq (1 - \delta)d(j, S)$ becoming inactive. The amount of the increase of the left-hand side of (32) is at least that of the right-hand side.

If h is a regular facility i , let (α, S, A, θ) be the state right before opening i , A' be the clients that are removed from A by the opening, which means $A' = \{j \in A : \tau_j \geq (1 - \delta)d(i, j)\}$. (Recall that the opening lets $\alpha_j \leftarrow \tau_j$ for $j \in A$.) Further, let $X := \{j \in I : d(j, i) < d(j, S)\}$ be the subset of clients of I that make positive bids to i . The change of cost of the right-hand side of (32) is at most

$$(\hat{f} - n\eta) + \sum_{j \in A'} (1 - \delta)d(i, j) + \sum_{j \in X} (1 - \delta)(d(i, j) - d(j, S)).$$

Since i is paid up to η (the third bullet of Definition 3), we also have

$$\hat{f} - n\eta \leq \sum_{j \in A'} (\tau_j - (1 - \delta)d(i, j)) + \sum_{j \in X} (1 - \delta)(d(j, S) - d(i, j)).$$

We thus get that the change of cost of the right-hand-side is at most $\sum_{j \in A'} \tau_j$, which is the change of the left-hand-side.

Stage 2. No facility is open at this stage, and every client j that becomes inactive at this stage has $\alpha_j = (1 - \delta)d(j, S)$, which implies that the left-hand-side and right-hand-side both increase with the same amount when a client j becomes inactive. $\square$

A.2 Dual Feasibility

We prove Lemma A.2 for dual feasibility. The following claim is identical to Claim 1 without any modification.

Claim 10 (No Over-Bidding). *At any point of the algorithm, for every facility i ,*

$$\sum_{j \in A} [\alpha_j - d(i, j)]^+ + \sum_{j \in I} [(1 - \delta)d(j, S) - d(j, i)]^+ \leq \hat{f}. \quad (33)$$

Proof. In the beginning, it is true because every client is active and $\alpha_j = 1 \leq d(i, j)$ for every $j \in D$. Let us prove that each step of the algorithm maintains this invariant. We first consider the steps that happen during the first stage and then those during the second stage.

Stage 1. Suppose that facility h' is open with the bids $(\tau_j)_{j \in A}$. It does (1) potentially increase the α -values of clients in $B(h', \varepsilon\theta) \cap A$, and (2) make clients in $A \cap B(h', \theta/(1 - \delta))$ inactive, which includes $B(h', \varepsilon\theta) \cap A$ considered in (1).

Consider any facility i and see how (33) is impacted by this change. First, observe that a client j that was inactive before the opening of h' does not increase its contribution to (33). This is because $d(j, S)$ monotonically decreases in S . We now turn our attention to the more interesting case. For a client j becoming inactive from active, the contribution to (33) changes from $[\alpha_j - d(i, j)]^+$ (where α -values are measured right before the opening) to $[(1 - \delta)d(j, S) - d(j, i)]^+$ (where S includes h').

If α_j does not change by the opening of h' , the fact that j becomes inactive means $(1 - \delta)d(j, S) \leq \alpha_j$, so j 's contribution to (33) cannot increase. If α_j does strictly increase by the opening of h' , it means that $d(j, S) \leq d(j, h') \leq \varepsilon\theta$ after the opening, so the contribution to (33) decreases by at least $\theta/2$ or becomes 0.

We have thus proved that no client increases their contribution to (33) during the steps of the first stage, and so these steps maintain the inequality.

Stage 2. Assume towards contradiction that at the end of the phase, if we increase α_j of every $j \in A$ to $\min((1 + \varepsilon^2)\theta, (1 - \delta)d(j, S))$, the claim is violated for a non-empty subset of facilities. Since any free facility is only farther away in the metric (depending on $u(\cdot)$) we thus must have that the claim is violated by a set of regular facilities. Let F' be that set. We select a “minimal” such counter example in the following way: let $\tau' \leq (1 + \varepsilon^2)\theta$ be the smallest value so that $\alpha'_j := \min(\tau', (1 - \delta)d(j, S))$ satisfies

$$\sum_{j \in A} [\alpha'_j - d(i, j)]^+ + \sum_{j \in I} [(1 - \delta)d(j, S) - d(j, i)]^+ = \hat{f},$$

for some $i \in F'$. We remark that $\tau' \geq \theta$ since (33) is satisfied for all facilities after the first stage by the previous arguments. Furthermore, $i \notin S$ is not yet open since if it was open then there would be no client $j \in A$ for which $\alpha'_j - d(i, j)$ is strictly positive. Hence the increase in α -values cannot cause i to violate (33) if it were already opened, which would contradict $i \in F'$.

We now show that this i with $\tau_j = \alpha'_j$ for $j \in A \cap B(i, \varepsilon\theta)$ and $\tau_j = \alpha_j$ for $j \in A \setminus B(i, \varepsilon\theta)$ satisfies the conditions of Definition 3 for i with respect to S . In other words, i is an η -openable facility with respect to S (that was not yet opened), which contradicts the *maximality* in Definition 4 for a η -valid sequence.

We verify the conditions of Definition 3 one-by-one. The first two bullets are satisfied by the

definition of the bids $(\tau_j)_{j \in A}$. For the third bullet,

$$\begin{aligned}
& \sum_{j \in A} [\tau_j - (1 - \delta)d(i, j)]^+ + (1 - \delta) \sum_{j \in I} [d(j, S) - d(i, j)]^+ \\
&= \sum_{j \in A \cap B(i, \varepsilon\theta)} [\alpha'_j - (1 - \delta)d(i, j)]^+ + \sum_{j \in A \setminus B(i, \varepsilon\theta)} [\alpha_j - (1 - \delta)d(i, j)]^+ + (1 - \delta) \sum_{j \in I} [d(j, S) - d(i, j)]^+ \\
&\geq \sum_{j \in A \cap B(i, \varepsilon\theta)} [\alpha'_j - d(i, j)]^+ + \sum_{j \in A \setminus B(i, \varepsilon\theta)} [\alpha'_j - d(i, j)]^+ + \sum_{j \in I} [(1 - \delta)d(j, S) - d(i, j)]^+.
\end{aligned}$$

For the second summation we used that $\alpha_j - (1 - \delta)d(i, j) \geq (1 + \varepsilon^2)\alpha_j - d(i, j) \geq \alpha'_j - d(i, j)$ when $d(i, j) \geq \varepsilon\theta$ (recall that $\alpha_j = \theta$ for all $j \in A$).

It remains to verify the fourth bullet of Definition 3. Suppose toward contradiction that there is a regular facility i_0 and $k \in A$,

$$\sum_{j \in A} [\tau_j - d(i_0, j)]^+ + \sum_{j \in I} [\tau_k - d(i_0, k) - 2d(i_0, j)]^+ > \hat{f}.$$

We would like to show

$$\begin{aligned}
& \sum_{j \in A} [\tau_j - d(i_0, j)]^+ + \sum_{j \in I} [(1 - \delta)d(j, S) - d(i_0, j)]^+ \\
&\geq \sum_{j \in A} [\tau_j - d(i_0, j)]^+ + \sum_{j \in I} [\tau_k - d(i_0, k) - 2d(i_0, j)]^+
\end{aligned}$$

which concludes the proof because then i_0 is violated with a smaller value of τ' which contradicts the minimal selection of τ' . To prove the above inequality, note that $k \in A$ implies $\tau_k \leq (1 - \delta)d(k, S)$ and for every $j \in I$,

$$(1 - \delta)d(k, S) \leq (1 - \delta)(d(k, i_0) + d(i_0, j) + d(j, S)),$$

and so

$$\tau_k - d(i_0, k) - 2d(i_0, j) \leq (1 - \delta)d(j, S) - d(i_0, j).$$

□

As usual, fix an arbitrary regular facility $i \in F$. The setup in this paragraph is also identical to Section 3.3. Let $D^* = \{j \in D : \alpha_j^* > 2d(i, j)\}$ be those clients that contribute to the left-hand-side. Further, order the clients in D^* according to α^* values by the time they are removed from A (in the algorithm) and break ties according to α^* values (in increasing order). We will use j, k to denote clients in D^* , and let us slightly abuse notation and use $j \leq k$ with respect to this ordering. In other words, we have $j \leq k$ if j was removed from A in an atomic step before k was removed, j and k was removed during the same atomic step and $\alpha_j^* \leq \alpha_k^*$. We also use the terminology that a facility i *freezes* i_0 if i becomes open when i_0 is not, and $d(i, i_0) \leq (1 + 3\varepsilon)\theta/2$. (Of course, i freezes itself when it is open.)

The following lemma and the proof are also identical to Lemma 3.9.

Lemma A.3. *For any $k \in D^*$ that becomes inactive strictly before i becomes frozen,*

$$\sum_{j \in D^*: j \geq k} [\alpha_k^* - d(i, j)]^+ + \sum_{j \in D^*: j < k} [\alpha_k^* - 2d(i, j) - d(i, k)]^+ \leq \hat{f}. \quad (34)$$

Proof. Let us do the following case analysis depending on the stage k becomes inactive.

Stage 1. k becomes inactive because of the opening of some h_0 . By the statement of the lemma, it suffices to handle the case that h_0 does not freeze i . So $d(i, h_0) > (1 + 3\varepsilon)\theta/2$. The opening of h_0 might have strictly increased the α -value of the clients in $B(h_0, \varepsilon\theta)$, but no such client j will be in D^* , since they immediately become inactive while

$$d(i, j) \geq d(i, h_0) - d(h_0, j) > ((1 + 3\varepsilon)/2 - \varepsilon)\theta \geq (1 + \varepsilon^2)\theta/2 \geq \alpha_j^*/2.$$

Applying this argument to every facility open in this phase (which did not freeze i), we can conclude that, right before h_0 is open, every $j \in D^*$ has $\alpha_j \leq \theta$ and all active ones have $\alpha_j = \theta$. (So, $\alpha_k^* = \theta$ as well since the opening of h_0 does not strictly increase α_k .) Claim 10, applied right before h_0 is open, ensures that

$$\sum_{j \in A} [\alpha_j - d(i, j)]^+ + \sum_{j \in I} [(1 - \delta)d(j, S) - d(i, j)]^+ \leq \hat{f}.$$

This satisfies (34) as (1) every $j \in D^*$ with $j \geq k$ is in $j \in A$ at that point, and (2) for every $j < k$ with $j \in I$,

$$\alpha_k^* < (1 - \delta)d(k, S) \leq (1 - \delta)d(j, S) + d(j, i) + d(i, k)$$

implies

$$\alpha_k^* - 2d(i, j) - d(i, k) \leq (1 - \delta)d(j, S) - d(i, j).$$

(Note that this S does not contain h_0 and k was not connected yet.)

Stage 2. Suppose that k becomes inactive by the increase of the α -values at the end of a phase. Then Claim 10 ensures that, at the end of this phase (after k becomes inactive),

$$\sum_{j \in A} [\alpha_j - d(i, j)]^+ + \sum_{j \in I} [(1 - \delta)d(j, S) - d(j, i)]^+ \leq \hat{f}. \quad (35)$$

Let us see that the left-hand-side of (35) is at least that of (34) by comparing the contribution of $j \in D^*$ to both. If $j \in A$, its contribution to the former is definitely at least that to the latter, since any active j has $\alpha_j = (1 + \varepsilon^2)\theta \geq \alpha_k^*$.

For $j \in I$, we must have

$$\alpha_k^* \leq (1 - \delta)d(k, S) \leq (1 - \delta)(d(k, j) + d(j, S)).$$

(since α_k is never increased above $(1 - \delta)d(k, S)$). and so by rearranging and using the triangle inequality ($d(k, j) \leq d(k, i) + d(i, j)$) we get

$$\alpha_k^* - 2d(i, j) - d(i, k) \leq (1 - \delta)d(j, S) - d(i, j)$$

which satisfies our goal when $j < k$. Finally, when $j \in I$ but $j \geq k$, it means that j was removed at the same time as k at the end of the phase. In that case $(1 - \delta)d(j, S) = \alpha_j^*$ and, as we break ties with α^* -values, $\alpha_j^* \geq \alpha_k^*$, which again satisfies our goal. $\square$

Equipped with the above lemma, we are ready to complete the proof of dual feasibility.

Proof of Lemma A.2. Consider the case that i is frozen by h_0 with $(\tau_j)_{j \in A}$. The following claim also remains unchanged from Section 3.3.

Claim. We have $\alpha_j^* = \tau_j$ for every $j \in D^* \cap A$.

Proof of Claim. Consider $j \in A$. First, note that if $\tau_j \geq (1 - \delta)d(j, h_0)$ then j is removed from A when h_0 is opened and so $\alpha_j^* = \tau_j$.

In the other case, when $\alpha_j \leq \tau_j < (1 - \delta)d(j, h_0)$ we show that $j \notin D^*$. Consider the (future) time right after j is removed from the active clients. The α -value of j then (which is equal to the final α_j^*) cannot be strictly greater than $(1 - \delta)d(j, h_0)$; whether it is increased in Stage 1 (as $\alpha \leftarrow \tau$) or Stage 2, since $h_0 \in S$ (where S is the set of open facilities right before j becomes inactive), the algorithm ensures that it cannot be strictly more than $(1 - \delta)d(j, S) \leq (1 - \delta)d(j, h_0)$. By the triangle inequality

$$d(j, i) \geq d(j, h_0) - d(h_0, i) = \left(1 - \frac{d(h_0, i)}{\theta} \frac{\theta}{d(j, h_0)}\right) d(j, h_0),$$

where θ is the value when h_0 was open. As h_0 freezes i , we have $d(h_0, i) \leq (1 + 3\varepsilon)\theta/2$ and, by the assumption of the case we have $\theta = \alpha_j \leq (1 - \delta)d(j, h_0)$. Plugging in those bounds to the above inequality yields,

$$d(j, i) \geq \left(1 - \frac{(1 + 3\varepsilon)}{2}(1 - \delta)\right) d(j, h_0).$$

We rewrite the above inequality (multiplying both sides by two and using that $\delta = 3\epsilon$) to obtain

$$2d(j, i) \geq (2 - (1 + 3\epsilon)(1 - \delta)) d(j, h_0) = (1 + 9\epsilon^2) d(j, h_0).$$

In other words we have $\alpha_j^* \leq d(j, h_0) < 2d(j, i)$, so j cannot be in D^* . $\square$

The following remaining part of the proof needs some changes from Section 3.3, since when h_0 is η -openable in the η -openable sequence, it might be with respect to a superset $S' \supseteq S$. (Here S denotes the set of all opened facilities right before h_0 .) Let A' the subset of A that is still active with respect to the superset S' for opening h_0 and let $I' = A \setminus A'$. As the α -values of clients in $A \cap D^*$ remain unchanged after the opening of h_0 , the fourth bullet of Definition 3 ensures that for any $k \in A' \cap D^*$,

$$\sum_{j \in A' \cap D^*} [\alpha_j^* - d(i, j)]^+ + \sum_{j \in (I \cup I') \cap D^*} [\alpha_k^* - 2d(i, j) - d(k, i)]^+ \leq \hat{f}. \quad (36)$$

Moreover, for every client $k \in D^* \cap I$ that became inactive strictly before h_0 was opened, Lemma A.3 says

$$\sum_{j \in D^*: j \geq k} [\alpha_k^* - d(i, j)]^+ + \sum_{j \in D^*: j < k} [\alpha_k^* - 2d(i, j) - d(i, k)]^+ \leq \hat{f}. \quad (37)$$

Finally for a client in $k \in D^* \cap I'$, note that $\alpha_k^* = \alpha_k = \theta$, because k was not active with respect to S' . Therefore, by Claim 10 applied right before h_0 was opened

$$\sum_{j \in A} [\alpha_k^* - d(i, j)]^+ + \sum_{j \in I} [(1 - \delta)d(j, S) - d(j, i)]^+ \leq \hat{f}.$$

which by the same argument as in the proof of Lemma 3.8,

$$\sum_{j \in A} [\alpha_k^* - d(i, j)]^+ + \sum_{j \in I} [\alpha_k^* - 2d(j, i) - d(i, k)]^+ \leq \hat{f}. \quad (38)$$

Let us consider the summation of (37) for every $k \in I \cap D^*$, (38) for every $k \in I' \cap D^*$, and (36) for every $k \in A' \cap D^*$, and consider how many times each term appears. Let $a' = |A' \cap D^*|$, $i' = |I' \cap D^*|$, $d = |D^*|$.

- $-d(i, j')$ when $j' \in I \cap D^*$: Say j' is the ℓ th client in D^* for some $\ell \leq d - i' - a'$. Then $-d(i, j')$ appears $d + (d - \ell)$ times in (36), (37), and (38) as $-d(i, j)$, and $(\ell - 1)$ times as $-d(i, k)$ in (37) when $k = j'$, so the total is $2d - 1$.
- $-d(i, j')$ when $j' \in I' \cap D^*$: It appears $d + a'$ times in (36), (37), and (38) as $-d(i, j)$, and $(d - i' - a')$ times as $-d(i, k)$ in (38) when $k = j'$, so the total is $2d - i'$.
- $-d(i, j')$ when $j' \in A' \cap D^*$: It appears d times in (36), (37), and (38) as $-d(i, j)$, and $(d - a')$ times as $-d(i, k)$ in (36) when $k = j'$, so the total is $2d - a'$.
- $\alpha_{j'}^*$ for $j' \in I \cap D^*$: d times in (37) as α_k^* when $k = j'$.
- $\alpha_{j'}^*$ for $j' \in I' \cap D^*$: d times in (38) as α_k^* when $k = j'$.
- $\alpha_{j'}^*$ for $j' \in A' \cap D^*$: $(d - a')$ times in (36) as α_k^* when $k = j'$ and once as α_j^* in (36) for each $k \in A'$, so it is still d times.

Therefore, $\sum_{j \in D^*} (\alpha_j^* - 2d(i, j)) \leq \hat{f}$, which proves the lemma in the case the opening of a facility h_0 freezes i . To complete the proof, we note that the remaining case, when no facility freezes i , corresponds to the easier situation in the above analysis when $D^* \cap A = \emptyset$, i.e., it corresponds to summing up (37) over all clients in D^* , and so $\sum_{j \in D^*} (\alpha_j^* - 2d(i, j)) \leq \hat{f}$ also in that case. $\square$

Acknowledgment

The authors are grateful to Rares-Darius Buhai for valuable feedback on the writing. In particular, for identifying the need to explicitly write Lemma 3.11.

References

- [ABS10] Pranjali Awasthi, Avrim Blum, and Or Sheffet. Stability yields a PTAS for k -median and k -means clustering. In *51th Annual IEEE Symposium on Foundations of Computer Science, FOCS 2010, October 23-26, 2010, Las Vegas, Nevada, USA*, pages 309–318, 2010.
- [ACKS15] Pranjali Awasthi, Moses Charikar, Ravishankar Krishnaswamy, and Ali Kemal Sinop. The hardness of approximation of Euclidean k -means. In Lars Arge and János Pach, editors, *31st International Symposium on Computational Geometry, SoCG 2015, June 22-25, 2015, Eindhoven, The Netherlands*, volume 34 of *LIPIcs*, pages 754–767. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2015.
- [AGK⁺04] Vijay Arya, Naveen Garg, Rohit Khandekar, Adam Meyerson, Kamesh Munagala, and Vinayaka Pandit. Local search heuristics for k -median and facility location problems. *SIAM J. Comput.*, 33(3):544–562, 2004.
- [ANSW20] Sara Ahmadian, Ashkan Norouzi-Fard, Ola Svensson, and Justin Ward. Better guarantees for k -means and Euclidean k -median by primal-dual algorithms. *SIAM J. Comput.*, 49(4), 2020.
- [ARR98] Sanjeev Arora, Prabhakar Raghavan, and Satish Rao. Approximation schemes for Euclidean k -medians and related problems. In *Proceedings of the Thirtieth Annual ACM Symposium on the Theory of Computing, Dallas, Texas, USA, May 23-26, 1998*, pages 106–113, 1998.
- [ARS03] Aaron Archer, Ranjithkumar Rajagopalan, and David B. Shmoys. Lagrangian relaxation for the k -median problem: New insights and continuity properties. In Giuseppe Di Battista and Uri Zwick, editors, *Algorithms - ESA 2003, 11th Annual European Symposium, Budapest, Hungary, September 16-19, 2003, Proceedings*, volume 2832 of *Lecture Notes in Computer Science*, pages 31–42. Springer, 2003.
- [AV07] David Arthur and Sergei Vassilvitskii. k -means++: the advantages of careful seeding. In *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2007, New Orleans, Louisiana, USA, January 7-9, 2007*, pages 1027–1035, 2007.
- [BA10] Jaroslav Byrka and Karen Aardal. An optimal bifactor approximation algorithm for the metric uncapacitated facility location problem. *SIAM J. Comput.*, 39(6):2212–2231, 2010.
- [Bar96] Yair Bartal. Probabilistic approximations of metric spaces and its algorithmic applications. In *37th Annual Symposium on Foundations of Computer Science, FOCS '96, Burlington, Vermont, USA, 14-16 October, 1996*, pages 184–193. IEEE Computer Society, 1996.
- [Bar98] Yair Bartal. On approximating arbitrary metrics by tree metrics. In Jeffrey Scott Vitter, editor, *Proceedings of the Thirtieth Annual ACM Symposium on the Theory of Computing, Dallas, Texas, USA, May 23-26, 1998*, pages 161–168. ACM, 1998.
- [BF24] Niv Buchbinder and Moran Feldman. Deterministic algorithm and faster algorithm for submodular maximization subject to a matroid constraint. *CoRR*, abs/2408.03583, 2024.
- [BPR⁺17] Jaroslav Byrka, Thomas W. Pensyl, Bartosz Rybicki, Aravind Srinivasan, and Khoa Trinh. An improved approximation for k -median and positive correlation in budgeted optimization. *ACM Trans. Algorithms*, 13(2):23:1–23:31, 2017.

- [CCGG98] Moses Charikar, Chandra Chekuri, Ashish Goel, and Sudipto Guha. Rounding via trees: Deterministic approximation algorithms for group Steiner trees and k -median. In Jeffrey Scott Vitter, editor, *Proceedings of the Thirtieth Annual ACM Symposium on the Theory of Computing, Dallas, Texas, USA, May 23-26, 1998*, pages 114–123. ACM, 1998.
- [CCPV11] Gruia Călinescu, Chandra Chekuri, Martin Pál, and Jan Vondrák. Maximizing a monotone submodular function subject to a matroid constraint. *SIAM J. Comput.*, 40(6):1740–1766, 2011.
- [CCSL21] Vincent Cohen-Addad, Karthik C. S., and Euiwoong Lee. On approximability of clustering problems without candidate centers. In *Proceedings of the 2021 ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 2635–2648. SIAM, 2021.
- [CEMN22] Vincent Cohen-Addad, Hossein Esfandiari, Vahab S. Mirrokni, and Shyam Narayanan. Improved approximations for Euclidean k -means and k -median, via nested quasi-independent sets. In Stefano Leonardi and Anupam Gupta, editors, *STOC '22: 54th Annual ACM SIGACT Symposium on Theory of Computing, Rome, Italy, June 20 - 24, 2022*, pages 1621–1628. ACM, 2022.
- [CFS19] Vincent Cohen-Addad, Andreas Emil Feldmann, and David Saulpic. Near-linear time approximations schemes for clustering in doubling metrics. In David Zuckerman, editor, *60th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2019, Baltimore, Maryland, USA, November 9-12, 2019*, pages 540–559. IEEE Computer Society, 2019.
- [CG05] Moses Charikar and Sudipto Guha. Improved combinatorial algorithms for facility location problems. *SIAM J. Comput.*, 34(4):803–824, 2005.
- [CGH⁺22] Vincent Cohen-Addad, Anupam Gupta, Lunjia Hu, Hoon Oh, and David Saulpic. An improved local search algorithm for k -median. In Joseph (Seffi) Naor and Niv Buchbinder, editors, *Proceedings of the 2022 ACM-SIAM Symposium on Discrete Algorithms, SODA 2022, Virtual Conference / Alexandria, VA, USA, January 9 - 12, 2022*, pages 1556–1612. SIAM, 2022.
- [CGK⁺19] Vincent Cohen-Addad, Anupam Gupta, Amit Kumar, Euiwoong Lee, and Jason Li. Tight FPT approximations for k -median and k -means. In Christel Baier, Ioannis Chatzigiannakis, Paola Flocchini, and Stefano Leonardi, editors, *46th International Colloquium on Automata, Languages, and Programming, ICALP 2019, July 9-12, 2019, Patras, Greece*, volume 132 of *LIPIcs*, pages 42:1–42:14. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2019.
- [CGLS23] Vincent Cohen-Addad, Fabrizio Grandoni, Euiwoong Lee, and Chris Schwiegelshohn. Breaching the 2 LMP approximation barrier for facility location with applications to k -median. In Nikhil Bansal and Viswanath Nagarajan, editors, *Proceedings of the 2023 ACM-SIAM Symposium on Discrete Algorithms, SODA 2023, Florence, Italy, January 22-25, 2023*, pages 940–986. SIAM, 2023.
- [CGTS99] Moses Charikar, Sudipto Guha, Éva Tardos, and David B. Shmoys. A constant-factor approximation algorithm for the k -median problem (extended abstract). In Jeffrey Scott Vitter, Lawrence L. Larmore, and Frank Thomson Leighton, editors, *Proceedings of the Thirty-First Annual ACM Symposium on Theory of Computing, May 1-4, 1999, Atlanta, Georgia, USA*, pages 1–10. ACM, 1999.
- [CGTS02] Moses Charikar, Sudipto Guha, Éva Tardos, and David B. Shmoys. A constant-factor approximation algorithm for the k -median problem. *J. Comput. Syst. Sci.*, 65(1):129–149, 2002.
- [CK19] Vincent Cohen-Addad and Karthik C. S. Inapproximability of clustering in l_p metrics. In David Zuckerman, editor, *60th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2019, Baltimore, Maryland, USA, November 9-12, 2019*, pages 519–539. IEEE Computer Society, 2019.

- [CL12] Moses Charikar and Shi Li. A dependent lp-rounding approach for the k-median problem. In Artur Czumaj, Kurt Mehlhorn, Andrew M. Pitts, and Roger Wattenhofer, editors, *Automata, Languages, and Programming - 39th International Colloquium, ICALP 2012, Warwick, UK, July 9-13, 2012, Proceedings, Part I*, volume 7391 of *Lecture Notes in Computer Science*, pages 194–205. Springer, 2012.
- [CS03] Fabián A. Chudak and David B. Shmoys. Improved approximation algorithms for the uncapacitated facility location problem. *SIAM J. Comput.*, 33(1):1–25, 2003.
- [CS17] Vincent Cohen-Addad and Chris Schwiegelshohn. On the local structure of stable clustering instances. In Chris Umans, editor, *58th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2017, Berkeley, CA, USA, October 15-17, 2017*, pages 49–60. IEEE Computer Society, 2017.
- [FRS19] Zachary Friggstad, Mohsen Rezapour, and Mohammad R. Salavatipour. Local search yields a PTAS for k-means in doubling metrics. *SIAM J. Comput.*, 48(2):452–480, 2019.
- [FW14] Yuval Filmus and Justin Ward. Monotone submodular maximization over a matroid via non-oblivious local search. *SIAM J. Comput.*, 43(2):514–542, 2014.
- [GI03] Venkatesan Guruswami and Piotr Indyk. Embeddings and non-approximability of geometric problems. In *Proceedings of the Fourteenth Annual ACM-SIAM Symposium on Discrete Algorithms, January 12-14, 2003, Baltimore, Maryland, USA.*, pages 537–538, 2003.
- [GK99] Sudipto Guha and Samir Khuller. Greedy strikes back: Improved facility location algorithms. *J. Algorithms*, 31(1):228–248, 1999.
- [Gon85] Teofilo F. Gonzalez. Clustering to minimize the maximum intercluster distance. *Theor. Comput. Sci.*, 38:293–306, 1985.
- [GOR⁺22] Fabrizio Grandoni, Rafail Ostrovsky, Yuval Rabani, Leonard J. Schulman, and Rakesh Venkat. A refined approximation for euclidean k-means. *Inf. Process. Lett.*, 176:106251, 2022.
- [GPST23] Kishen N. Gowda, Thomas W. Pensyl, Aravind Srinivasan, and Khoa Trinh. Improved bi-point rounding algorithms and a golden barrier for k -median. In Nikhil Bansal and Viswanath Nagarajan, editors, *Proceedings of the 2023 ACM-SIAM Symposium on Discrete Algorithms, SODA 2023, Florence, Italy, January 22-25, 2023*, pages 987–1011. SIAM, 2023.
- [GT08] Anupam Gupta and Kanat Tangwongsan. Simpler analyses of local search algorithms for facility location. *CoRR*, abs/0809.2554, 2008.
- [HS86] Dorit S. Hochbaum and David B. Shmoys. A unified approach to approximation algorithms for bottleneck problems. *J. ACM*, 33(3):533–550, 1986.
- [JMM⁺03] Kamal Jain, Mohammad Mahdian, Evangelos Markakis, Amin Saberi, and Vijay V. Vazirani. Greedy facility location algorithms analyzed using dual fitting with factor-revealing LP. *J. ACM*, 50(6):795–824, 2003.
- [JMS02] Kamal Jain, Mohammad Mahdian, and Amin Saberi. A new greedy approach for facility location problems. In *Proceedings on 34th Annual ACM Symposium on Theory of Computing, May 19-21, 2002, Montréal, Québec, Canada*, pages 731–740, 2002.
- [JV01] K. Jain and V. Vazirani. Approximation algorithms for metric facility location and k -median problems using the primal-dual schema and Lagrangian relaxation. *J. ACM*, 48(2):274–296, 2001.
- [KMN⁺04] Tapas Kanungo, David M. Mount, Nathan S. Netanyahu, Christine D. Piatko, Ruth Silverman, and Angela Y. Wu. A local search approximation algorithm for k-means clustering. *Comput. Geom.*, 28(2-3):89–112, 2004.
- [KPR00] Madhukar R. Korupolu, C. Greg Plaxton, and Rajmohan Rajaraman. Analysis of a local search heuristic for facility location problems. *J. Algorithms*, 37(1):146–188, 2000.

- [KSS10] Amit Kumar, Yogish Sabharwal, and Sandeep Sen. Linear-time approximation schemes for clustering problems in any dimensions. *J. ACM*, 57(2):5:1–5:32, 2010.
- [Li13] Shi Li. A 1.488 approximation algorithm for the uncapacitated facility location problem. *Inf. Comput.*, 222:45–58, 2013.
- [LS13] Shi Li and Ola Svensson. Approximating k-median via pseudo-approximation. In *Symposium on Theory of Computing Conference, STOC’13, Palo Alto, CA, USA, June 1-4, 2013*, pages 901–910, 2013.
- [LS16] Shi Li and Ola Svensson. Approximating k-median via pseudo-approximation. *SIAM J. Comput.*, 45(2):530–547, 2016.
- [LSW17] Euiwoong Lee, Melanie Schmidt, and John Wright. Improved and simplified inapproximability for k-means. *Inf. Process. Lett.*, 120:40–43, 2017.
- [MP03] Ramgopal R. Mettu and C. Greg Plaxton. The online median problem. *SIAM J. Comput.*, 32(3):816–832, 2003.
- [MYZ06] Mohammad Mahdian, Yinyu Ye, and Jiawei Zhang. Approximation algorithms for metric facility location problems. *SIAM J. Comput.*, 36(2):411–432, 2006.
- [ORSS12] Rafail Ostrovsky, Yuval Rabani, Leonard J. Schulman, and Chaitanya Swamy. The effectiveness of Lloyd-type methods for the k-means problem. *J. ACM*, 59(6):28, 2012.
- [STA97] David B. Shmoys, Éva Tardos, and Karen Aardal. Approximation algorithms for facility location problems (extended abstract). In Frank Thomson Leighton and Peter W. Shor, editors, *Proceedings of the Twenty-Ninth Annual ACM Symposium on the Theory of Computing, El Paso, Texas, USA, May 4-6, 1997*, pages 265–274. ACM, 1997.
- [WS11] David P. Williamson and David B. Shmoys. *The Design of Approximation Algorithms*. Cambridge University Press, 2011.